

claude-windows / d4cf3fd6-b5bd-4ed1-9b57-88eea69333cd

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\d4cf3fd6-b5bd-4ed1-9b57-88eea69333cd\subagents\workflows\wf\_d2c10282-601\agent-a67e3e151830df5e2.jsonl`
- SHA-256: `d7d3eb07c6251b20f67685eac2e0225457577cb70fc324eb6f40ce0d1835dd07`
- Source modified: `2026-07-04T12:03:09+00:00`
- Imported at: `2026-07-05T16:48:23+00:00`
- project: `wf\_d2c10282-601`
- session_id: `d4cf3fd6-b5bd-4ed1-9b57-88eea69333cd`

Transcript

1. user (2026-07-04T11:55:06.092Z)

You are drafting ONE section of the A13 "alpha launch review" for the KhelSutra badminton-highlight alpha
(repo Khelsutra/badminton-highlight-indexer, engine checkout at
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer; SPA/site repo at
C:/Users/avidu/Projects/khelsutra-guru/khelsutra).
Context: the A1 serving stack is LIVE on Cloud Run asia-southeast1 ? control plane service
'rally-control-plane' (rev 00007, edge-auth ON via Cloudflare Access at api.khelsutra.guru,
4-email
allowlist, maxScale=1, per-instance ephemeral SQLite state), L4 GPU Job 'rally-worker' (WASB-
only alpha:
indexing.skip_ai_handoff=true ? no Gemini in the serving config), storage gs://khelsutra-rally-
serving.
Deploy tooling: deploy/cloudrun/gen_service_yaml.py + README.md + Dockerfile; e2e evidence:
docs/COMPUTE_DECOUPLED_SERVING/runlogs/DEPLOY_REPORT_cloud-serving_a1-e2e_2026-07-04.md (READ
IT).
Tracker: docs/ALPHA_LAUNCH_READINESS.md (\$7 rows; A10/A11 ship-gate + WASB-C3 provenance are
OWNER-GATED
and still open ? factor that into anything launch-facing).
READ-ONLY: do not modify any file, do not run git writes, do not touch GCP. Windows box,
PowerShell tool.
Ground EVERY claim in a file:line or issue number you actually opened ? no invented provenance.
If you
can't verify something, mark it [unverified]. Your output is a MARKDOWN SECTION consumed by an
orchestrator
assembling the review doc ? return it via the schema, self-contained, concise, evidence-cited.

You are the COMPLETENESS CRITIC for the assembled A13 launch review sections below. Ask: what is
missing
that an owner needs for a GO/NO-GO? What contradicts the repo/runlog evidence? Check each
section's
[unverified] claims where cheap. Do NOT rewrite the sections ? list concrete missing
items/contradictions.

DRAFT runbook

Operator runbook ? live A1 alpha (Cloud Run asia-southeast1)

> Scope note: this is an **operations** runbook for the stack that is already live. It does not
imply launch GO ? A10 (ship-gate target) and A11 (WASB-C3 provenance) are still **GATED** owner
rows (`docs/ALPHA\_LAUNCH\_READINESS.md:109-110`).

Live topology (e2e runlog `docs/COMPUTE_DECOUPLED_SERVING/runlogs/DEPLOY_REPORT_cloud-
serving_a1-e2e_2026-07-04.md:15-21,95-104`): project `gen-lang-client-0306026826`, region `asia-

```
southeast1`; control-plane **Service** `rally-control-plane` rev 00007 (edge-auth ON,
`maxScale=1`, image digest `31475f27?` = master `3c97117`); L4 GPU **Job** `rally-worker`
(nvidia-l4, 8 vCPU/32Gi, GCS-FUSE `khelsutra-rally-serving`?`/gcs`); storage `gs://khelsutra-
rally-serving`; WASB-only (`indexing.skip_ai_handoff=true` ? no Gemini key anywhere in serving,
`deploy/cloudrun/gen_service_yaml.py:55-56,30-32`). `api.khelsutra.guru` fronts the service via
a Cloud Run domain mapping behind a Cloudflare Access app (4-email allowlist).
```

1. Deploy / redeploy the control plane

Canonical path is the **committed manifest generator** (`deploy/cloudrun/README.md:72-79`;
generator defaults bake project/region/service/bucket/aud at
`deploy/cloudrun/gen\_service\_yaml.py:42-47`):

```
```powershell
python deploy/cloudrun/gen_service_yaml.py --edge-auth on -o service.yaml
gcloud run services replace service.yaml --region asia-southeast1
```
```

- `--edge-auth` is **required and explicit**; `off` is only for smoke/e2e windows (\$4 below) (`gen_service_yaml.py:124-131``).
- Run gcloud from **PowerShell**, not git-bash ? MSYS mangles `/tmp`- and `/gcs`-style values before gcloud sees them (`deploy/cloudrun/README.md:129-133``; `gen_service_yaml.py:12-13``).
- Do NOT add `CLOUD\_RUN\_JOB` to the manifest ? it's a reserved runtime env name and the manifest is rejected; the code default `rally-worker` is already correct (`gen_service_yaml.py:28-29``).

Gotcha A ? `services replace` with an unchanged manifest is a NO-OP*: it does not re-resolve `:latest` after a rebuild. Force a new revision with `gcloud run services update rally-control-plane --image asia-southeast1-docker.pkg.dev/gen-lang-client-0306026826/rally/rally-worker:latest --region asia-southeast1` (that exact fallback was used live) or stamp a revision suffix; a generator `--revision-suffix` flag is a filed follow-up candidate (e2e runlog \$5, `?a1-e2e\_2026-07-04.md:83-85`).

Gotcha B ? a redeploy WIPES per-instance state* (engine issue #471, confirmed open): the jobs/videos DB is ephemeral SQLite in the instance's `/tmp`, so any redeploy invalidates in-flight jobs, job history, and past `/api/highlights` download URLs ? the handler 404s on the DB record check (`backend/main.py:1052`` per the runlog) even though the reel mirror still exists in gs://; re-processing regenerates state (e2e runlog `?a1-e2e\_2026-07-04.md:86-89`; issue #471 body). Corollary: **keep `maxScale=1`** ? >1 instance splits the queue and `/api/jobs/{id}` polls (issue #471; encoded as the generator default, `gen_service_yaml.py:145-150``).
Operationally: redeploy only when no tester job is in flight.

2. Rebuild the image + re-pin the L4 job

One image serves both the worker Job and the control plane (post-#470 it carries the `cloud-run` + `auth` extras; `gen_service_yaml.py:136-140``).

```
```powershell
```

### 1. Rebuild (~8-10 min heavy CUDA build; `gcloud builds submit` has NO -f flag, hence

```
gcloud builds submit --config=deploy/cloudrun/cloudbuild.yaml --project gen-lang-
client-0306026826 .
```
```

```
(`deploy/cloudrun/cloudbuild.yaml:3-6,26`; `deploy/cloudrun/README.md:23-26`)
```

```
```powershell
```

### 2. Re-pin the Job to the new digest

```
gcloud run jobs update rally-worker --image asia-southeast1-docker.pkg.dev/gen-lang-
client-0306026826/rally/rally-worker:latest --region asia-southeast1
```

### 3. Re-point the service (Gotcha A: replace won't re-resolve :latest)

```
gcloud run services update rally-control-plane --image asia-southeast1-docker.pkg.dev/gen-lang-
client-0306026826/rally/rally-worker:latest --region asia-southeast1
```
```

The re-pin step is what the e2e deploy did after each build ("the Job re-pinned via `gcloud run

jobs update`", `?a1-e2e_2026-07-04.md:18`); the exact flag expansion above follows the session record ("`gcloud run jobs update rally-worker --image :latest`, memory `current-state.md:64`) [full flag string unverified against a committed doc].

- ****First execution after a push**** sits minutes in "Container image import in progress" ? visible in `gcloud run jobs executions describe <exec> --region asia-southeast1` `status.conditions`, NOT in logs; it's progress, not a hang (`deploy/cloudrun/README.md:134-137`).

3. Add / remove an alpha tester (Cloudflare Access)

Tester auth is entirely CF-side ? ****no engine redeploy needed**** (the app's aud/team-domain stay constant in the manifest; `gen\_service\_yaml.py:46-47`).

- Where: Cloudflare Zero Trust dashboard, team `khelsutra.cloudflareaccess.com` ? Access ? Applications ? *****KhelSutra Studio (alpha)***** (one self-hosted app covering both `app.khelsutra.guru` and `api.khelsutra.guru`, aud `c980da67?082b`) ? edit the Allow policy's ****Emails**** rule (session notes: memory `session-handoff.md:167-174`; app shape + email-OTP policy per `khelsutra/deploy/DEPLOY\_ALPHA.md:111-115`; the a1-cuj runlog confirms the same app/allowlist fronts the Cloud Run service, `?a1-cuj\_2026-07-04.md:80-83`).

- Current allowlist is 4 emails (enumerated in memory `session-handoff.md:169`). Free tier covers 250 users; one-time-PIN means testers need no Cloudflare account (`khelsutra/deploy/DEPLOY\_ALPHA.md:113`).

- API alternative: the scoped CF token at `G:\My Drive\cloudflare.token.txt` has Access-Apps edit (memory `session-handoff.md:170-171`).

- Remove = delete the email from the same policy. [unverified] whether an already-issued CF session survives until expiry after removal ? verify posture after removing a tester.

- Triage quirk to know: testers' uploads below ****1280x720 are rejected**** by validation (`resolution\_fps\_check`) ? a real e2e first-attempt failure, not an auth problem (`?a1-e2e\_2026-07-04.md:41-42`).

4. Edge-auth OFF/ON windows

****OFF is only for identity-token smoke/e2e tests**** (`deploy/cloudrun/README.md:80-81`; `gen\_service\_yaml.py:18-20,124-131`). Why it's dangerous: the service carries `allUsers` run.invoker so Cloudflare's proxy can reach it ? with edge-auth ON the app-level Cf-Access JWT check is what keeps the direct `\*.run.app` URL at 401 (`?a1-cuj\_2026-07-04.md:94-96`); with edge-auth OFF that last gate is gone, so keep the window short and restore immediately.

```powershell

#### open the smoke window

```
python deploy/cloudrun/gen_service_yaml.py --edge-auth off -o service.yaml
gcloud run services replace service.yaml --region asia-southeast1
```

#### drive it with an identity token during the window

```
curl -H "Authorization: Bearer $(gcloud auth print-identity-token)" https://rally-control-plane-2bosgeukpa-as.a.run.app/api/health
```

#### restore

```
python deploy/cloudrun/gen_service_yaml.py --edge-auth on -o service.yaml
gcloud run services replace service.yaml --region asia-southeast1
```
```

(commands: `deploy/cloudrun/README.md:77-79`; identity-token pattern: memory `session-handoff.md:119-120`, e2e runlog \$2. Note the replace here is NOT a no-op because the edge-auth flip changes the baked config. Remember Gotcha B: each flip wipes SQLite state.)

****Posture verification after restore**** (exactly what the e2e verified, `?a1-e2e\_2026-07-04.md:95-104`):

1. `GET /api/health` on the run.app URL ? ****200**** (deliberately exempt; it's the no-auth liveness probe, `backend/main.py:321-325`).
2. `POST /api/process` with valid JSON but no CF JWT ? ****401**** `edge auth: missing Cf-Access-Jwt-Assertion header`.
3. `curl -i https://api.khelsutra.guru/api/health` with no CF session ? ****302**** to `https://khelsutra.cloudflareaccess.com/cdn-cgi/access/login/api.khelsutra.guru??` with aud

`c980da67?082b` in the login meta.

4. Triage quirk: a ****malformed-JSON**** body gets FastAPI's 422 **before** the edge-auth dependency ? a 422 is not an auth bypass (`?a1-e2e\_2026-07-04.md:91-93`).

5. Health / capabilities checks and logs

- `GET /api/health` ? `200 {"status":"ok",?}`; `GET /api/capabilities` ? `wasb\_hybrid` present, `gemini\_key\_present=false` (correct for the WASB-only alpha), ffmpeg/ffprobe present (`?a1-e2e\_2026-07-04.md:25-27`; `?a1-cuj\_2026-07-04.md:31-33`). Both endpoints are app-level no-auth probes (`backend/main.py:321-325,359-363`) but sit behind the CF login on `api.khelsutra.guru`.
- ****Job (L4) logs****: `gcloud run jobs executions list --job=rally-worker --region=asia-southeast1`, then `gcloud beta run jobs executions logs read <execution> --region=asia-southeast1` ? the committed pattern for Cloud Run jobs in this repo (`deploy/gcp/nightly\_regression/README.md:111`, adapted from the nightly job to `rally-worker`).
- ****Cloud Logging filters**** [unverified ? no committed filter string exists; standard resource labels]:
 - service: `gcloud logging read "resource.type=cloud\_run\_revision AND resource.labels.service\_name=rally-control-plane" --project gen-lang-client-0306026826 --limit 50`
 - job: `gcloud logging read "resource.type=cloud\_run\_job AND resource.labels.job\_name=rally-worker" --project gen-lang-client-0306026826 --limit 50`
- Polling `gcloud logging read` ~every 60s and grepping `worker|wasb|rally|ERROR|Traceback|CUDA|OOM|Killed` is the established practice from the live runs (memory `feedback-tail-long-job-logs.md:52-53`).
- ****Standing owner rule****: any GPU execution expected to exceed ~20 min must be actively tailed with a ~5-min heartbeat, alerting on stall ? not just awaited (memory `feedback-tail-long-job-logs.md:18-28,49-53`).
- Dispatch completion is a ****bucket marker**** (`<out\_uri>/\_dispatch/<token>.done`), not Cloud Run API status; a failed Job writes a failure marker with the real rc so polls terminate fast (`deploy/cloudrun/README.md:109-115`).

6. Cost hygiene

- ****What runs idle****: the control plane holds `min-instances 1` + `no-cpu-throttling` ? one always-warm ~2-vCPU CPU instance ("cheap CPU; NOT the D7 warm ****GPU**** pool", `deploy/cloudrun/README.md:86,90-93`). The L4 Job is scale-to-zero and ****bills only per execution**** (`deploy/cloudrun/README.md:5`; one full CUJ ? 14 min of L4+8vCPU, `?a1-cuj\_2026-07-04.md:105`). Owner directive as of 2026-07-04: keep the stack running (`?a1-cuj\_2026-07-04.md:106`).
- ****Teardown**** (if the alpha pauses): `gcloud run services delete rally-control-plane --region asia-southeast1` and `gcloud run jobs delete rally-worker --region asia-southeast1`; the bucket/secret/Artifact-Registry repo are retained (`?a1-cuj\_2026-07-04.md:107-108`).
- ****GCP VM always-DELETE rule**** (applies to any GPU-VM diagnostics, not the Cloud Run stack): a STOPPED VM still bills its disk; `deploy/gcp/teardown.sh` defaults to `delete` (owner cost rule 2026-06-20; `deploy/gcp/README.md:248-257`, `deploy/gcp/teardown.sh:9-12`; memory `MEMORY.md:33` ? also audit `gcloud compute disks list` for orphans).
- Budget guardrail: keep the ****\$100 budget alert**** live (`deploy/gcp/README.md:18`; D17 per `deploy/cloudrun/README.md:8-9`). [unverified] whether the alert threshold was raised after the owner lifted the spend gate to \$300 pre-A1.

7. Cert-renewal watch item (issue #472, open)

`api.khelsutra.guru` = Cloud Run ****domain mapping**** (CNAME `ghs.googlehosted.com`, ****proxied**** through Cloudflare) + a Google-managed cert. Google-managed certs renew ~every 90 days, and renewal validation ****can stall behind the proxied CNAME**** (Google's challenge sees Cloudflare's edge, not the expected record) (issue #472 body).

- ****Check before ~2026-09-30****:

```
```powershell
gcloud beta run domain-mappings describe --domain api.khelsutra.guru --region asia-southeast1
```
```

and confirm the cert status shows renewal is not stalled (issue #472 body).
- ****If it stalls****: either temporarily grey-cloud (DNS-only) the CNAME so validation passes,

then re-proxy; or switch the mapping to a **Cloudflare Origin CA cert** so renewal never depends on what Google sees through the proxy (issue #472 body). CF Access (M9) is unaffected either way ? this is only the Google-side TLS cert on the mapping.

DRAFT observability

Observability & job-failure triage

Scope: the live A1 stack ? Cloud Run service `rally-control-plane` (rev 00007, edge-auth ON) + L4 Job `rally-worker`, bucket `gs://khelsutra-rally-serving` (DEPLOY_REPORT_cloud-serving_a1-e2e_2026-07-04.md:1-22). All engine paths below are relative to `badminton-highlight-indexer`.

Reading a job's state. `GET /api/jobs/{job_id}` returns `{job_id, status, progress, video_id, error, result, reports, created_at, started_at, finished_at}` (backend/main.py:1008-1027). Two failure channels, deliberately distinct (backend/api/job_worker.py:148-153):

- `status='done'` + `result.status='failed'` ? the **pipeline's verdict** (validation, staging, dispatch, cloud rc?0, stitch?) with a human `result.message`;
- `status='failed'` + `error="<ExcType>: <msg>"` ? the **worker itself crashed** (backend/api/job_worker.py:191-193).

The `progress` field is the first triage signal ? the worker stamps every stage: `starting` ? hashing ? validating ? staging upload to bucket ? dispatching (cloud_run) ? ingesting (cloud_run) / segmenting (<name>) ? finalizing; compile: `resolving` ? stitching ? uploading reel ? finished (backend/infrastructure/database_jobs.py:105-118; backend/orchestration.py:287, 324, 363, 447, 516, 559, 608, 793, 816, 851, 861). A stuck job's last `progress` value tells you which stage it died in.

Triage table

"CP logs" = `rally-control-plane` service logs (Cloud Logging, `cloud_run_revision`); "worker logs" = `rally-worker` Job execution logs (`cloud_run_job`). The whole backend logs via `stdout logging` only (backend/main.py:26-30) ? there is no metrics/tracing layer.

| Failure class | CLIENT sees | OPERATOR sees (log pattern ? where) | First debugging step |
|---------------------------------|---|---|--|
| Edge-auth reject (M9) | <code>401 {"detail":"edge auth: missing Cf-Access-Jwt-Assertion header ?"}</code> (or verification-failure detail) on process/compile/annotations/assets (backend/main.py:910-913, 1107-1110; backend/api/auth.py:134-147) Nothing app-level ? only the request-log 401 on CP. | NB: malformed JSON gets FastAPI's 422 before edge-auth runs (deploy report §5, runlogs/?a1-e2e?md:91-93) Confirm the caller came through <code>api.khelsutra.guru</code> (CF Access) not the raw run.app URL; check the 4-email allowlist in CF | Submit-time input validation <code>400</code> unsupported scheme / null byte (backend/main.py:923-933); <code>404</code> "Video file not found at '?'" (main.py:940-943); <code>400</code> "Segmenter '?' not found. Available: [?]" (main.py:953-958); upload-init <code>400</code> bad filename/extension (backend/api/uploads.py:91-99) No app log ? HTTPException raised before any logging Reproduce the request; the <code>detail</code> string is self-describing |
| Input root jail | 465 input-root jail (hosted mode) <code>400 {"detail":"hosted mode: 'gs://?' is not under the allowed input root 'gs://?'"}</code> ? a URI outside <code>storage.input_root</code> , or any URI when no uri-root is configured (backend/utils/paths.py:97-103, wired at main.py:915-933; PR #465 merged) | No app log (pre-work 400) Compare the submitted URI against <code>storage.input_root</code> in the deployed config (<code>GET /api/config</code> is redacted but shows storage roots) | In-worker video validation failure (e.g. resolution floor ?1280x720) job <code>done</code> , <code>result.status='failed'</code> , <code>message="Video failed validation: resolution_fps_check: ?"</code> + per-validator <code>results</code> (backend/orchestration.py:68-76, 520-536) CP: <code>Running validations for <path>...</code> (orchestration.py:517) then the terminal <code>[COMPUTE] video_id=? status=failed</code> line (orchestration.py:632-640) Read <code>result.results[0].details</code> (ffprobe width/height/fps are stored); the 640x360 e2e reject is a known floor (deploy report §3.2, lines 41-42) |
| Upload sequencing / size / disk | <code>404</code> unknown upload_id (uploads.py:134); <code>409</code> "Out-of-order part: expected N, got M" (uploads.py:136-139); <code>409</code> "Expected N part(s), received M" on complete (uploads.py:181-185); <code>413</code> per-part/total overflow ? upload aborted, re-init (uploads.py:157-161) or declared-size (uploads.py:100-104); <code>507</code> "Not enough free disk space for upload?" (uploads.py:109; backend/utils/freespace.py:77-83) Only the 507 logs: <code>free-space guard (upload): <path> needs X MB but only Y MB free ? refusing (507).</code> (freespace.py:70-76). | | |

409/413 log **nothing** app-side. Upload state is in-process ? an instance restart silently orphans it ? next part 404s (uploads.py:6-8) | For 409 after a gap: assume instance recycle ? client re-inits. For 507: check CP instance disk |

| Staging-to-bucket failure (#461 path) | job `done`, `result.status='failed'`, `message="cloud-serving: failed to stage the uploaded file to <input\_root>: <e>"`; `compute.path='not\_run'`, `dispatched=false` (orchestration.py:296-307, 198-221) | CP: `[API] cloud-serving: failed to stage <src> -> <staged\_uri>: <e>` with traceback (orchestration.py:297-303) | Check the CP runtime SA's write access to `storage.input\_root` and that the bucket exists; retry the job |

| Cloud dispatch failure (SDK missing / project unset / Run API error) | job `done`, `result.status='failed'`, `message="cloud dispatch error: <e>"` (orchestration.py:348-350). Distinct pre-flight RuntimeErrors: `GOOGLE\_CLOUD\_PROJECT is unset?` and `google-cloud-run is required?` (backend/compute/cloud_run.py:93-106) | CP: `[API] cloud dispatch error: <e>` with traceback (orchestration.py:349) | If pre-flight: fix CP env (`CLOUD\_RUN\_JOB`/`GOOGLE\_CLOUD\_PROJECT`) or install the SDK. Else check IAM (run.jobs.run) + `gcloud run jobs executions list --job rally-worker` |

| L4 execution failure (worker rc?0 / garbled marker) | job `done`, `result.status='failed'`, `message="cloud job failed (returncode N)."` (orchestration.py:351-352). A missing/garbled marker defaults to rc=1, never success (cloud_run.py:216-226) | CP: `cloud-run: dispatched job=rally-worker exec=<execution>; bucket-polling <done\_uri>` then `cloud-run: job=? done (video\_id=? rc=N)` (cloud_run.py:200-205, 229-231). **Worker logs** carry the real failure; the worker writes the failure marker itself so the poll fails fast (backend/worker/__main__.py:236-241) | `gcloud run jobs executions describe <execution>` then read that execution's logs; inspect `gs://khelsutra-rally-serving/outputs/<video\_id>/` and the `\_dispatch/<token>.done` marker JSON |

| Bucket-poll timeout (default budget 1800s @10s cadence ? DEFAULT_POLICY, hard-wired: backend/infrastructure/execution_policy.py:49-54; backend/compute/__init__.py:39,76) | job `done`, `result.status='failed'`, `message="cloud dispatch error: cloud-run-infer not ready within max\_wait\_sec=1800.0s [cloud-run job=rally-worker execution=<name>; cancel requested ? verify the execution state in the GCP console]"` (cloud_run.py:281-285 ? orchestration.py:348-350) ? the execution name IS in the client-visible message | CP: `cloud-run-infer: still not ready after Ns (max\_wait) ? giving up` (execution_policy.py:184-189); then either `done-marker found on the final post-timeout check ? rescuing` (cloud_run.py:261-267) or `requested cancel of execution=?` / `best-effort cancel ? failed` (cloud_run.py:270-280) | Describe the execution: if it completed just after cancel, the GPU run is lost ? re-submit; if it hung, read worker logs for the stall stage. Confirm the cancel landed (otherwise L4 bills to --task-timeout 3600s) |

| Result-ingest failure (missing/corrupt `intervals.csv`) | job `done`, `result.status='failed'`, `message="cloud-serving: could not read the result from <outputs\_uri>: <e>"`; partial rows pruned, prior segments preserved (orchestration.py:369-376, 107-117) | CP: `[API] cloud ingest failed: <e>` with traceback (orchestration.py:372) | `gsutil ls gs:///outputs/<video\_id>/` ? if `intervals.csv` is absent the worker died between marker and artifact push: read worker logs |

| Compile / ffmpeg failure | Submit-time: `404 "Indexed video record not found."` / `400` bad segments/filename/min_shot_count floor (backend/main.py:1111-1120; orchestration.py:699-745). Run-time: job `done`, `result.status='failed'`, `message="Highlight stitching failed: <e>"` (orchestration.py:829-837) | CP: `[compile] stitching failed for video\_id=: <e>` with traceback (orchestration.py:830-832). Mirror-upload failure is separate and non-fatal: `[compile] could not mirror reel to <uri>: <e>` (orchestration.py:856) | Check ffmpeg presence via `GET /api/capabilities` (main.py:384-389); a mirror-only failure means the download will hit the 32 MiB local-fallback trap below |

| Signed-URL failure (#475?#476) | On success: `307` to a v4 signed URL. On failure: silent fallback to serving the local file ? for a >32 MiB reel that resurfaces the Cloud Run response-cap edge-500 (deploy report §4, lines 61-79) | CP, post-#476 (merged): `[gcs] keyless signBlob path failed (<e>); falling back to key-based signing` or `[gcs] keyless signing skipped: runtime SA email unresolved` (backend/storage/gcs.py:104-113), then `[highlights] signed-url path failed for <video\_id>; serving local: <e>` (backend/main.py:1062-1063) | Grep CP logs for `[gcs]`; verify the SA still holds self-`roles/iam.serviceAccountTokenCreator` and `iamcredentials.googleapis.com` is enabled (the #475 root-cause pair, deploy report lines 68-73) |

| Job stuck `queued`/`waiting` | `GET /api/jobs/{id}` stays `queued` (progress null) or `waiting` indefinitely; no timeout ever flips it | Usually **no log at all** ? known O2 risk: a hung job on the drain blocks everything behind it; there is no per-job wall-clock timeout (job_worker.py:40-48). Parked jobs do log `Job <id> parked for Ns (resume when due).` (job_worker.py:189); a broken claim path logs `Failed to claim a due job: <e>` (database_jobs.py:394-396) | Check the head job's `progress`/`started\_at` ? a hung

Gemini/GPU/poll call in the head job explains everything queued behind it. A restart converts the backlog to a clean state (next row) |

| Server restart / instance recycle mid-job | ****Self-hosted (persistent DB):**** process job ? ``status='failed'`, `error="interrupted by server restart"``; compile job ? silently re-queued and finishes (database_jobs.py:420-450). ****Cloud Run:**** SQLite is per-instance ephemeral ? the new instance has a fresh DB, so ``GET /api/jobs/{id}`` ? ``404 "Unknown job_id"`` and past download URLs 404 on the DB check (main.py:1012-1014, 1052; issue #471 OPEN; deploy report lines 86-89) |

Startup log: ``[JOBS] Recovered stale jobs from a previous run ? N compile re-queued, M process failed.`` (main.py:1475-1480) ? on Cloud Run this line reports 0/0 because the DB is fresh |

Correlate the client's job_id timestamp with CP revision/instance-start events in Cloud Logging; the client must re-submit (re-process regenerates state ? outputs and reels in gs:// survive) |

| Any other worker crash | ``status='failed'`, `error="<ExcType>: <msg>"`` | CP: ``Job <job_id> crashed: <e>`` + full traceback (job_worker.py:191-193) ? the one log line guaranteed to carry the job_id | Grep CP logs for ``Job <job_id>`` |

Correlating a client `job\_id` to a Cloud Run execution

There is no single trace id ? the chain is: ****job_id ? video_id ? execution name****, stitched from the job row plus two log lines:

1. ``job_id ? video_id``: the worker stamps ``video_id`` onto the job row as soon as the result carries one (job_worker.py:177-178); it's in every ``GET /api/jobs/{id}`` response.
2. ****Dispatch token****: each dispatch mints ``done_uri = <output_root>/_dispatch/<uuid4-token>.done`` (cloud_run.py:179-182) and logs ``cloud-run: dispatched job=<job> exec=<execution>; bucket-polling <done_uri>`` (cloud_run.py:200-205) ? this CP line is the token?execution join point. The worker's side logs ``[worker] wrote completion marker -> <done_uri>`` in the ****execution's**** logs (backend/worker/__main__.py:188), closing the loop from the other end.
3. ****Provenance on the result****: a successful cloud job's ``result.compute`` = ``{path:"cloud_run", job, region, execution, outputs_uri}`` and the CP logs ``[COMPUTE] cloud_run dispatched: execution=? job=? region=? out=? video_id=?`` (orchestration.py:384-400); the docstring's own cross-check is ``gcloud run jobs executions describe <execution>`` (orchestration.py:386). Every pipeline run also ends with ``[COMPUTE] video_id=? requested=? actual=? dispatched=? status=?`` (orchestration.py:632-640).
4. On a ****poll timeout**** the execution name is carried inside the failure message itself (cloud_run.py:281-285), so it reaches the client. But on other dispatch failures the ``compute`` dict in the failed result has ****no execution field**** (orchestration.py:206-221) ? the CP ``dispatched`` log line is then the only record. Note the token/execution are ****not persisted on the job row****; if the instance recycles, this mapping survives only in Cloud Logging.

GAPS ? what an operator cannot currently see

1. ****No request tracing/replay**** ? issue ****#326**** (OPEN: "Hosted-app observability: request tracing + replay for debugging production CUJ failures"). No request id crosses the API?worker?L4 boundary.
2. ****No per-video runlog surface**** ? issue ****#328**** (OPEN: "Surface the per-video runlog / execution trace to the user (endpoint + UI), not just server logs"). ``emit_indexer_report`` writes per-video reports on every run (orchestration.py:645-660) and the paths ride the job result (main.py:1023), but they are ****server-local paths on an ephemeral instance**** ? unreachable by client or operator after recycle.
3. ****`job\_id` is absent from nearly all pipeline logs**** ? ``[COMPUTE]`, `[API] cloud-serving?`, `cloud-run: dispatched?`` key on video_id/execution, not job_id (orchestration.py:325-329, 393-400, 632-640; cloud_run.py:200-205). Only crash/park lines carry it (job_worker.py:189-193). Triage from a user's job_id starts with an extra DB/API hop that dies with the instance.
4. ****Job history is ephemeral**** (issue #471 OPEN) ? the ``error`` column and ``result`` JSON, the primary triage record, vanish on every deploy/recycle; only what was logged survives.
5. ****Stuck-queue blindness**** ? no per-job wall-clock timeout and no log when a job wedges the single drain lane (job_worker.py:40-48); the failure mode is pure silence.
6. ****Client 4xx are unlogged app-side**** ? upload 409/413, jail 400s and auth 401s raise HTTPException without a logger call (uploads.py:127-170; main.py:910-913), leaving only status codes in request logs.
7. ****No metrics/alerting**** ? zero counters; failure detection is entirely reactive log-grepping (nothing beyond stdout logging is wired in the serving backend).

****Top 3 cheapest wins**** (all log-only, no infra):

1. ****One structured terminal line per job**** in ``_run_claimed_job`` ? ``[JOB] id=? kind=? status=? video_id=? compute=? execution=? wall=?s error=?`` at `done/failed/parked` (`job_worker.py:177-193`). Single choke point; makes every triage class above greppable by the id the client actually has, and closes gap 3 for ~10 lines.
2. ****Persist/log the execution name + dispatch token at dispatch time keyed by job_id**** ? e.g. thread `job_id` into ``notify()``/the dispatch logs, or stamp ``execution`` into ``progress`` (`"dispatching (cloud_run exec=?)"`). Cost: a few lines; makes `job?execution` correlation survive crash-before-result (gap in correlation \$4).
3. ****Mirror the per-video report JSON to `gs://?/outputs/<video\_id>/`**** next to ``intervals.csv`` (the worker already pushes artifacts there ? `worker/___main__.py:327`) and/or serve it via a small authenticated endpoint ? a minimal first slice of #328 that also survives instance recycling (gaps 2 and 4).

***Launch-facing note:** these gaps are operability debt, not A13 blockers by themselves ? but with A10 (ship-gate) and A11 (WASB-C3 provenance) still OWNER-GATED (`docs/ALPHA_LAUNCH_READINESS.md:109-112`), the review should treat gaps 4+5 (silent stuck jobs on an ephemeral store) as the ones most likely to burn an alpha user before the owner gates close.

DRAFT privacy

Privacy & consent copy audit (A13)

****Verdict:**** the marketing site has a real, decent privacy policy in all 6 locales ? but the Studio SPA (the surface testers actually upload through) contains ****zero privacy/consent copy****, and its first-run wizard affirmatively tells users their videos never leave the device, which is false on the hosted alpha. One code-level guarantee (training capture hard-denied) is ***stronger*** than the copy promises; storage location, retention reality, and deletion mechanics are ***weaker***.

1. What the product says today

****Marketing site** (``khelsutra`` repo, ``/privacy``, localized in `en/kn/hi/es/da/id` ? all 6 catalogs carry the policy keys; English authoritative per ``locales/en/common.json:574``):

- Full plain-language policy, "Last updated: 22 June 2026" (``locales/en/common.json:491-577``). Key promises:
 - ****We do not**** use your footage to train or improve our models ****unless you give us clear, explicit, opt-in permission**** ? and even then, never footage involving minors." (``:523``)
 - ****We may use trusted third-party AI and cloud services**** (for example, Google's Gemini API) to detect rallies." (``:529``) ? conditional wording, so not false in the alpha, but see \$2.
 - ****We never publish**** your footage" (``:530``); processors named: Cloudflare, Google (Gemini API), email delivery (``:538-540``). ****Google Cloud Storage / Cloud Run is not named; no storage location is given.****
 - Retention: ****keep your information and footage only as long as we need it to run the alpha? then delete or anonymise it. You can ask us to delete? at any time**** (``:551``); rights include ****Delete your data and uploaded footage**** (``:558``); minors never trained/published (``:564``); ****gated behind an authenticated, invite-only sign-in**** (``:568``).
 - Access-request form: email-consent checkbox ****Okay to email me about the KhelSutra alpha.**** + ****We never publish your footage; footage involving minors is never used to improve the tool.**** with a ``/privacy`` link (``locales/en/common.json:180-181``); capture checklist: ****Please record adults with their okay.**** (``:280``).

****Studio SPA** (``khelsutra/web/src`` ? what alpha testers upload through):

- ****No privacy copy at all****: a repo-wide search of ``web/src`` for privacy/consent strings and ``/privacy`` links returns nothing (grep, 0 matches). No link to the policy, no upload-moment disclosure.
- The only pre-upload acknowledgment is about ****cost****, not data: ****I understand cloud processing may cost a little.**** (``locales/en/common.json:110-112``, rendered ``web/src/components/IngestPanel.tsx:211-224``).
- ****Contradiction:**** the first-run wizard ? shown once per browser before the ingest panel (``web/src/App.tsx:347-349``) ? unconditionally renders ****Everything runs on this device ? your videos never leave it.**** (``web/src/components/SetupWizard.tsx:58``, string at ``locales/en/common.json:612``). On the hosted alpha the cloud path uploads footage to GCS. Wizard step 1 also advertises connecting a Gemini key (``:604-608``), which the alpha build never uses.

- Upload copy is purely functional: **"Pick an .mp4 or .mov from this device ? it uploads, then KhelSutra finds the rallies."** (:101).

2. Engine reality vs. copy

| Alpha reality | Evidence | Disclosed today? |
|---|---|---|
| Uploads staged to `gs://khelesutra-rally-serving/videos/?` in **asia-southeast1** (Singapore)**;
reels mirrored to `?/highlights/?` in the same bucket |
`deploy/cloudrun/gen\_service\_yaml.py:42-45,57-62`; e2e runlog
`docs/COMPUTE\_DECOUPLED\_SERVING/runlogs/DEPLOY\_REPORT\_cloud-serving\_a1-e2e\_2026-07-04.md:37,50`
| **No.** Policy says "our own infrastructure"; never names Google Cloud storage or a region.
Cross-border (India?Singapore) transfer undisclosed |
| Reel downloads are **bearer signed URLs**, ~1 h expiry** ? anyone holding the link can fetch |
runlog `:53` (`X-Goog-Expires=3600`) | No |
| **Per-instance SQLite** ? a redeploy wipes job/reel records while the bytes **persist** in the
bucket** (orphaned data; old download URLs die) | runlog `:87-89` (#471) | No |
| **No retention automation** ? sweep tool exists (7-day default) but nothing schedules it; **no**
user-facing delete endpoint** (no DELETE routes in `backend/main.py`) |
`tools/cleanup\_caches.py:10-12`; **#301 OPEN**** (verified via `gh issue view`, 2026-07-04: "run
the upload retention sweep as an automated scheduled job (hosted alpha)") | **Over-promised.**
Policy promises deletion on request + limited retention; today fulfillment is manual operator
gsutil work |
| **Training capture HARD-DENIED**: `consent\_attested()` returns `False` for every job, and
`flywheel.enabled` defaults off ? two independent guards; claimed consent fields are ignored |
`backend/flywheel.py:44-52,144-151`; `tests/test\_flywheel.py:38-55` | Copy's "never train
without explicit opt-in" (:523) is true and **code-enforced stronger** today. A launch asset ?
say it |
| **Gemini/AI handoff DISABLED** (`indexing.skip\_ai\_handoff=true`; `gemini\_key\_present=false`
live) ? no footage reaches any third-party AI | `gen\_service\_yaml.py:56`; runlog `:25-26` |
Policy's conditional "may use Gemini" over-discloses (fine); SPA wizard still sells a Gemini
step (:604-608) ? stale for the alpha |
| Reachability = **Cloudflare Access**, same 4-email allowlist policy**, on `api.khelesutra.guru`;
edge-auth verified live (401 sans JWT, CF login 302) | `DEPLOY_REPORT_cloud-
serving_a1-cuj_2026-07-04.md:82`; e2e runlog `:99-103`; `gen_service_yaml.py:46-47,63-68` |
Matches policy's "invite-only sign-in" (:568) |
| **No content-safety/NSFW pre-flight** ? nothing screens what a tester routes into the
bucket/GPU | **#332 OPEN**** (verified via `gh issue view`, 2026-07-04) | No acceptable-use line
anywhere in tester-facing copy |
| Upload validation floor ?1280x720 (rejects smaller) | e2e runlog `:41-42` | No (expectation-
setting, minor) |

Launch-facing constraint: **A10** (ship-gate/claims) and **A11** (WASB-C3 provenance / "no commercial
model sharing" copy guardrail) are **OWNER-GATED** and open**
(`docs/ALPHA\_LAUNCH\_READINESS.md:109-110`) ? alpha copy must make no model-quality claims and no
model-sharing/commercial claims until resolved.

3. What we must tell testers (checklist)

- Where footage goes:** uploads and finished reels are stored in a private Google Cloud bucket in **Singapore** (asia-southeast1); Google Cloud must be named as a processor alongside Cloudflare.
- Who can reach it:** invite-only ? a Cloudflare Access email allowlist (currently 4 people) gates the app; operators can access stored footage.
- Download links are shareable:** reel URLs work for ~1 hour for anyone who has them ? don't forward them if the footage is sensitive.
- Retention is manual today:** footage is kept while the alpha runs; deletion is on request via hello@khelesutra.guru (automated cleanup not live ? #301); a redeploy may kill in-flight jobs and old links while the footage bytes remain until swept.
- Training is OFF, in code:** no footage is captured for model training in this build ? hard-disabled, not just policy ? and any future training will be a separate explicit opt-in, never involving minors.
- No third-party AI in this build:** nothing is sent to Gemini or any external AI (skip_ai_handoff).
- Tester responsibility:** adults-only, with their okay, badminton footage only ? there is

****no automated content screening**** yet (#332), so uploads are trusted.
8. ****Quality/limits framing:**** ?720p required; no model-quality or model-sharing claims (A10/A11 gated).
9. Ship the above in ****all 6 locales**** (or explicitly English-only with the `:574` authoritative-English caveat).

4. Draft alpha disclosure copy (landing page / pre-upload)

> ****Before you upload ? what happens to your footage.**** This alpha runs in the cloud: when you add a match video, it's uploaded to KhelSutra's private storage on Google Cloud (Singapore region) so a rented GPU can find your rallies, and your finished reel is stored there too. Access is invite-only, and we never publish or share your footage. We keep uploads and reels only while the alpha runs and delete them whenever you ask (email hello@khelsutra.guru ? automatic cleanup is still being built). Your footage is ****not**** used to train our models ? training capture is switched off in this build, in the code, and would only ever happen later with your separate, explicit opt-in (never footage involving minors).
>
> - ****Stored:**** private Google Cloud bucket, Singapore region ? uploads + reels, invite-only access.
> - ****Not sent to AI services:**** this build uses no Gemini or other third-party AI on your footage.
> - ****Not training data:**** capture is hard-disabled; future training = separate opt-in, adults only.
> - ****Download links:**** work for about 1 hour and for anyone who has them ? share with care.
> - ****Deletion:**** on request today, automatic later; a redeploy may cancel in-progress jobs and old links (just re-process).
> - ****Please upload only**** badminton footage of adults who are okay being recorded ? there's no automated screening yet, so we're trusting you.

5. Gaps, ranked

1. ****SPA wizard says "your videos never leave it"***** (`SetupWizard.tsx:58`) ? factually wrong on the hosted alpha; must be fixed or suppressed for the cloud deployment ****before any tester uploads****.
2. ****Storage processor + location undisclosed**** ? Google Cloud / Singapore appear nowhere in the policy (`:538-540` names only Cloudflare/Gemini/email); India?Singapore transfer is a counsel-review item.
3. ****Retention/deletion over-promise**** ? policy promises timely deletion (`:551,:558`) but there is no automation (#301), no delete endpoint, and red deploys orphan bucket data; either land minimum automation or soften/date the promise.
4. ****No privacy surface in the Studio SPA**** ? no policy link, no upload-moment disclosure; the only checkbox is about cost (`:110-112`). Add the \$4 copy + link at the upload panel.
5. ****No acceptable-use/content-screening line**** (#332 open) ? mitigated by the 4-email allowlist, but the copy should state the trust assumption before the allowlist grows.
6. ****Signed-URL shareability undisclosed**** (1 h bearer links).
7. ****Stale Gemini copy**** ? SPA wizard Gemini step (`:604-608`) and policy `:529` overstate third-party AI for this build; a one-line "this build sends nothing to third-party AI" is both accurate and a selling point.
8. ****A11 copy guardrail**** ? until WASB-C3 is resolved, keep all "own your model" marketing (`:287`) free of commercial-sharing implications (owner-gated, `ALPHA\_LAUNCH\_READINESS.md:110`).

DRAFT limits

Known limits + rollback/disable steps

K1. Locked alpha shape (the limits that are ***by design***)

These are deliberate, config-pinned constraints of the live A1 stack ? launch copy and tester expectations must match them.

```
Locked decision	Evidence
**WASB-only detection ? no Gemini anywhere in serving.** `indexing.skip_ai_handoff: true` is	
baked into the control-plane manifest ("candidate windows ARE the rallies; no Gemini key in this  
service") | `deploy/cloudrun/gen_service.yaml.py:55-56`; live-verified
```

```
`gemini_key_present=false` (e2e runlog $2,
`docs/COMPUTE_DECOUPLED_SERVING/runlogs/DEPLOY_REPORT_cloud-serving_a1-e2e_2026-07-04.md:25-26`)
|
| **Single region, single GPU.** Everything is in asia-southeast1 (`gen_service_yaml.py:43`);
the project's GPU quota is `GPUS_ALL_REGIONS=1` ? one L4 execution at a time, globally |
`docs/archives/decisions/DECISIONS.md:675` ("quota granted (`GPUS_ALL_REGIONS=1`)"); worker Job
shape e2e runlog $1 |
| **One control-plane instance (`maxScale=1`).** Deliberate #471 option-1 mitigation ? per-
instance SQLite state means >1 instance splits the job queue |
`gen_service_yaml.py:25-27,97,145-150`; e2e runlog $1 (rev 00007 deployed with maxScale=1) |
| **Upload size cap: 4096 MB whole-file, 95 MB parts.** `SecurityConfig.max_upload_mb: int =
4096`, `max_part_mb: int = 95` (`backend/config/models.py:370-371`), enforced at
`backend/api/uploads.py:57-58` with the 413-style rejection at `uploads.py:153` ("Upload exceeds
the {N} MB total limit"). Parts stay ?95 MB because free-tier edge proxies cap bodies ~100 MB
(`models.py:368-369`) | files cited |
| ? [analysis] the 4 GB cap is theoretical on this deploy: uploads land under
`/tmp/apphome/output/uploads` (a1-cuj runlog $5.2) on a **4Gi-memory** instance
(`gen_service_yaml.py:152`) whose `/tmp` is RAM-backed (`deploy/cloudrun/README.md:47-48`) ? a
multi-GB upload would exhaust instance memory well before the configured cap. Not observed live;
treat effective cap as ?4 GB | [analysis] |
| **Upload validation floor: ?1280x720 and ?29.9 fps.** A 640x360 clip was rejected live by
`resolution_fps_check` | e2e runlog $3.2 + $5 (lines 41-42, 90); defaults at
`backend/config/models.py:25-27` |
| **Auth posture.** Cloudflare Access at `api.khelsutra.guru`, same "KhelSutra Studio (alpha)"
Access app, **4-email allowlist**, plus in-app CF-JWT verification (401 without `CF-Access-Jwt-
Assertion`) | a1-cuj runlog $6 (`DEPLOY_REPORT_cloud-serving_a1-cuj_2026-07-04.md:78-90`); e2e
runlog $6 (restore verified, rev 00007) |
```

K2. Known-limits table (open issues + live rough edges)

| Limit | Impact on the alpha | Source |
|--|--|--------|
| Control-plane state is per-instance ephemeral SQLite | a redeploy/restart wipes jobs/videos/segments; `/api/highlights` then 404s on the DB check (`backend/main.py:1052`) even though the reel still exists in gs://. A deploy invalidates in-flight jobs and past download URLs Any redeploy mid-tester-session breaks their job polls and download links; re-processing regenerates them #471 (open, bug); e2e runlog \$5 (lines 86-89) | |
| Google-managed cert on `api.khelsutra.guru` can stall at ~90-day renewal | behind the proxied CF CNAME ? verify before ~2026-09-30 (`gcloud beta run domain-mappings describe --domain api.khelsutra.guru --region asia-southeast1`) Silent future outage risk; calendar item for the owner #472 (open) | |
| CORS default is `allow_origins=['*']` | and the deployed manifest sets no `RALLY_CORS_ALLOW_ORIGINS` (env block `gen_service_yaml.py:111-119`), so the wildcard is active on the hosted service too [analysis]. Edge auth + no-credentials CORS blunt it, but it is an unresolved audit residual (R1-12) Residual cross-origin exposure, mainly on non-PNA browsers #443 (open) | |
| No content-safety/NSFW pre-flight | before processing. The WASB-only alpha removes the send-to-Gemini ToS risk (no Gemini in serving), but abusive uploads still burn paid L4 time and produce garbage "Some users will try" ? accepted for a 4-email allowlist, not for wider invites #332 (open) | |
| Path-jail vs storage schemes | gs:// ingest through the jail was unblocked by PR #465 (verified live: gs:// ? 202 with jail enforced, a1-cuj \$6), but the general scheme-exempt design (s3://, gdrive-api://) remains open gs:// works today; other cloud schemes 400 #327 (open); a1-cuj runlog \$6 | |
| No automated upload retention | ? the T11 sweep tool exists but nothing schedules it, and it targets the *local* upload dir. Staged tester uploads persist in `gs://khelsutra-rally-serving/videos/<video_id>/?` indefinitely; no bucket lifecycle policy found in-repo [unverified] Tester footage accumulates in the bucket until manually deleted (see R5) #301 (open); staging path `backend/orchestration.py:284` | |
| Observability gaps | no user-facing per-video trace endpoint (#328); no request capture/replay, and CF-edge failures (302s at Access) never reach engine logs so they're invisible in-app (#326) Tester-reported failures need manual CF-dashboard + Cloud Logging triage #328, #326 (open) | |
| Performance | hosted processing is CPU-bound (ffmpeg/cv2) ? measured ~65% of wait is software encode (#331); WASB decode is single-threaded in streaming mode so the paid L4 sits | |

~80% idle, ~2x realtime detection (#349). Live A1 numbers: ~13m45s process (incl. cold pull) + ~3m26s~4m54s compile for a ~7-min match | Set tester latency expectations in minutes, not seconds; GPU cost per match is inflated | #331, #349 (open); a1-cuj §3; e2e §3.4 |

| ****Golden eval data is machine-split**** (GPU box ? dev sessions, OneDrive sync incomplete) | Slows quality-lane reproduction; no direct tester impact | #175 (open) |

| ****gcloud run services replace**** with an unchanged manifest is a NO-OP** ? it does not re-resolve `:latest`` after a rebuild; force a new revision with `gcloud run services update --image`` | Operator footgun: a "redploy" can silently ship the old image | e2e runlog §5 (lines 83-85) |

| ****422-before-401 quirk****: malformed-JSON bodies get FastAPI's 422 before edge-auth runs; well-formed requests without a CF JWT still 401 | No exposure; don't misread a 422 during auth triage | e2e runlog §5 (lines 91-93) |

| ****No video-delete API**** ? no supported way to remove a video/job record; only manual DB/bucket surgery | See R5 for the actual deletion procedure | #447 (open) |

K3. GATED items that constrain launch copy (A10 / A11 ? both OWNER-GATED, still open)

****A11 ? WASB-C3 provenance (weights NOT commercially cleared).**** The worker image itself carries the warning: **"the pretrained badminton weights are academic-data-trained ? provenance NOT cleared for commercial sharing? Resolve WASB-C3 before any public, non-owner serving"** (`deploy/cloudrun/Dockerfile:14-16``). The tracker keeps A11 GATED on owner/legal (`docs/ALPHA_LAUNCH_READINESS.md:110``) and names C3 a ****hard model-release gate**** that must not be hidden inside product-launch tasks (§5 risk table, `ALPHA_LAUNCH_READINESS.md:85``). Consequences for the alpha:

- ****Who may use it:**** only the current 4-email invite-only allowlist (a1-cuj §6). Note honestly for the GO/NO-GO: those testers are already ***non-owner*** users, which sits in tension with the Dockerfile's "before any public, non-owner serving" ? A13 needs the owner to either resolve C3 or ****explicitly defer it with the documented "no commercial model sharing" copy guardrail**** (that's exactly what the A11 row requires). No wider invite wave until then.
- ****What copy must NOT claim:**** an "owned" or commercially cleared rally-detection model, any model sharing/download, or any commercial license posture. Tracker guardrail: **"Keep all alpha copy scoped to invite-only processing, not owned-model claims"** (`ALPHA_LAUNCH_READINESS.md:80``; also §0 TL;DR line 15 and D1, line 31). Mitigating fact for the record: the weights are **\*\*not baked into the image or distributed\*\*** ? they're staged at runtime from the bucket and used server-side only (`Dockerfile:15-16``; `deploy/cloudrun/README.md:15,29-30``).

****A10 ? ship-gate target not owner-ratified (#172).**** The F1@tIoU targets in `gate_verdict`` are explicitly provisional ("the owner has explicitly stated they did not commit to that figure" ? #172), the A10 row is GATED on the owner decision (`ALPHA_LAUNCH_READINESS.md:109``), and the latest Gen-0 shadow run is `ship=False`` (sign test 9/15, p=0.3036; D6 + §8 A9 record, `ALPHA_LAUNCH_READINESS.md:37,214``). Gate-A served default was ****rejected**** (served mean ? F1 ?0.0328, `ALPHA_LAUNCH_READINESS.md:205``). Consequence: **\*\*the alpha can make no quality claims\*\*** ? no accuracy numbers, no "best-in-class", not even the internal R2/tolF1 contrast as a headline (its strict per-video guard still blocks blanket promotion language, `ALPHA_LAUNCH_READINESS.md:206``). Allowed framing until A10 is ratified: invite-only alpha that detects candidate rally segments and compiles reels, quality under active measurement.

K4. Rollback / disable inventory

All commands run from PowerShell (not git-bash ? MSYS mangles paths/flags, `deploy/cloudrun/README.md:129-133``), region `asia-southeast1``, project `gen-lang-client-0306026826`` (`gen_service.yaml.py:42-43``).

****R1. Disable the whole alpha (fast, reversible, keeps GCS data).**** Two independent switches ? throwing either one cuts testers off:

1. ***Edge off:*** disable (or delete the policy of) the "KhelSutra Studio (alpha)" CF Access application in Zero Trust ? Access ? Applications ? the login path dies at the edge (app identified at a1-cuj §6, lines 81-82; "disable the Access application" is the documented teardown lever, `khelsutra/deploy/DEPLOY_ALPHA.md:156`` ? that runbook is RETIRED but kept as the CF-mechanics reference, line 3). Dashboard click-op; exact click path [unverified].
2. ***Origin off:*** remove the public invoker that CF-proxied traffic rides in on (the inverse of the documented enable step, a1-cuj §6 remaining-(a), lines 93-95):

```
gcloud run services remove-iam-policy-binding rally-control-plane --region asia-southeast1
--member=allUsers --role=roles/run.invoker
...
```

Requests then 403 at Google Front End; `min-instances 1` CPU cost continues until you also scale down or delete.

3. *Full teardown* (documented in the runlog; remember state is ephemeral anyway ? but staged videos/reels in gs:// survive):

```
gcloud run services delete rally-control-plane --region asia-southeast1
gcloud run jobs delete rally-worker --region asia-southeast1
...
```

(a1-cuj \$7, lines 107-108: bucket/secret/repo retained.)

****R2. Roll back to a previous image digest.**** Do NOT use `services replace` with an unchanged manifest ? it's a no-op (e2e \$5). Pin the digest explicitly:

```
...
gcloud run services update rally-control-plane --region asia-southeast1 `
--image asia-southeast1-docker.pkg.dev/gen-lang-client-0306026826/rally/rally-
worker@sha256:<digest>
gcloud run jobs update rally-worker --region asia-southeast1 `
--image asia-southeast1-docker.pkg.dev/gen-lang-client-0306026826/rally/rally-
worker@sha256:<digest>
...
```

The `jobs update` re-pin is the exact mechanism used in the e2e run (e2e \$1); the service image path/name from `gen\_service\_yaml.py:157` (one image serves both roles, #470). ****Last-known-good digest: `sha256:31475f27`**** (rev 00007, edge-auth ON ? e2e \$1/\$6). Do not roll back to `sha256:ed2fe448?`: it predates #476 and reintroduces the broken >32 MiB signed-URL download (#475, e2e \$4). Caveats: any new revision ****wipes the per-instance SQLite state**** ? in-flight jobs and existing download URLs die (#471; e2e \$5); and if config (not code) is what regressed, regenerate the manifest instead: `python deploy/cloudrun/gen\_service\_yaml.py --edge-auth on --image <pinned-digest-ref> -o service.yaml` then `gcloud run services replace service.yaml --region asia-southeast1` (`gen\_service\_yaml.py:6-8,136-140`).

****R3. Revoke a tester.**** Zero Trust ? Access ? Applications ? "KhelSutra Studio (alpha)" ? edit the Allow policy's *Emails* rule and remove the address (policy mechanics per `khelsutra/deploy/DEPLOY\_ALPHA.md:113`; same-app/allowlist confirmed live at a1-cuj \$6). Also revoke the user's active Access session in Zero Trust so an existing cookie doesn't outlive the policy edit ? [unverified: exact dashboard path; not in repo docs]. Defense-in-depth already holds: every `/api/process` re-verifies the CF JWT in-app (`backend/config/models.py:374-379`; e2e \$6 shows 401 without it), so a revoked user can't fall back to hitting the run.app URL directly. There is no app-side account/data scoping to clean up (per-user subdirs are a future PR-3 item, `models.py:364-366`).

****R4. Emergency-stop a running L4 execution.****

```
...
gcloud run jobs executions list --job rally-worker --region asia-southeast1
gcloud run jobs executions cancel <execution-name> --region asia-southeast1
...
```

Execution names look like `rally-worker-k49kg` (a1-cuj \$3.2). The `cancel` verb is standard gcloud but not exercised in any repo runlog [unverified in-repo]. Known consequence chain, from repo sources: a killed container writes ****no completion/failure marker****, so the control plane's bucket-marker poll runs to its bounded timeout ? `ExecutionPolicy.max\_wait\_sec` default ****1800 s**** (`backend/infrastructure/execution\_policy.py:54`; `backend/compute/cloud\_run.py:245`) ? then surfaces a clean `cloud dispatch error` job failure, not a hang (`deploy/cloudrun/README.md:112-115`). GPU billing stops with the execution; the job row on the CP resolves ~30 min later or dies with the instance on the next redeploy (#471).

****R5. Delete a user's data for a given `video\_id`.**** Bucket is the durable store; the DB is ephemeral.

- *Bucket objects* (all three prefixes, plus the user's own source object if they supplied a raw

```
gs:// path outside `videos/<video_id>/`):  
```
```

```
gcloud storage rm -r gs://khelsutra-rally-serving/videos/<video_id>/ # staged upload ?
orchestration.py:284 (input_root gen_service_yaml.py:61)
gcloud storage rm -r gs://khelsutra-rally-serving/outputs/<video_id>/ # worker outputs ?
a1-cuj §3.2 ("outputs at gs://?/outputs/915473?/")
gcloud storage rm -r gs://khelsutra-rally-serving/highlights/<video_id>/ # mirrored reels ?
orchestration.py:748-755
```
```

Also sweep the dispatch done-markers (`<out\_uri>/\_dispatch/<token>.done`,
`deploy/cloudrun/README.md:109-110`) ? exact prefix in the serving bucket [unverified].
- *DB reality:* rows live in per-instance ephemeral SQLite (#471) with **no delete API** (#447
open ? the only known method is manual row surgery, impractical inside a Cloud Run instance).
Practically: after the bucket `rm`, any surviving DB record is inert (`/api/highlights` 404s
once the mirror check fails / on the `main.py:1052` record check after the next restart), and
any redeploy wipes all DB rows anyway (e2e §5) ? a blunt but real "forget everything" lever.
This is adequate for a 4-email alpha but is not a real GDPR-style deletion story; flag alongside
#447/#301 in the launch review.

Cross-cutting caution for every lever above: rollback ? data rollback. `gs://khelsutra-
rally-serving` (weights, staged videos, outputs, reels) is untouched by service/job changes;
state loss is confined to the CP's ephemeral SQLite (#471).

2. user (2026-07-04T11:55:31.934Z)

```
1      # A1 alpha serving ? post-merge e2e deploy test (cloud-serving)  
2  
3      **Date:** 2026-07-04 . **Region:** asia-southeast1 . **Project:** gen-lang-  
client-0306026826  
4      **Scope:** A1 (ALPHA_LAUNCH_READINESS §7) closure ? prove the merged serving fixes  
5      (#465 input jail, #467 upload staging, #468 drain concurrency, #469 signed-URL download,  
6      #470 auth-in-base-image) plus the committed manifest generator (#474) on a REAL deployed  
build,  
7      then restore the live edge-auth posture.  
8  
9      > Companion to `DEPLOY_REPORT_cloud-serving_a1-cuj_2026-07-04.md` (PR #464): that report  
stood the  
10     > stack up and FILED #461/#463/#466; THIS report proves their fixes end-to-end on merged  
master and  
11     > records the one new bug the run surfaced (#475, fixed by #476 mid-run).  
12  
13     ## 1. What was deployed  
14  
15     | Resource | Value |  
16     |---|---|  
17     | Source | merged `master` ? `195932f` (first build) ? **`3c97117`** (final build, incl.  
#476) |  
18     | Worker image | `?/rally/rally-worker:latest` ? build 1 `sha256:ed2fe448?` (195932f),  
build 2 **`sha256:31475f27?`** (3c97117); the Job re-pinned via `gcloud run jobs update` after  
each build |  
19     | L4 worker **Job** | `rally-worker` ? unchanged shape (nvidia-l4, 8 vCPU/32Gi, FUSE  
`khelsutra-rally-serving`?`/gcs`) |  
20     | Control-plane **Service** | `rally-control-plane` ? rev **00005** (auth-off, build 1)  
? rev **00006** (auth-off, final digest) ? rev **00007** (auth-ON restore, final digest).  
Deployed FROM THE BASE IMAGE directly ? no derived PyJWT overlay (**#470 proven**); `maxScale=1`  
(#471 option 1) |  
21     | Manifest | generated by the **committed** `deploy/cloudrun/gen_service_yaml.py` (#474)  
? `--edge-auth off` for the test window, `--edge-auth on` to restore. The scratchpad-artifact  
gap (#473) is closed: this deploy used only repo-committed tooling |  
22  
23     ## 2. Boot / health (rev 00005+, identity-token auth)  
24  
25     - `GET /api/health` ? `200 {"status":"ok",?}`; `GET /api/capabilities` ? `wasb_hybrid`  
present,  
26     `gemini_key_present=false` (skip_ai_handoff honored), ffmpeg/ffprobe present.
```

```

27 - Before-state captured on the outgoing rev 00004: `/api/process` without a CF JWT ?
**401**
28 `edge auth: missing Cf-Access-Jwt-Assertion header` ? the M9 posture was live as
documented.
29
30 ## 3. CUJ on the FINAL image (rev 00006, digest `31475f27?`) ? all green
31
32 1. **gs:// process** ? `POST /api/process {gs://?/MahadevpuraSingles.MP4, wasb_hybrid,
33 compute=cloud}` ? 202 ? `validating ? dispatching (cloud_run) ? done` ? **48 play
segments**
34 (same count as the #464 run; job `18dc6502?`).
35 2. **#461 upload staging** ? chunked upload (`init ? part/0 ? complete`) of a
105,345-byte 720p
36 clip ? `POST /api/process` with the returned **local** path + `compute=cloud` ? the
job staged
37 it to `gs://khelsutra-rally-serving/videos/d8335f51?/upload-e2e-720p.mp4` (byte-exact
size
38 verified in the bucket) and dispatched to the L4 ? **success** (0 segments ? correct
for a
39 synthetic test pattern). NOT the pre-fix "cloud-serving needs a cloud input" hard-
fail.
40 Verified on BOTH builds (`ed2fe448?` and the final `31475f27?`).
41 - ? Upload validation floor: a 640x360 test clip is rejected (`resolution_fps_check`,
42 min 1280x720) ? the first attempt failed on this; alpha docs should mention the
floor.
43 3. **#468 drain concurrency, observed live** ? the upload job staged + dispatched
**while** the
44 48-segment reel compile was stitching on the CP: remote dispatch overlapped the local
compile
45 lane exactly as designed (no serialization, no false-empty stall). On rev 00006 the
two process
46 jobs (gs:// + upload) also ran the drain concurrently.
47 4. **compile** ? `POST /api/compile {video_id, segment_ids:[1..48], "e2e.mp4"}` ? 202 ?
done;
48 measured stitch wall on rev 00005: created 10:39:32 ? finished 10:44:26 (**~4m54s**,
incl. the
49 gs:// source re-fetch); rev 00006 re-run equally green. Reel **mirrored to
50 `gs://khelsutra-rally-serving/highlights/915473?/e2e.mp4` ? 174,046,228 bytes (~166
MiB)**.
51 5. **#463 signed-URL download ? VERIFIED**:
```

- 52 - `GET /api/highlights/915473?/e2e.mp4` ? **~307` ?
- 53 `https://storage.googleapis.com/khelsutra-rally-`
- 54 `serving/highlights/?/e2e.mp4?X-Goog-Algorithm=GOOG4-RSA-SHA256&X-Goog-`
- 55 `Credential=492532266377-compute@?&X-Goog-Expires=3600&X-Goog-Signature=?`
- 56 (v4, signed AS the runtime SA via keyless signBlob ? no key file anywhere).
- 57 - `curl -L` through the redirect: **full 174,046,228 bytes in 8.1s** (single GET ? no
- 58 32 MiB
- 59 edge-500, no Range workaround), and the file **ffprobes valid**: H.264 1920x1080 @
- 60 29.97 fps
- 61 + AAC, duration 244.29s (the 48-rally reel).

```

58
59 ## 4. Bug surfaced mid-run: #475 ? fixed by #476 (merged, redeployed)
60
61 On the FIRST attempt (rev 00005, build `ed2fe448?` = pre-#476), the #463 check
**failed**:
62 `GET /api/highlights/...` ? **500**; log:
63 `[highlights] signed-url path failed ? you need a private key to sign credentials`.
64
65 - The reel WAS mirrored correctly; the **keyless signBlob path** failed and the handler
fell back
66 to serving the 166 MiB file locally ? Cloud Run's ~32 MiB cap ? edge-500 (the original
#463
67 symptom, resurrected by a different root cause).
68 - **Root cause (verified live):** `GCSStorage.signed_url()` reused the storage client's
own token

```

```

69     for `iamcredentials.signBlob`; `google.cloud.storage.Client.SCOPE` is **devstorage-
only**, which
70     signBlob rejects (`ACCESS_TOKEN_SCOPE_INSUFFICIENT`) ? and the branch's bare `except:
pass`
71     swallowed the real error, so the fallback raised the misleading private-key
message.
72     Environment was clean: the SA holds self-`serviceAccountTokenCreator` and
73     `iamcredentials.googleapis.com` is enabled ? code bug, not config.
74     - **Why tests never saw it:** the fakes carry no `_credentials`; this was the FIRST run
of #469's
75     path on real compute credentials. Filed **475** ? fixed by **476**
(`_signblob_identity`)
76     mints a fresh `cloud-platform`-scoped ADC credential; failures now LOGGED before
falling back;
77     regression tests reproduce the credential shape). Suite 1659? / cov 85.46% / ruff+mypy
clean;
78     merged on green CI per the standing agreement; rebuilt (`31475f27?`) and redeployed
(rev 00006);
79     $3.5 is the pass on the fixed build.
80
81     ## 5. Notes / observations (rough edges captured)
82
83     - **`services replace` with an unchanged manifest is a NO-OP** ? it does not re-resolve
`:latest`
84     after a rebuild. Force a new revision (`gcloud run services update --image ?:latest`)
or stamp a
85     revision suffix. Follow-up candidate: `--revision-suffix` in `gen_service_yaml.py`.
86     - **A redeploy wipes the per-instance SQLite state** ? `/api/highlights` then 404s on
the DB
87     record check (`backend/main.py:1052`) even though the reel mirror exists in gs://.
Known, filed
88     as **471**; the CUJ was re-run on rev 00006 to regenerate state. Alpha-operator
implication: a
89     deploy invalidates in-flight jobs and past download URLs (re-process regenerates
them).
90     - Upload validation floor (?1280x720) ? $3.2.
91     - 422-before-401 quirk: a **malformed-JSON** body gets FastAPI's 422 before the edge-
auth
92     dependency runs; any well-formed request without a CF JWT still 401s. No exposure;
noted so a
93     422 in edge-auth triage isn't misread.
94
95     ## 6. Edge-auth restore + live posture (rev 00007, digest `31475f27?`)
96
97     - Redeployed with the generator's `--edge-auth on` manifest ? rev **00007**.
98     - `GET /api/health` ? **200** (deliberately exempt).
99     - `POST /api/process` (valid JSON, no CF JWT) ? **401**
100     `edge auth: missing Cf-Access-Jwt-Assertion header`.
101     - `https://api.khelsutra.guru/api/health` (no CF session) ? **302** ?
102     `https://khelsutra.cloudflareaccess.com/cdn-cgi/access/login/api.khelsutra.guru??`
with the
103     Access-app aud `c980da67?082b` in the login meta ? the exact pre-test posture,
restored.
104
105     ## 7. A1 DoD checklist
106
107     | DoD item | Status |
108     |---|---|
109     | Serving endpoint deployed + healthy | ? rev 00007, final digest `31475f27?` |
110     | Upload ? process ? jobs ? compile ? download CUJ | ? end-to-end on the final image |
111     | Edge-auth (M9) restored + verified | ? 401 sans JWT; api.khelsutra.guru ? CF login 302
|
112     | >32 MiB download via signed URL (#463) | ? 307 + full 174 MB single-GET + ffprobe-
valid |
113     | Browser-upload staging to gs:// (#461) | ? staged byte-exact + dispatched + success |

```



```
114 | Parallel drain (#466/#468) observed live | ? dispatch overlapped compile; 2 process
jobs drained concurrently |
115
116 **A1 ? DONE** (flipped in `docs/ALPHA_LAUNCH_READINESS.md` $7 in this PR).
117
```

3. user (2026-07-04T11:55:32.650Z)

```
1 # Alpha Launch Readiness Plan
2
3 > **Status:** `DRAFT` - cross-repo execution tracker; **Owner:** @avidullu; **Created:**
2026-07-03; **Last updated:** 2026-07-04
4 > **Lifecycle:** `DRAFT -> IN PROGRESS -> DONE -> archived`
5 > **Tracking anchors:** Section 7 is the progress tracker and source of truth; indexed
in `docs/README.md` and `docs/DOC_STATUS.md`.
6 > **Relation to existing docs:** consolidates the launch-relevant pending items from
`NEXT_STEPS.md`, `ROADMAP.md`, `docs/data_pipeline/GOLDEN_VIDEOS.md`,
`R2_EVAL_METRIC_DESIGN.md`, `COMPUTE_DECOUPLED_SERVING/`, and the sibling `khelsutra` Studio
docs.
7 > **Honesty note:** `[verified]` means checked against the local synced repos on
2026-07-03. `[analysis]` means derived from a local command run on that snapshot. This is not
legal, financial, or launch approval.
8
9 ---
10
11 ## 0. TL;DR
12
13 The corpus is now large enough to move from "grow labels first" into alpha-readiness
work, but the 15-video headline is not itself a launch gate pass. `[verified]` The golden
manifest has 15 badminton videos; `[analysis]` A2-A9 made the local manifest/path, full-
fusion/audio, decision, and shadow-training surfaces explicit; `[analysis]` A4/A5 refreshed the
n=15 nightly and heuristic floors; `[analysis]` A8/A9 still reject served Gate A default
promotion and owned-model serving claims on the current evidence.
14
15 The first controlled alpha should be a hosted, authenticated product loop: upload a
video, process it, select rallies, compile, and download a reel. The owned-model/commercial
rally-detection claim remains a separate NO-GO until the ship-gate target is ratified and the
WASB checkpoint provenance is resolved.
16
17 ## 1. Problem & goal
18
19 The previous launch blockers were spread across roadmap docs and mixed together three
different questions:
20
21 - **Can an invited tester use the product without SSH?**
22 - **Can the 15-video golden corpus now support honest quality gates?**
23 - **Can we claim or ship an owned/commercial rally detector?**
24
25 This doc separates those into executable workstreams. The goal is to reach an invite-
only alpha where the product is usable end-to-end, the quality claims are honest and
reproducible, and the remaining model/legal gates are explicit rather than accidentally implied
by launch activity.
26
27 ## 2. Decisions locked
28
29 | # | Decision | Source / date | Implication |
30 |---|---|---|---|
31 | D1 | **Product alpha and owned-model launch are separate decisions.** |
`NEXT_STEPS.md` NO-GO banner, 2026-06-30; verified 2026-07-03 | We can launch a controlled
hosted product loop without claiming the owned model is commercially cleared. |
32 | D2 | **First alpha must be page-driven, not SSH/CLI-driven.** | `NEXT_STEPS.md` P0 UI-
driven/shareable item; verified 2026-07-03 | Hosted service, edge auth, upload, job polling,
compile, and download are launch-critical. |
33 | D3 | **The 15-video corpus is now a measurement asset, not a free launch pass.** |
`[verified]` `docs/data_pipeline/GOLDEN_VIDEOS.md`; `[analysis]` local eval runs, 2026-07-03 |
```

Rebase/portability, feature completeness, baseline refresh, and gate ratification come before quality claims. |

34 | D4 | ****Do not promote served Gate A by default from the current n=15 evidence.**** | ``[analysis]` backend.eval.serve_contrast`` run, 2026-07-04 | Proposal-side lift does not survive served-windowing safety; keep it as an opt-in/analysis lever until owner ratification. |

35 | D5 | ****P10/P11 corpus promotion and train/re-eval are alpha-plus unless owner reopens D13.**** | sibling ``kshelsutra/docs/STUDIO_MEDIA_LIFECYCLE.md``; verified 2026-07-03 | First alpha can use manual/offline corpus promotion; automated flywheel follows after serving and gates are stable. |

36 | D6 | ****Gen-0 n=15 shadow training is non-serving.**** | ``[analysis]` training.gen0.harness`` run, 2026-07-04 | Learned mean beats the heuristic floor, but the paired sign test is not significant; #172 and WASB C3 remain hard blockers. |

37

38 **## 3. Verified snapshot, 2026-07-03**

39

40 All five Git checkouts were fast-forwarded before analysis:

41

| Repo | Branch | Synced HEAD |
|--|-----------------------|------------------------|
| <code>`badminton-highlight-indexer`</code> | <code>`master`</code> | <code>`e4914d1`</code> |
| <code>`kshelsutra`</code> | <code>`master`</code> | <code>`b0f70a5`</code> |
| <code>`rally-annotator`</code> | <code>`main`</code> | <code>`ef9e088`</code> |
| <code>`sports-data-collector`</code> | <code>`master`</code> | <code>`bfc014f`</code> |
| <code>`sports-obsreport`</code> | <code>`main`</code> | <code>`a0c7252`</code> |

49

50 Corpus/eval facts:

51

52 - ``[verified]` docs/data_pipeline/GOLDEN_VIDEOS.md`` says ``output/human_lovo_manifest.json`` is the ingested golden corpus at ``n = 15``.

53 - ``[analysis]`` The local manifest had stale absolute paths. A temporary rebased manifest found all 15 trajectories and labels in this workspace.

54 - ``[analysis]`` A7 generated full shuttle/person fusion schema for all 15 videos and A8 audio augmentation populated audio features for all 15.

55 - ``[analysis]`` ``python -m backend.eval.fusion_compare --features-dir output`` now runs over all 15 and measures person-fusion ``Delta F1 = +0.008``; the paired CI crosses zero, so person fusion is not promotable by itself.

56 - ``[analysis]`` ``python -m backend.eval.fusion_audio --features-dir output --compare`` measures audio ``Delta F1 = +0.018``; the paired CI crosses zero, so audio is a shadow/suggestive cue, not a default-flip.

57 - ``[analysis]`` ``python -m backend.eval.serve_contrast --manifest output/human_lovo_manifest.json`` measured proposal Gate A mean ``Delta F1 +0.0287``, but served mean ``Delta F1 -0.0328``; not served-safe.

58 - ``[analysis]`` ``python -m training.gen0.harness --manifest output/human_lovo_manifest.json --floor eval_baselines/heuristic_lovo_n15.json --version 0.1.1-n15-shadow.20260704 --out artifacts/rally_tal_gen0`` produced learned LOVO mean F1 ``0.551``, floor passed versus heuristic ``0.446``, but sign test ``9/15``, ``p = 0.3036``, ``ship=False``.

59 - ``[analysis]`` ``python scripts/nightly_regression.py --manifest <rebased-n15> --features-dir output --no-report`` reported default/milestone ``tolF1 0.3636`` vs baseline-off ``tolF1 0.1382``, but marked the frozen baseline stale due to config/corpus changes.

60 - ``[analysis]`` Before A6, ``python -m backend.eval.ablation --features-dir output --granularity group`` failed at import due to a circular import between ``backend/eval/ablation.py`` and ``backend/eval/ablation_pdf.py``; A6 fixes the module entrypoint and keeps full-fusion ablation scoped to the 6 videos with full schema until A7.

61

62 **## 4. Launch strategy**

63

64 **### 4.1 Alpha lane: make the product usable**

65

66 The alpha lane is successful when an invited tester can open the app, authenticate, upload a video, watch the job progress, pick rallies, compile a reel, and download it. This lane should avoid model-quality promises beyond the measured state and should default to the most reliable serving path.

67

68 **### 4.2 Quality lane: make the 15-video corpus gate-ready**

69

70 The quality lane is successful when the 15-video corpus is portable, reproducible,
feature-complete, and backed by refreshed baselines. Only then should R2 tolF1,
Gen-0/TemporalMaxer, or other model/promotion decisions be ratified.

71

72 ### 4.3 Flywheel lane: make tester feedback useful

73

74 The flywheel lane is successful when user corrections can flow into the collector/vault
path with consent/provenance and a visible re-eval delta. This is valuable for alpha learning,
but it should not block the first controlled alpha if manual promotion can bridge the gap.

75

76 ## 5. Risks and guardrails

77

78 | Risk | Why it matters | Guardrail |

79 |---|---|---|

80 | Launch activity implies model launch approval | The engine docs explicitly say rally-
detection product launch is NO-GO until ship target and WASB C3 clear | Keep all alpha copy
scoped to invite-only processing, not owned-model claims |

81 | Stale local manifests create false confidence | Manifest-driven tools skipped or
failed until paths were rebased | Make the corpus manifest portable and add a path/fixture smoke
check |

82 | 15-video corpus is only partially feature-complete | Fusion/person/audio eval is still
n=6 today | Regenerate or upconvert full-fusion features for all 15 before fusion decisions |

83 | Baseline refresh accidentally rubber-stamps a regression | Nightly runner correctly
reported stale baselines | Refresh baselines only after reviewer sign-off and record the
command/output |

84 | P10/P11 expands scope before hosted alpha is usable | Corpus flywheel is valuable but
can consume the alpha launch window | Keep P10/P11 behind explicit owner reopen unless alpha
lane is already stable |

85 | WASB checkpoint provenance remains unresolved | Blocks commercial sharing of a trained
model regardless of F1 | Treat C3 as a hard model-release gate; do not hide it inside product
launch tasks |

86

87 ## 6. Honest limits

88

89 Completing this doc's alpha lane does ****not**** prove the owned model is ready, that WASB
weights are commercially clean, that the product can serve unbounded public traffic, that multi-
sport is ready, or that every tester correction automatically improves the model.

90

91 Completing the quality lane does ****not**** by itself deploy a service. It only makes the
measurement surface honest enough for a launch decision.

92

93 ## 7. Deliverables & progress tracker

94

95 Legend: TODO = not started, IN PROGRESS = actively being worked, DONE =
shipped/verified, GATED = blocked on owner/legal/platform decision. One PR per row unless the
row explicitly names a cross-repo batch.

96

97 | ID | Deliverable | Repo / owner | Depends on | Gated? | Status | PR |

98 |----|-----|-----|-----|-----|-----|----|

99 | A0 | Create this alpha readiness tracker and index it in docs | `badminton-highlight-
indexer` | - | No | DONE | #451 |

100 | A1 | Stand up alpha serving endpoint: Cloud Run/GPU or approved host, health check,
edge auth ON, upload -> process -> jobs -> compile -> download verified | `khelsutra` +
`badminton-highlight-indexer` | A0 | Owner spend / CF config | DONE |
#462/#464/#465/#467/#468/#469/#470/#474/#476; e2e runlog
`docs/COMPUTE\_DECOUPLED\_SERVING/runlogs/DEPLOY\_REPORT\_cloud-serving\_a1-e2e\_2026-07-04.md` |

101 | A2 | Make the n=15 golden manifest portable/reproducible; add a path/corpus smoke
check that fails loudly when labels or trajectories are missing | `badminton-highlight-indexer`
| A0 | No | DONE | #452 |

102 | A3 | Promote the 15-video source-video/MD5 records into the collector/vault path;
reconcile collector's six-video source manifest with the 15-video eval corpus | `sports-data-
collector` + vault | A2 | Owner upload/storage scope | DONE | collector #62 |

103 | A4 | Refresh the n=15 nightly regression baseline intentionally and record the exact
command/output; keep stale-baseline warnings until reviewed | `badminton-highlight-indexer` | A2
| Reviewer sign-off | DONE | #453 |

```

104 | A5 | Recompute the heuristic n=15 floor and replace the stale `heuristic_lovo_n6`
floor for future Gen-0 comparisons | `badminton-highlight-indexer` | A2 | No | DONE | #454 |
105 | A6 | Fix `backend.eval.ablation` CLI circular import, then run the R2/tolF1 ablation
on the n=15 corpus | `badminton-highlight-indexer` | A2 | No | DONE | #455 |
106 | A7 | Regenerate or upconvert full-fusion feature JSONs for the newer 9 videos so
fusion/person/audio analyses run over all 15 | `badminton-highlight-indexer` | A2 | GPU/data
availability | DONE | #458 |
107 | A8 | Re-run `fusion_compare`, group ablation, and served contrast after A7; explicitly
promote/reject person fusion, Gate A served-default, and any other R-series default flips |
`badminton-highlight-indexer` | A6, A7 | Reviewer sign-off | DONE | #458 |
108 | A9 | Run Gen-0/TemporalMaxer shadow training against the refreshed n=15 floor; produce
a non-serving artifact bundle and gate verdict | `badminton-highlight-indexer` | A5, A8 | Owner
approval for M0.4 scope | DONE | #458 |
109 | A10 | Decide and record the alpha ship-gate target: metric, corpus slice, minimum
threshold, significance rule, and what claims are allowed | Owner + `badminton-highlight-
indexer` | A4, A5, A8 | Owner decision | GATED | - |
110 | A11 | Resolve or explicitly defer WASB C3 provenance for alpha messaging; if deferred,
document "no commercial model sharing" copy guardrail | Owner/legal | A10 | Legal/provenance |
GATED | - |
111 | A12 | Reopen or defer P10/P11 corpus flywheel: `PATCH /api/segments`, collector
promotion, `train`/`reeval` job, delta-F1 surface, and alignment with
`COMPUTE_DECOUPLED_SERVING` M11 | `kheksutra` + `badminton-highlight-indexer` + collector | A1,
A3 | Owner D13 reopen (`kheksutra/docs/STUDIO_MEDIA_LIFECYCLE.md`) | GATED | - |
112 | A13 | Alpha launch review: runbook, privacy/consent copy, observability/job-failure
triage, known limits, rollback/disable steps, and owner GO/NO-GO | Cross-repo | A1, A4, A10, A11
| Owner launch decision | TODO | - |
113
114 ## 8. Concrete next PR sequence
115
116 1. **Docs PR:** land this tracker.
117 2. **Corpus portability PR:** rebase/portable manifest tooling plus missing-file smoke
test.
118 3. **Hosted alpha PR/batch:** health/auth/upload/process/job/compile/download
verification and runbook updates.
119 4. **Baseline PR:** intentional n=15 nightly baseline refresh and heuristic floor
refresh.
120 5. **Ablation/fusion PRs:** fix ablation import, complete 15-video features, rerun
promotion/rejection decisions.
121 6. **Launch decision PR/docs update:** record ship-gate target, C3 status, alpha copy
limits, and launch checklist.
122
123 A2 smoke command:
124
125 ```powershell
126 python -m backend.eval.golden_manifest --manifest output/human_lovo_manifest.json --min-
count 15
127 python -m backend.eval.golden_manifest --manifest output/human_lovo_manifest.json --min-
count 15 --write output/human_lovo_manifest.local.json
128 ```
129
130 The checker resolves stale absolute paths against the repo's sibling `Annotation Setup`
folders and `output/`, then exits non-zero if any required `trajectory` or `labels` file is
absent.

```

4. user (2026-07-04T11:56:06.818Z)

```

1 """Generate the control-plane Cloud Run *service manifest* (COMPUTE_DECOUPLED_SERVING
M8/M9, #473).
2
3 The A1 alpha deploy drove `rally-control-plane` from a hand-generated YAML that lived in
a
4 session scratchpad ? this commits the generator so the deploy is reproducible from a
clean
5 clone. It emits a Knative service manifest for::
6

```

```

7         python deploy/cloudrun/gen_service_yaml.py --edge-auth off -o service.yaml
8         gcloud run services replace service.yaml --region asia-southeast1
9
10        Why a MANIFEST (`services replace`) and not `gcloud run deploy` flags: the container
command is
11        a /bin/bash wrapper carrying shell metacharacters (pipes, redirects, &&) ? on Windows
the gcloud
12        cmd-shim mangles those through `--command/--args`, while a YAML round-trips them
verbatim. Run
13        gcloud from PowerShell, not git-bash (MSYS rewrites `/tmp`-style paths).
14
15        The app is configured through a config.json baked INTO the manifest: the wrapper
base64-decodes
16        it to `/tmp/apphome/config.json` and exports `RALLY_APP_HOME=/tmp/apphome` before
exec'ing the
17        server, so one image serves every config shape (config.json wins over the cloud-serving
preset ?
18        backend/config/loader.py). `--edge-auth` is REQUIRED and explicit: `off` is only for the
19        identity-token smoke/e2e window; redeploy with `on` (the Cf-Access JWT verify, M9)
before the
20        service faces testers again.
21
22        Deliberate manifest choices (learned on the A1 deploy):
23        * `run.googleapis.com/cpu-throttling: "false"` + minScale 1 ? the in-process job
worker drains
24        on a background thread; without always-allocated CPU a 202'd compile stalls (README
§3a).
25        * maxScale DEFAULTS TO 1 ? the jobs/videos DB is per-instance ephemeral SQLite, so a
second
26        instance splits the queue and `/api/jobs/{id}` polls (issue #471, option 1). Raise
it only
27        once state is shared.
28        * `CLOUD_RUN_JOB` is NOT set ? it's a reserved runtime env name and the manifest is
rejected
29        with it; the code default ("rally-worker") is already correct.
30        * No secrets here: bucket/project/aud/team-domain are the same non-secret identifiers
the
31        committed runlogs carry; the Gemini key stays in Secret Manager (the alpha runs
32        `skip_ai_handoff=true` and never needs it).
33        ""
34
35        from __future__ import annotations
36
37        import argparse
38        import base64
39        import json
40        import sys
41
42        DEFAULT_PROJECT = "gen-lang-client-0306026826"
43        DEFAULT_REGION = "asia-southeast1"
44        DEFAULT_SERVICE = "rally-control-plane"
45        DEFAULT_SERVING_BUCKET = "gs://khelesutra-rally-serving"
46        DEFAULT_CF_TEAM_DOMAIN = "https://khelesutra.cloudflareaccess.com"
47        DEFAULT_CF_ACCESS_AUD =
"c980da67ed888fff935fcd913ae1eb4da63f9d65805d6e1c5ca76f92f517082b"
48
49
50        def build_config(args: argparse.Namespace) -> dict:
51            """The app config the wrapper materializes at RALLY_APP_HOME/config.json."""
52            bucket = args.serving_bucket.rstrip("/")
53            return {
54                "deployment": {"profile": "cloud-serving", "compute_target": "cloud_run"},
55                # WASB-only alpha: candidate windows ARE the rallies; no Gemini key in this
service.
56                "indexing": {"skip_ai_handoff": True},

```

```

57         "storage": {
58             "backend": "gcs",
59             "output_root": bucket,
60             "input_backend": "gcs",
61             "input_root": f"{bucket}/videos",
62         },
63         "security": {
64             "edge_auth_enabled": args.edge_auth == "on",
65             "cf_team_domain": args.cf_team_domain,
66             "cf_access_aud": args.cf_access_aud,
67             "allowed_video_root": "/tmp/apphome/output",
68         },
69     }
70
71
72 def build_yaml(args: argparse.Namespace) -> str:
73     """Render the Knative service manifest for `gcloud run services replace`."""
74     cfg_b64 = base64.b64encode(
75         json.dumps(build_config(args), separators=(",", ":")).encode("utf-8")
76     ).decode("ascii")
77     # mkdir BEFORE the redirect (the redirect can't create the directory), then exec so
78     # server is PID 1 and Cloud Run's SIGTERM reaches it directly.
79     wrapper = (
80         "mkdir -p /tmp/apphome && "
81         f"printf %s '{cfg_b64}' | base64 -d > /tmp/apphome/config.json && "
82         "export RALLY_APP_HOME=/tmp/apphome && "
83         "exec python3 -m backend.main --server --host 0.0.0.0 --port 8080"
84     )
85     return f"""
86 apiVersion: serving.knative.dev/v1
87 kind: Service
88 metadata:
89   name: {args.service}
90   labels:
91     cloud.googleapis.com/location: {args.region}
92 spec:
93   template:
94     metadata:
95       annotations:
96         autoscaling.knative.dev/minScale: "{args.min_instances}"
97         autoscaling.knative.dev/maxScale: "{args.max_instances}"
98         run.googleapis.com/cpu-throttling: "false"
99     spec:
100       timeoutSeconds: {args.timeout}
101       containers:
102         - image: {args.image}
103           command: ["/bin/bash"]
104           args: ["-c", "{wrapper}"]
105           ports:
106             - containerPort: 8080
107       resources:
108         limits:
109           cpu: "{args.cpu}"
110           memory: {args.memory}
111       env:
112         - name: DEPLOYMENT_PROFILE
113           value: cloud-serving
114         - name: CLOUD_RUN_REGION
115           value: {args.region}
116         - name: GOOGLE_CLOUD_PROJECT
117           value: {args.project}
118         - name: RALLY_ALLOWED_HOSTS
119           value: "*"
120 """

```

```

121
122
123 def main(argv: "list[str] | None" = None) -> int:
124     p = argparse.ArgumentParser(description=__doc__.splitlines()[0])
125     p.add_argument(
126         "--edge-auth",
127         choices=("on", "off"),
128         required=True,
129         help="REQUIRED and explicit: 'off' only for the identity-token smoke/e2e window;
"
130         "redeploy 'on' (M9 Cf-Access verify) before testers touch it again.",
131     )
132     p.add_argument("--project", default=DEFAULT_PROJECT)
133     p.add_argument("--region", default=DEFAULT_REGION)
134     p.add_argument("--service", default=DEFAULT_SERVICE)
135     p.add_argument(
136         "--image",
137         default=None,
138         help="Container image; default = the project's rally-worker:latest (post-#470 it
"
139         "carries cloud-run + auth, so the worker image IS the control-plane image).",
140     )
141     p.add_argument("--serving-bucket", default=DEFAULT_SERVING_BUCKET)
142     p.add_argument("--cf-team-domain", default=DEFAULT_CF_TEAM_DOMAIN)
143     p.add_argument("--cf-access-aud", default=DEFAULT_CF_ACCESS_AUD)
144     p.add_argument("--min-instances", type=int, default=1)
145     p.add_argument(
146         "--max-instances",
147         type=int,
148         default=1,
149         help="Default 1: per-instance SQLite state splits across instances (issue
#471).",
150     )
151     p.add_argument("--cpu", type=int, default=2)
152     p.add_argument("--memory", default="4Gi")
153     p.add_argument("--timeout", type=int, default=300)
154     p.add_argument("-o", "--output", default="-", help="Output path, or '-' for
stdout.")
155     args = p.parse_args(argv)
156     if args.image is None:
157         args.image = f"{args.region}-docker.pkg.dev/{args.project}/rally/rally-
worker:latest"
158
159     text = build_yaml(args)
160     if args.output == "-":
161         sys.stdout.write(text)
162     else:
163         with open(args.output, "w", encoding="utf-8", newline="\n") as f:
164             f.write(text)
165     return 0
166
167
168 if __name__ == "__main__":
169     raise SystemExit(main())
170

```

5. user (2026-07-04T11:56:11.787Z)

```

1     # Cloud Run GPU serving ? build / deploy / run (COMPUTE_DECOUPLED_SERVING M6/M7)
2
3     The cloud-serving compute path: a control plane (`CloudRunCompute`,
`backend/compute/cloud_run.py`)
4     dispatches a Cloud Run Job execution running this image's worker on an L4, then
bucket-polls
5     a done-marker for completion (D16). Scale-to-zero (D7) ? no warm pool.

```

```

6
7     > **Status:** the dispatch shim + image are **landed + mock-tested** (M6). The **real
build + push +
8     > run on an L4** is the **M7** owner step, within the **$100/?10k** budget (D17) ?
confirm cmd + hourly
9     > cost first, **delete every resource after** ([[gcp-vm-always-delete]]). CI does
**not** build this image.
10
11     ## 0. Prerequisites (owner)
12     - A GCP project with billing on + the Cloud Run, Artifact Registry, and Cloud Build APIs
enabled.
13     - GPU quota for L4 (`GPUS_ALL_REGIONS` ? 1) in the chosen region (asia-south1, else
Singapore ? see
14         `deploy/gcp/README.md`; L4 is often stocked-out, fall back per that runbook).
15     - The WASB weights staged in the bucket (WASB-C3 unresolved ? owner-gated):
`gs://<bucket>/models/wasb_badminton_best.pth.tar`.
16
17     ## 1. Build + push the image (Artifact Registry)
18     ```bash
19     PROJECT=<gcp-project>; REPO=rally
20     # L4 GPU is region-limited ? use asia-southeast1 (Singapore), NOT asia-south1 (Mumbai).
21     REGION=asia-southeast1
22     gcloud artifacts repositories create $REPO --repository-format=docker --location=$REGION
2>/dev/null || true
23     # `gcloud builds submit` has NO -f flag, so a non-root Dockerfile needs a build config:
24     gcloud builds submit --config=deploy/cloudrun/cloudbuild.yaml --project $PROJECT .
25     ```
26     *(A CUDA + micromamba image is large; the first build is ~8?10 min. Budget-watch per
D17.)*
27
28     ## 2. Create the Cloud Run **Job** (GPU, scale-to-zero)
29     The proven shape (first real M7 deploy, 2026-07-03): weights staged in a **same-region
bucket**
30     mounted read-only via GCS FUSE (no weights in the image ? WASB-C3; no init-fetch step
needed).
31     ```bash
32     gcloud run jobs create rally-worker \
33         --image $REGION-docker.pkg.dev/$PROJECT/$REPO/$IMG:latest \
34         --region $REGION --gpu 1 --gpu-type nvidia-l4 --no-gpu-zonal-redundancy \
35         --cpu 8 --memory 32Gi \
36         --max-retries 0 --task-timeout 3600 \
37         --set-env-vars WASB_WEIGHTS=/gcs/models/wasb_badminton_best.pth.tar \
38         --add-volume name=serving,type=cloud-storage,bucket=$SERVING_BUCKET \
39         --add-volume-mount volume=serving,mount-path=/gcs
40     ```
41     - **`--no-gpu-zonal-redundancy` is REQUIRED on the CLI**, not just a cost choice (it
also halves
42     the GPU rate ? right for batch jobs): without the flag `gcloud` asks an interactive
Y/n which,
43     in a non-interactive shell, **hangs forever with zero output and nothing created**. An
empty
44     `gcloud` that "did nothing" ? suspect a hidden prompt.
45     - 8 vCPU / 32 GiB: decode + x264 are CPU-side (NVDEC is a deferred D16 knob), so CPU
headroom is
46     what keeps the paid GPU busy. `/tmp` is **RAM-backed** ? the pulled input video lives
in memory,
47     so size `--memory` for the largest expected upload (a 5.8 GB GoPro file ? 6 GiB of
tmpfs).
48
49     ## 3. Wire the dispatcher (control plane)
50     The control plane runs with the **cloud-serving** profile (`DEPLOYMENT_PROFILE=cloud-
serving` ?
51     `compute_target=cloud_run`). `get_compute_backend(config)` then builds `CloudRunCompute`
from env:
52     ```bash

```



```

53     export DEPLOYMENT_PROFILE=cloud-serving
54     export CLOUD_RUN_JOB=rally-worker CLOUD_RUN_REGION=$REGION
GOOGLE_CLOUD_PROJECT=$PROJECT
55     # A `worker infer` (or the API job path) with a gs:// video now DISPATCHES to the Job
instead of
56     # running in-process; the container runs the SAME worker INLINE (compute_target forced
local_gpu).
57     python -m backend.worker infer --video-uri gs://<bucket>/videos/clip.mp4 --out-uri
gs://<bucket> --segmenter wasb_hybrid
58     ```
59     **The AI handoff needs its key IN THE JOB env** (`GEMINI_API_KEY` ? mount a Secret
Manager secret
60     on the job), or run fully-local with `--set indexing.skip_ai_handoff=true` (candidate
windows
61     emitted as rallies ? the eval-parity mode). With neither, the segmenter used to bail to
a silent
62     0-rally "success" **after** the paid GPU pass; the cloud-serving startup checks now fail
this at
63     boot (`AI_HANDOFF_KEY_MISSING`, rc 2) before the video is even pulled.
64
65     ### 3a. Control-plane SERVICE deploy ? required flags for reliable async jobs
66
67     The API server (`/api/process`, `/api/compile` ? #329) runs its **job worker IN-
PROCESS**: the
68     request returns `202` and a background thread drains the job (GPU detection dispatches
to the Job;
69     the reel stitch runs on the control plane itself). On Cloud Run that background thread
only makes
70     progress while the instance has CPU, so a scale-to-zero **service** deploy MUST set:
71
72     **The canonical deploy path is the committed manifest generator** (#473) ? it bakes the
73     cloud-serving config.json into a startup wrapper, sets every flag below, pins
`maxScale=1`
74     (per-instance SQLite state, issue #471), and deliberately does NOT set `CLOUD_RUN_JOB`
(a
75     reserved runtime env name that gets a manifest rejected; the code default is already
right):
76     ```bash
77     python deploy/cloudrun/gen_service_yaml.py --edge-auth on -o service.yaml
78     gcloud run services replace service.yaml --region $REGION # from PowerShell on Windows
79     ```
80     `--edge-auth off` is only for the pre-CF identity-token smoke/e2e window; redeploy with
`on`
81     before testers touch the service again. The flag-based equivalent (for reference ? a
manifest
82     round-trips the bash wrapper's metacharacters, flags don't on Windows):
83     ```bash
84     gcloud run deploy rally-control-plane --image <image> --region $REGION \
85         --no-cpu-throttling \           # CPU stays allocated between requests ? the drain thread
finishes
86         --min-instances 1 \           # one always-warm instance drains reliably (cheap CPU;
NOT the D7 GPU pool)
87         --cpu 2 --memory 4Gi --port 8080 --allow-unauthenticated \
88         --set-env-vars DEPLOYMENT_PROFILE=cloud-
serving,CLOUD_RUN_REGION=$REGION,GOOGLE_CLOUD_PROJECT=$PROJECT
89     ```
90     Without `--no-cpu-throttling`, a `/api/compile` returns 202 and then **stalls** ? the
reel never
91     finishes and startup crash-recovery would mark it terminal. `min-instances 1` does NOT
contradict
92     D7 (that was the ~$500/mo warm **GPU** pool); this is a ~1-vCPU CPU instance. Defense-
in-depth is in
93     the code: `Database.recover_stale_jobs()` **re-queues** a compile interrupted by a
restart (idempotent
94     stitch) rather than failing it, so even an unlucky scale-down mid-stitch self-heals on

```

the next drain.

95 Reserve `--allow-unauthenticated`` for the pre-CF-auth smoke test only; lock ingress to the CF proxy

96 (M9) before inviting testers. The image ships the ``auth`` extra (PyJWT[crypto]), so flipping

97 ``security.edge_auth_enabled`` on needs no derived image ? the M9 Cf-Access JWT verify works out of

98 the box.

99

100 ## 4. M7 acceptance (the owner runlog)

101 Capture a ``DEPLOY_REPORT_cloud-serving_<date>.md`` under

102 ``docs/COMPUTE_DECOUPLED_SERVING/runlogs/`` (mirrors the truly-local template): the exact commands,

103 rally counts, ``gpu_active_seconds`/`wall_seconds`/`bytes_out``, a reel smoke-test (``ffmpeg -f null -``

104 `exit 0 + a 5-frame contact sheet`), gotchas, and the `**$0 teardown audit**`.

105

106 ## Gotchas

107 - The dispatched container `**must run inline**` ? ``CloudRunCompute`` forces ``compute_target=local_gpu``

108 + ``indexing.detector_impl=native`` in the per-execution args so the Job never re-dispatches itself.

109 - Completion is a `**bucket marker**` (``<out_uri>/_dispatch/<token>.done``), not the Cloud Run API status

110 ? so it reuses the storage layer + ``execution_policy.poll_until`` and survives a control-plane restart.

111 - A `**failed**` Job (validation rc 2 / segmenter rc 3) now writes a `**failure marker**` carrying the

112 real non-zero rc, so the dispatcher's poll terminates `**FAST + accurately**` (no 30-min hang on a bad

113 video). Only a `**hung/crashed**` container (no marker at all) makes the bounded poll time out

114 (``ExecutionPolicy.max_wait_sec``); the worker CLI maps that to a clean `**rc 4**`, and the API path

115 surfaces it as a ``cloud dispatch error`` job failure (not a traceback).

116 - libx264 encode (parity, D16) + NVDEC decode-only; the image keeps ffmpeg's default libx264.

117 - `**`GOOGLE_CLOUD_PROJECT` is required**` for the real client ? ``GoogleCloudRunJobClient.run_job``

118 fails fast with a clear ``RuntimeError`` if it is unset/empty (it must resolve a real ``job_path``;

119 an unset project would otherwise build a bogus ``projects/None/...`` name). Export it (step 3) before

120 dispatching. The guard runs before the ``google-cloud-run`` import, so it is exercised in CI even

121 though the SDK isn't a CI/test dependency.

122 - `**Hand-rolled `jobs execute --args` with `gs://` URIs:**` gcloud's default comma-split plus its

123 alternate-delimiter syntax both bite ? repeated `--set`` tokens trip the comma parser, and a

124 ``^:^`` delimiter splits the ``:`` inside ``gs://``. Use a delimiter that appears nowhere in the args,

125 e.g. ``--args='^@@^~v@@infer@@--video-uri=gs://?'``. (The dispatcher path doesn't have this

126 problem ? prefer it; this is for one-off diagnostics.) Container-arg overrides must include the

127 dispatcher's two forced `--set`s: `deployment.compute_target=local_gpu``,

128 ``indexing.detector_impl=native``.

129 - `**Git Bash (MSYS) on Windows mangles Unix-looking values**` before gcloud sees them:

130 ``--add-volume-mount ?mount-path=/gcs`` became ``C:/Program Files/Git/gcs``, and

131 ``--command=/bin/bash`` became ``C:/Program Files/Git/usr/bin/bash`` (the container then exec-fails).

132 Run gcloud commands with such flags from PowerShell (``MSYS_NO_PATHCONV=1`` breaks gcloud's own

133 wrapper). PowerShell has its own trap: quote any flag value containing commas.

```

134 - **First execution after an image push** spends minutes in "Container image import in
progress"
135 (visible in `executions describe status.conditions`, NOT in logs) ? that's progress,
not a hang;
136 later executions reuse the imported image.
137

```

6. user (2026-07-04T11:56:12.370Z)

```

1 # A1 alpha serving deploy ? hosted CUJ run report (cloud-serving)
2
3 **Date:** 2026-07-04 . **Region:** asia-southeast1 . **Project:** gen-lang-
client-0306026826
4 **Scope:** A1 (ALPHA_LAUNCH_READINESS S7) ? stand up the alpha serving endpoint and
verify the
5 hosted `process` ? jobs ? compile ? download` CUJ against the WASB-only owned detector.
6
7 > Companion to `DEPLOY_REPORT_cloud-serving_2026-07-04.md` (PR #457, the GPU-batching
runlog). That
8 > report proved the L4 worker + measured the batching win via the M6 dispatch **shim**;  
THIS report
9 > proves the deployed **control-plane service** end-to-end. They do not overlap.
10
11 ## 1. What was deployed
12
13 | Resource | Value |
14 |---|---|
15 | L4 worker **Job** | `rally-worker` ? nvidia-l4, 8 vCPU / 32Gi, `--no-gpu-zonal-
redundancy`, `--max-retries 0 --task-timeout 3600`, GCS-FUSE mount `khelsutra-rally-
serving` ? `gcs`, `WASB_WEIGHTS=/gcs/models/wasb_badminton_best.pth.tar` |
16 | Control-plane **Service** | `rally-control-plane` ? rev 00002, `--no-cpu-throttling`,
`min/maxScale 1/3`, cpu 2 / mem 4Gi, port 8080, **private (IAM-gated, no allUsers)** |
17 | Service image | `asia-southeast1-docker.pkg.dev/?/rally/rally-control-plane:latest` (=
`rally-worker:latest` + `google-cloud-run`; see S5 bug 1) |
18 | Service URL | `https://rally-control-plane-2bosgeukpa-as.a.run.app` |
19 | Config (via base64 startup wrapper ? `RALLY_APP_HOME=/tmp/apphome/config.json`) |
`deployment.profile=cloud-serving`, `compute_target=cloud_run`, `indexing.skip_ai_handoff=true`,
`storage.backend=gcs`, `output_root=gs://khelsutra-rally-serving`, `input_backend=gcs`,
`input_root=?/videos` |
20 | Env | `DEPLOYMENT_PROFILE=cloud-serving`, `CLOUD_RUN_REGION=asia-southeast1`,
`GOOGLE_CLOUD_PROJECT=?`, `RALLY_ALLOWED_HOSTS=*` (CLOUD_RUN_JOB is reserved ? code default
`rally-worker` is correct) |
21
22 **Boot-crash trap avoided:** the `cloud-serving` preset forces
`indexing.skip_ai_handoff=False`
23 (Gemini ON), which would trip the `AI_HANDOFF_KEY_MISSING` startup check (no Gemini key)
and
24 crash-loop. Verified `loader.py:194` deep-merges `defaults < preset < config.json`, so
the
25 explicit `skip_ai_handoff=true` wins. Also pinned `storage.backend=gcs` explicitly to
satisfy the
26 `STORAGE_INCOHERENT` check. Boot log confirmed: `deployment profile ACTIVE: cloud-
serving
27 (compute_target=cloud_run)` + `Uvicorn running on http://0.0.0.0:8080`.
28
29 ## 2. Health / capabilities (authenticated via `gcloud auth print-identity-token`)
30
31 - `GET /api/health` ? `200`
`{"status":"ok","sport":"badminton","default_segmenter":"yolo_hybrid"}`
32 - `GET /api/capabilities` ? `segmenters` incl. `wasb_hybrid`;
`deployment.cloud_available=true`;
33 `gemini_key_present=false` (no crash ? skip_ai_handoff honored); `gcs` backend usable.
34
35 ## 3. gs:// bucket-path CUJ ? PROVEN end-to-end
36

```

```

37      1. `POST /api/process {"video_path":"gs://khelsutra-rally-
serving/videos/MahadevpuraSingles.MP4","segmenter":"wasb_hybrid","compute":"cloud"}` ? `202
{job_id: d16654d1?}`
38      2. Poll `GET /api/jobs/{id}`: `validating ? dispatching (cloud_run) ? done`.
39      - CP pulled + validated the video (format/resolution/pts/scene/blur/relevance all
pass).
40      - Dispatched L4 Job execution `rally-worker-k49kg`; worker log: `WASB in FULLY-LOCAL
mode
41      (skip_ai_handoff=True) ? no gemini handoff, no API key needed`, `torch 1.11.0+cu113
cuda True`.
42      - `**Result: `success` ? "Successfully indexed 48 play segments (cloud_run /
'wasb_hybrid')***;
43      outputs at `gs://khelsutra-rally-serving/outputs/915473?/`.
44      - Wall: created 02:12:42 ? finished 02:26:27 (**~13m45s**, incl. L4 cold image pull +
inference).
45      3. `POST /api/compile {video_id, segment_ids:[1..48],
output_filename:"mahadevpura_wasb_highlights.mp4"}` ? `202 {job_id: d947a9e6?}` ? poll ?
`**success`**. In-process ffmpeg stitch (trim 48 clips w/ padding ? concat ? watermark) on the
CP; `**`Dynamic highlight reel saved ? (166.0 MB)`**`; wall ~3m26s (02:27:47?02:31:13, incl. re-
fetch of the gs:// source).
46      4. `GET /api/highlights/{video_id}/mahadevpura_wasb_highlights.mp4` ? the reel is served
(correct `206`/`video/mp4`/etag/size headers), and the `**full 174,038,655-byte reel was
retrieved via <32 MiB Range requests and ffprobes as a VALID video: H.264 1920x1080 @ 29.97fps +
AAC, duration 244.3s**` (48 rallies). ? `**A single full `GET` returns edge-500**` ? Cloud Run's
~32 MiB response cap vs. the `FileResponse` fixed `Content-Length` (see §5 bug 3).
47
48      ## 4. Notes / observations
49
50      - `**In-process compile serializes behind process jobs**` (drain
`ThreadPoolExecutor(max_workers=1)`):
51      a queued compile waits while a process job holds the worker in its bucket-poll. Fine
for the alpha
52      (low concurrency); worth noting for load.
53      - `**WASB batch tuning is NOT forwarded on the dispatch path.**` `_maybe_dispatch_remote`
forwards only
54      `skip_ai_handoff` (orchestration.py:274-283); the L4 job runs its baseline batch (not
the preset's
55      batch64/prefetch4), so the measured #457 ?30.5% win is not applied via the deployed
CP. Minor
56      (correctness unaffected); follow-up = forward `indexing.wasb.*` overrides like
`skip_ai_handoff`.
57      - `player_pool` / `max_frames` are dropped on the cloud path (already noted at
orchestration.py:273).
58
59      ## 5. Bugs surfaced + handled
60
61      1. `**`google-cloud-run` missing from the image**` (BLOCKER). First real CP deploy failed
every dispatch
62      with `**"google-cloud-run is required for CloudRunCompute's real client" ? the image
installed only
63      `[storage-gcs]`; the reused-as-control-plane role needs `google.cloud.run_v2`.
`**Unblocked**` with a
64      derived image (`rally-worker` + `pip install google-cloud-run`). `**Durable fix: PR
#462**` (adds a
65      `cloud-run` extra + `[storage-gcs,cloud-run]` in the Dockerfile + test guard; 19
tests pass).
66      2. `**Browser-upload ? gs:// staging gap.**` Uploads land on the CP's local FS
(`/api/upload/complete` ?
67      `/tmp/apphome/output/uploads/?`), and `_maybe_dispatch_remote` hard-rejects local
input under
68      cloud-serving (orchestration.py:250-255). Upload endpoints verified working; the gap
is staging.
69      `**Filed: issue #461**` (fix = stage local?gs://input_root before dispatch).
70      3. `**`/api/highlights` download 500s for reels >32 MiB.**` `FileResponse` sets a fixed
`Content-Length`;

```

```

71      Cloud Run caps a single buffered HTTP/1 response at ~32 MiB (empirically: 29.6 MB
Range ? 206,
72      32.4 MB ? 500). App logs 200; Google Frontend returns 500. Every real reel exceeds 32
MiB.
73      **Filed: issue #463** (fix = serve reels via a gs:// signed-URL redirect; also fixes
the ephemeral
74      `/tmp` reel location). Workaround verified: ranged download reassembles a byte-exact,
valid reel.
75
76      ## 6. M9 edge auth ? ENABLED + gate verified (DNS routing remains, owner-handoff)
77
78      Cloudflare Access edge-auth is **ON** (rev `rally-control-plane-00004-tvr`) and
enforcing:
79      - Image rebuilt to add `PyJWT[crypto]` (the `auth` extra) + the `google-cloud-run`
client.
80      - Config: `security.edge_auth_enabled=true`,
`cf_team_domain=https://kheIsutra.cloudflareaccess.com`,
81      `cf_access_aud=c980da67?082b` (the existing "KheIsutra Studio (alpha)" CF Access app
already lists
82      `api.kheIsutra.guru` in `self_hosted_domains` ? no new app needed; same 4-email
allowlist policy),
83      `allowed_video_root=/tmp/apphome/output`.
84      - **Bug found + fixed to make M9 compatible with cloud-serving:** enabling edge-auth
requires
85      `allowed_video_root`, and its jail (`validate_input_path`?`resolve_under_root`)
`realpath()`-mangled
86      and **rejected gs:// inputs** ? so edge-auth would have 400'd the proven gs:// CUJ.
**PR #465** makes
87      the local-path jail skip storage URIs. Overlaid into the deployed image (build-time
asserted);
88      runtime-verified (Phase 1): gs:// ? 202 accepted; a local out-of-root path ? 400 (jail
still enforced).
89      - **Verified (Phase 2):** `/api/health` (exempt) ? 200; `/api/process` without a CF JWT
? **401**
90      ("missing Cf-Access-Jwt-Assertion header"). Service boots (no crash-loop from the
exposure checks).
91
92      **Remaining (owner, CF/GCP dashboard ? outward-facing production DNS):**
`api.kheIsutra.guru` is still a
93      stale CNAME to a dead CF tunnel. To make the alpha reachable-with-auth by allowlisted
testers: (a) add
94      `allUsers`?`roles/run.invoker` on the Cloud Run service (so CF's proxied requests reach
the app ? the
95      app still requires the CF JWT, so direct run.app access stays 401); (b) route
`api.kheIsutra.guru` to the
96      service (Cloud Run domain-mapping in asia-southeast1, or a CF origin-rule Host
override); (c) re-point the
97      CF DNS record (proxied) off the dead tunnel. Then the authed CUJ works end-to-end
through the CF login.
98      Full end-to-end authed CUJ verification needs (a)+(b) (a CF-issued JWT only arrives via
the CF edge).
99
100     Rollback (make it owner-usable again without CF): redeploy with `EDGE_AUTH` unset (edge-
auth off) via
101     `scratchpad/gen_service_yaml.py` ? `gcloud run services replace`.
102
103     ## 7. Cost ledger + teardown
104
105     - One L4 execution (~14 min, 8 vCPU + 1xL4) for the CUJ. CP service holds `min-instances
1` (small
106     always-on CPU cost). Owner directive: **keep running** (closed alpha).
107     - Teardown (if needed): `gcloud run services delete rally-control-plane`, `gcloud run
jobs delete
108     rally-worker`, both `--region asia-southeast1`; bucket/secret/repo retained.
109

```

7. user (2026-07-04T11:56:57.836Z)

```
130     The checker resolves stale absolute paths against the repo's sibling `Annotation Setup`
folders and `output/`, then exits non-zero if any required `trajectory` or `labels` file is
absent.
131
132     A4 nightly baseline refresh record:
133
134     ```powershell
135     python scripts\nightly_regression.py --manifest output\human_lovo_manifest.json
--features-dir output --update-baseline
136     ```
137
138     Result:
139
140     ```text
141     INFO baseline snapshot written to C:\Users\avidu\Projects\khelsutra-guru\badminton-
highlight-indexer\eval_baselines\nightly_baseline.json
142     INFO     default: n=15 tolF1=0.364 f1@0.5=0.472 merge_rate=0.135
143     INFO     milestone: n=15 tolF1=0.364 f1@0.5=0.472 merge_rate=0.135
144     INFO     baseline_off: n=15 tolF1=0.138 f1@0.5=0.237 merge_rate=0.638
145     ```
146
147     Pass-mode proof:
148
149     ```powershell
150     python scripts\nightly_regression.py --manifest output\human_lovo_manifest.json
--features-dir output --no-report
151     ```
152
153     Result: `PASS`, exit code `0`, baseline snapshot `2026-07-03`, default/milestone `tol_f1
= 0.3636`, `f1@0.5 = 0.4723`, `merge_rate = 0.1350`; baseline-off `tol_f1 = 0.1382`, `f1@0.5 =
0.2366`, `merge_rate = 0.6380`.
154
155     A5 heuristic floor refresh record:
156
157     ```powershell
158     python -m backend.eval.calibrate_local --manifest output\human_lovo_manifest.json
--metric f1 --floor-out eval_baselines\heuristic_lovo_n15.json
159     ```
160
161     Result:
162
163     ```text
164     baseline (untuned) mean held-out f1: 0.237
165     calibrated mean held-out f1:          0.446 +/- 0.239 (gap +0.000)
166     ```
167
168     The committed floor is `eval_baselines/heuristic_lovo_n15.json`: `n = 15`, `mean_f1 =
0.4456`, `std = 0.2390`, `metric = f1`, `iou = 0.5`. Every leave-one-video-out fold selected
`(0.05, 1.0, 2.0, 0)`.
169
170     A6 ablation/R2 record:
171
172     ```powershell
173     python -m backend.eval.ablation --features-dir output --granularity group
174     python -m backend.eval.rally_seg_eval --manifest output\human_lovo_manifest.json
--features-dir output --preset served --max-window-duration 0 --inactivity-end 0 --vs served
--vs-max-window-duration 6 --vs-inactivity-end 1.5 --json-out output\a6_r2_ablation.json
175     ```
176
177     Result: the ablation CLI now runs, but full-fusion feature ablation still reports `n=6`
because only the original six feature JSONs carry the full fusion schema; A7 must
regenerate/upconvert the newer nine before fusion/person/audio promotion decisions. The n=15
R2/tolF1 contrast is positive for the served milestone versus baseline-off: baseline/off `tolF1
```

= 0.138`, served `tolF1 = 0.364`, `Delta tolF1 = +0.225` with CI `[+0.165, +0.283]`, sign
`14/0`, `p = 0.000`; `Delta F1@0.5 = +0.236` with CI `[+0.161, +0.313]`; merge rate improves
from `0.638` to `0.135`. Net over-segmentation guard passes (`1.276 -> 0.378`), while the strict
segment-count rule remains blocked by increased splits, so R2/tolF1 is the supported gate for
the baseline-off to served milestone contrast rather than a blanket fusion/default-flip
approval.

178

179 A7 full-fusion/audio feature refresh record:

180

181 ```powershell

182 python -m backend.eval.fusion_golden --manifest output\human_lovo_manifest.json --out-
dir output --source-manifest ..\sports-data-collector\data\golden_source_videos.json --missing-
fusion-only --smallest-first --max-gpu-temp-c 87

183 python -m backend.eval.fusion_audio --manifest output\human_lovo_manifest.json
--features-dir output --source-manifest ..\sports-data-collector\data\golden_source_videos.json
--augment

184 ```

185

186 Result: the GPU pass completed without a thermal abort after fans were set to max
performance. Full-fusion schema is now present for `15/15` feature JSONs, and audio features are
present for `15/15`. The large regenerated clips completed as follows: `mahadevpura\_1` `1`
window / `0` positives / `10` GT, `GX020094` `99` / `30` / `40`, `GX030094` `79` / `21` / `41`,
`GX010141` `26` / `11` / `29`, `GX010137` `56` / `31` / `34`, `Boxhill\_Doubles` `59` / `12` /
`43`, `mahadevpura\_2` `14` / `2` / `40`, `GX010128` `51` / `15` / `52`, `Badminton\_BXH\_2` `29` /
`1` / `44`.

187

188 A3 source-manifest follow-up: `sports-data-collector` PR #62 is merged. The
authoritative collector manifest now records `13/15` present local source/proxy paths and two
Boxhill run-log-only eval inputs: `GX010141\_1080p.mp4` (`daaf8af198640516d14b217f50445d99`) and
`GX010137\_1080p.mp4` (`74eedbc35d739c231344b8929df4b4a7`). The F-drive 4K originals are
preserved there only as `original\_` provenance, not as eval join targets, because the golden
trajectories/labels are 1920-wide. Freshly re-extracting those two Boxhill full-fusion files
requires recovering the exact 1080p eval inputs or adding a coordinate-normalization fix; do not
substitute the 4K originals for person-cue extraction.

189

190 A8 fusion/default-decision record:

191

192 ```powershell

193 python -m backend.eval.fusion_compare --features-dir output

194 python -m backend.eval.fusion_audio --features-dir output --compare

195 python -m backend.eval.ablation --features-dir output --granularity group

196 python -m backend.eval.serve_contrast --manifest output\human_lovo_manifest.json

197 python -m backend.eval.rally_seg_eval --manifest output\human_lovo_manifest.json

--features-dir output --preset served --max-window-duration 0 --inactivity-end 0 --vs served
--vs-max-window-duration 6 --vs-inactivity-end 1.5 --json-out output\A6_r2_ablation.json

198 ```

199

200 Result:

201

202 - Person fusion is **not promoted**: shuttle-only F1 `0.302`, shuttle+person F1 `0.311`,
`Delta F1 +0.008`, paired CI `[-0.040, +0.055]`, sign `8W/6L`, `p = 0.791`.

203 - Audio is **shadow/suggestive, not promoted**: shuttle+person F1 `0.311`,
shuttle+person+audio F1 `0.329`, `Delta F1 +0.018`, paired CI `[-0.017, +0.053]`, sign `10W/4L`,
`p = 0.180`.

204 - Group ablation remains non-significant for cue promotion: audio `+0.018` CI `[-0.017,
+0.053]`; shuttle `+0.013` CI `[-0.002, +0.029]`; gate\_a `-0.001` CI `[-0.012, +0.008]`; person
`-0.009` CI `[-0.045, +0.021]`.

205 - Gate A served default is **rejected**: proposal mean `Delta F1 +0.0287`, CI `[-0.0222,
+0.0680]`; served mean `Delta F1 -0.0328`, CI `[-0.0799, +0.0052]`, `regresses=True`.

206 - R2/cap/inactivity remains the strong quality result: served candidate F1 `0.472` vs
baseline-off `0.237`, `Delta F1 +0.236`, CI `[+0.161, +0.313]`; tolF1 `0.364` vs `0.138`, `Delta
tolF1 +0.225`, CI `[+0.165, +0.283]`; merge rate `0.135` vs `0.638`. Net over-seg guard passes,
strict per-video split-count guard still blocks blanket promotion language.

207

208 A9 Gen-0 shadow-training record:

```

209
210     ```powershell
211     python -m training.gen0.harness --manifest output\human_lovo_manifest.json --floor
eval_baselines\heuristic_lovo_n15.json --out artifacts\rally_tal_gen0 --version
0.1.1-n15-shadow.20260704
212     ```
213
214     Result: current Gen-0 LogReg/TAL harness produced local bundle
`artifacts\rally_tal_gen0\0.1.1-n15-shadow.20260704` with learned LOVO mean F1 `0.551` versus
heuristic floor `0.4456`, so `floor_passed=True`. The gate remains **non-serving**: paired sign
test `9/15`, `p = 0.3036`, `ship=False`; blockers are uncalibrated ship-gate target #172 and
WASB checkpoint provenance C3. TemporalMaxer itself is not implemented in this branch; the
evidence supports either closing A9 as a Gen-0 shadow baseline or opening a follow-up M0.4
scorer-swap PR.
215
216     ## 9. Open questions
217
218     1. **Owner:** Is the first alpha allowed to use manual/offline corpus promotion, or
should P10/P11 be reopened before any tester wave?
219     2. **Owner/platform:** Which hosted compute target is the next stand-up: Cloud Run GPU,
GCE L4 VM, or another approved GPU host?
220     3. **Owner/legal:** What is the acceptable alpha wording while WASB C3 remains
unresolved?
221     4. **Engineering:** Should the portable corpus manifest live as a generated artifact, a
checked-in redacted manifest, or both?
222     5. **Engineering:** Should A9 close on the current Gen-0 non-serving bundle, or should
the actual TemporalMaxer scorer swap be implemented before marking A9 done?
223
224     ## 10. Definition of done
225
226     - [ ] Invited tester can complete the hosted record/upload -> reel download CUJ without
SSH.
227     - [ ] Alpha service is gated behind the chosen auth posture and has a documented
disable/rollback path.
228     - [x] n=15 corpus is portable enough that manifest-driven quality commands do not rely
on stale local absolute paths.
229     - [x] n=15 nightly baseline and heuristic floor are refreshed with recorded commands.
230     - [x] R2/tolF1 gate is promoted or rejected with a recorded ablation result.
231     - [x] Fusion/person/Gate-A served-default decisions are explicitly promoted or rejected
on the 15-video corpus.
232     - [ ] Ship-gate target and allowed alpha claims are recorded.
233     - [ ] WASB C3 status is recorded and reflected in alpha messaging.
234     - [ ] P10/P11 are either reopened with a committed sequence or explicitly deferred for
alpha-plus.
235     - [ ] Owner records GO/NO-GO for the first alpha wave.
236
237     ## 11. References
238
239     Internal docs:
240
241     - `docs/NEXT_STEPS.md`
242     - `docs/ROADMAP.md`
243     - `docs/data_pipeline/GOLDEN_VIDEOS.md`
244     - `docs/R2_EVAL_METRIC_DESIGN.md`
245     - `docs/RALLY_DETECTION_FIXES_PLAN.md`
246     - `docs/COMPUTE_DECOUPLED_SERVING/README.md`
247     - sibling `khelsutra/NEXT_STEPS.md`
248     - sibling `khelsutra/docs/STUDIO_MEDIA_LIFECYCLE.md`
249     - sibling `khelsutra/docs/PER_RALLY_SELECTION.md`
250     - sibling `sports-data-collector/data/golden_source_videos.json`
251
252     Local verification commands used for the 2026-07-03 snapshot:
253
254     ```powershell
255     python -m backend.eval.fusion_compare --features-dir output

```



```

256     python -m backend.eval.serve_contrast --manifest <rebased-n15-manifest>
257     python -m training.gen0.harness --manifest <rebased-n15-manifest> --floor
eval_baselines\heuristic_lovo_n15.json --version analysis-n15 --out <temp-out>
258     python scripts\nightly_regression.py --manifest <rebased-n15-manifest> --features-dir
output --no-report
259     python -m backend.eval.ablation --features-dir output --granularity group
260     python -m backend.eval.rally_seg_eval --manifest output\human_lovo_manifest.json
--features-dir output --preset served --max-window-duration 0 --inactivity-end 0 --vs served
--vs-max-window-duration 6 --vs-inactivity-end 1.5 --json-out output\a6_r2_ablation.json
261     ``
262
263     ---
264
265     ### Changelog
266
267     - 2026-07-03 - Created from a synced five-repo review of `NEXT_STEPS`, `ROADMAP`,
golden-corpus docs, local eval artifacts, and the current 15-video corpus state.
268     - 2026-07-04 - Recorded A7 full-fusion/audio regeneration, A8 n15 fusion/default
decisions, and A9 Gen-0 n15 shadow-training gate verdict.
269     - 2026-07-04 - Marked A3/A7/A8/A9 complete after collector #62 and indexer #458
review/CI; clarified that the two Boxhill eval inputs are run-log-only 1080p hashes and that
F-drive 4K originals are provenance-only.
270

```

8. user (2026-07-04T11:56:58.365Z)

```

1     # Cloud Run GPU **Job** image for the rally worker (COMPUTE_DECOUPLED_SERVING M6/M7,
ADR-012/D16).
2     #
3     # Runs `python -m backend.worker infer ?` INLINE on an L4 (detect + window + stitch co-
located so
4     # CPU/wall + egress are metered into one job ? D4). Built + pushed + verified by the
owner on the
5     # M7 real run (the $100/?10k budget, D17); CI does NOT build this (a CUDA image is too
heavy) ? the
6     # dispatch shim is proven with a fake client (tests/test_compute_backend.py).
7     #
8     # Two python stacks, deliberately separate (a framework conflict otherwise ? same split
as the
9     # native GPU box, deploy/digitalocean/gpu_setup.sh):
10    # * the MAIN app env (this repo + ffmpeg) drives the pipeline + stitching;
11    # * a conda `wasb` env (py3.8 + torch 1.11.0+cu113) runs the WASB shuttle detector as
a subprocess
12    # (the native runner shells out to $WASB_PYTHON), so the app's torch never co-
installs with it.
13    #
14    # ? WASB-C3: the pretrained badminton weights are academic-data-trained ? provenance NOT
cleared for
15    # commercial sharing. They are NOT baked into this image; stage them at runtime (a
bucket fetch or a
16    # mount) and point $WASB_WEIGHTS at them. Resolve WASB-C3 before any public, non-owner
serving.
17
18    FROM nvidia/cuda:12.4.1-runtime-ubuntu22.04
19
20    ENV DEBIAN_FRONTEND=noninteractive \
21        PYTHONUNBUFFERED=1 \
22        PIP_NO_CACHE_DIR=1 \
23        CONDA_DIR=/opt/conda \
24        WASB_REPO_DIR=/opt/models/WASB-SBDT
25
26    # ffmpeg (stitch/decode) + DejaVu fonts (the watermark filter) + git/curl for the WASB
env build.
27    RUN apt-get update && apt-get install -y --no-install-recommends \
28        ffmpeg fonts-dejavu-core git curl ca-certificates build-essential \

```

```

29         python3 python3-pip python3-venv \
30         && rm -rf /var/lib/apt/lists/*
31
32     # --- WASB detector env (micromamba + conda-forge ? BSD-licensed, NO Anaconda ToS; P3a,
33     ADR-005) ---
34     # Enforces ADR-005 (every dependency ? code, data, weights, AND the build toolchain ?
35     stays
36     # MIT/Apache/BSD): micromamba (BSD-3) + conda-forge (community) replaces Miniconda's
37     Anaconda-ToS-gated
38     # `defaults` channels (the ?200-employee Terms-of-Service trap flagged in
39     PACKAGING_PLAN.md §3).
40     # MAMBA_ROOT_PREFIX=$CONDA_DIR (= /opt/conda) keeps the env at $CONDA_DIR/envs/wasb so
41     $WASB_PYTHON below
42     # is UNCHANGED. Same verified torch-1.11.0+cu113 WASB stack; the pip wheels are
43     unaffected by the switch.
44     ENV MAMBA_ROOT_PREFIX=/opt/conda
45     RUN curl -sSL https://micro.mamba.pm/api/micromamba/linux-64/latest \
46         | tar -xj -C /usr/local bin/micromamba \

```

9. assistant (2026-07-04T11:57:13.697Z)

Now let me verify the khelsutra SPA claims (the privacy section's central contradiction) and key engine code cites.

10. user (2026-07-04T11:57:14.671Z)

```

40         className="mt-2 inline-flex min-h-11 items-center text-sm font-medium text-
41         green underline"
42         >
43         {t("wizard.key.getKey")}
44         </a>
45         </>
46         )}
47     </div>
48 );
49 }
50
51 function ComputeStep({ caps }: { caps: EngineCapabilities | null }) {
52     const { t } = useTranslation();
53     const ffmpegOk = caps?.ffmpeg?.ffmpeg === true && caps?.ffmpeg?.ffprobe === true;
54     const gpu = caps?.gpu?.nvidia_smi_present === true;
55     const detector = caps?.default_segmenter ?? "motion";
56     return (
57         <div>
58             <h3 className="font-semibold">{t("wizard.compute.title")}</h3>
59             <p className="mt-2 text-ink/80">{t("wizard.compute.local")}</p>
60             <ul className="mt-3 space-y-1 text-sm text-ink/70">
61                 <li>{t("wizard.compute.detector", { name: detector })}</li>
62                 <li>{ffmpegOk ? t("wizard.compute.ffmpegOk") :
63                 t("wizard.compute.ffmpegMissing")}</li>
64                 <li>{gpu ? t("wizard.compute.gpu") : t("wizard.compute.noGpu")}</li>
65             </ul>
66         </div>
67     );
68 }
69
70 function VideosStep() {
71     const { t } = useTranslation();
72     return (
73         <div>
74             <h3 className="font-semibold">{t("wizard.videos.title")}</h3>
75             <p className="mt-2 text-ink/80">{t("wizard.videos.body")}</p>
76             <p className="mt-2 font-medium text-green">{t("wizard.videos.ready")}</p>
77         </div>
78     );
79 }

```

```

77     }
78
79     export function SetupWizard({ onClose }: { onClose: () => void }) {

```

11. user (2026-07-04T11:57:15.596Z)

```

95         "reloadHint": "You can reload this page ? KhelSutra will keep checking this job.",
96         "retry": "Try again",
97         "reassure": "Safe to wait ? KhelSutra keeps working even if your connection is slow;
a long video can take several minutes.",
98         "statusFailed": "Couldn't process",
99         "chooseFile": "Choose a video file",
100        "uploadAria": "Choose a video file to upload",
101        "fileHint": "Pick an .mp4 or .mov from this device ? it uploads, then KhelSutra
finds the rallies.",
102        "orLink": "Or use a cloud link",
103        "uploadingStatus": "Uploading?",
104        "sourceFile": "File: {filename}",
105        "uploadProgress": "Upload progress",
106        "compute": {
107            "legend": "Where should this run?",
108            "local": "On my computer",
109            "cloud": "In the cloud",
110            "cloudCost": "Cloud processing runs on a rented GPU ? faster for long videos, but
it may cost a little (often just a few cents per match).",
111            "ack": "I understand cloud processing may cost a little.",
112            "ackRequired": "Please accept the note above to run in the cloud."
113        },
114        "medium": {
115            "legend": "Where should we store this video?",
116            "destAria": "Storage destination ? a bucket URL or a Google Drive folder",
117            "bucketPlaceholder": "gcs://my-bucket/videos",
118            "drivePlaceholder": "gdrive://Matches",
119            "destHint": "We keep a durable copy here, keyed to the video ? after checking
there's room.",
120            "destRequired": "Enter a destination to store this video.",
121            "storing": "Storing in your medium?"
122        }
123    },
124    "errors": {

```

12. user (2026-07-04T11:57:23.352Z)

```

485        "capture": "Capture checklist",
486        "privacy": "Privacy",
487        "flyer": "Academy flyer (print):",
488        "trust": "? Served securely over HTTPS · ? Run from Bengaluru, India · Reels carry
a \"Generated by Khelsutra.guru\" mark",
489        "copyright": "© 2026 KhelSutra · Invite-only alpha"
490    },
491    "privacy": {
492        "back": "? Back to KhelSutra",
493        "title": "Privacy Policy",
494        "sub": "How KhelSutra handles your information and your match videos ? and the
choices you have.",
495        "updated": "Last updated: 22 June 2026",
496        "lead": "KhelSutra (\"we\", \"us\") turns a full badminton match into a clean
highlight reel. We're an invite-only alpha run from Bengaluru, India. This page explains, in
plain language, what information we collect, how we use it, who else touches it, and how you
stay in control. Questions? Email <a
href=\"mailto:hello@khelsutra.guru\">hello@khelsutra.guru</a>.",
497        "tocHeading": "On this page",
498        "toc": {
499            "collect": "What we collect",
500            "use": "How we use it",

```

```

501         "video": "Your videos & how they're processed",
502         "sharing": "Who else touches your data",
503         "cookies": "Cookies & analytics",
504         "retention": "How long we keep it",
505         "rights": "Your choices & rights",
506         "minors": "Minors",
507         "security": "Security",
508         "changes": "Changes & contact"
509     },
510     "collect": {
511         "h": "1. What we collect",
512         "intro": "We only collect what we need to run the alpha and produce your
reels:",
513         "li1": "<b>Request-access details</b> ? when you fill in the access form: your
name and email, and optionally your city, academy/club name, how many courts you record, whether
you have a powerful (GPU) computer, where your videos live, and any message you send us.",
514         "li2": "<b>Match videos you choose to give us</b> ? the footage you upload or
share so we can index the rallies and build your reel.",
515         "li3": "<b>Basic technical data</b> ? like your approximate country and browser
type, handled by our hosting and edge-security provider (Cloudflare) to keep the site working
and safe.",
516         "note": "We do <b>not</b> ask for, or want, sensitive personal data, and we
don't run intrusive cross-site advertising trackers."
517     },
518     "use": {
519         "h": "2. How we use it",
520         "li1": "To <b>contact you about the alpha</b> ? but only if you ticked the
consent box on the form.",
521         "li2": "To <b>provide the service</b> ? to index your match, strip the dead
time, and generate the reel you download.",
522         "li3": "To <b>understand demand and improve the product experience</b> in
aggregate (e.g. how many academies vs. individuals are interested).",
523         "note": "We do <b>not</b> sell, rent, or trade your personal data. We do
<b>not</b> use your footage to train or improve our models <b>unless you give us clear,
explicit, opt-in permission</b> ? and even then, never footage involving minors."
524     },
525     "video": {
526         "h": "3. Your videos & how they're processed",
527         "intro": "Your match videos are large files that need real computing to analyse,
so here's exactly what happens to them:",
528         "li1": "We process your video on our own infrastructure to find the rallies and
build your reel.",
529         "li2": "As part of the analysis we may use <b>trusted third-party AI and cloud
services</b> (for example, Google's Gemini API) to detect rallies. When we do, we send only the
footage needed for that step.",
530         "li3": "We <b>never publish</b> your footage, and we never make it public.",
531         "li4": "We never use your footage to improve the tool without your explicit opt-
in (see above).",
532         "boxTitle": "If you run your own model:",
533         "boxBody": "The \"own your model\" path lets advanced users process footage on
their own hardware. In that case your video stays on your machine and is not sent to us or to
third-party AI services."
534     },
535     "sharing": {
536         "h": "4. Who else touches your data",
537         "intro": "We keep the list of processors small and name them honestly:",
538         "li1": "<b>Cloudflare</b> ? hosts the website, secures it at the edge, and
provides privacy-friendly analytics.",
539         "li2": "<b>Google (Gemini API)</b> ? used, when applicable, to analyse footage
and detect rallies.",
540         "li3": "<b>Email delivery</b> ? to send the messages you asked us to send about
the alpha.",
541         "note": "These providers act on our behalf to deliver the service. We don't give
anyone your data to use for their own marketing."
542     },

```

```

543     "cookies": {
544         "h": "5. Cookies & analytics",
545         "intro": "We keep this light:",
546         "li1": "We store a single <b>language preference</b> in your browser so the site
remembers the language you picked. That's a functional preference, not a tracking cookie.",
547         "li2": "We use <b>privacy-first analytics</b> (Cloudflare Web Analytics) to
count visits and see which pages are useful. It does not use tracking cookies and does not build
a profile of you across other sites."
548     },
549     "retention": {
550         "h": "6. How long we keep it",
551         "body": "We keep your information and footage only as long as we need it to run
the alpha and provide the service, and then delete or anonymise it. You can ask us to delete
your uploaded videos and your account details at any time ? see your rights below."
552     },
553     "rights": {
554         "h": "7. Your choices & rights",
555         "intro": "You're in control. At any time you can ask us to:",
556         "li1": "See what personal data we hold about you;",
557         "li2": "Correct anything that's wrong;",
558         "li3": "Delete your data and uploaded footage;",
559         "li4": "Withdraw your consent to be emailed (every email we send also has an
easy opt-out).",
560         "note": "To exercise any of these, just email <a
href=\"mailto:hello@khelsutra.guru\">hello@khelsutra.guru</a> and we'll take care of it."
561     },
562     "minors": {
563         "h": "8. Minors",
564         "body": "We take footage involving minors seriously. Footage involving minors is
<b>never</b> used to improve the tool and is <b>never</b> published ? full stop. If you believe
a minor's personal data has been shared with us inappropriately, contact us and we'll remove
it."
565     },
566     "security": {
567         "h": "9. Security",
568         "body": "The site is served only over <b>HTTPS</b> with an auto-renewing TLS
certificate, so your connection is encrypted. The alpha app itself is gated behind an
authenticated, invite-only sign-in, and access to the processing system is restricted. No system
is perfectly secure, but we take reasonable, layered measures to protect your data."
569     },
570     "changes": {
571         "h": "10. Changes & contact",
572         "body": "This policy reflects our current alpha practices and may change as the
product evolves; we'll update the \"last updated\" date above when it does. For any privacy
question or request, email <a href=\"mailto:hello@khelsutra.guru\">hello@khelsutra.guru</a>."
573     },
574     "authoritative": "This page is provided in several languages for convenience. If
anything differs between versions, the English version is the one that applies.",
575     "home": "Home",
576     "footTrust": "? Served securely over HTTPS · © 2026 KhelSutra · Invite-only alpha"
577 },
578 "meta": {
579     "index": {
580         "title": "KhelSutra ? every rally indexed, the dead time gone"
581     },
582     "capture": {
583         "title": "KhelSutra ? 1-page capture checklist"
584     },
585     "ownModel": {
586         "title": "KhelSutra ? Annotate your footage & train your own model"
587     },
588     "privacy": {
589         "title": "Privacy Policy ? KhelSutra"
590     }
591 }

```

```

592     },
593     "wizard": {
594         "title": "Welcome to KhelSutra",
595         "intro": "Three quick steps and you're ready to turn a match into a highlights
reel.",
596         "step": "Step {n} of 3",
597         "checking": "Checking what this device can do?",
598         "offline": "Couldn't reach KhelSutra ? start it, then reopen setup. You can skip for
now.",
599         "back": "Back",
600         "next": "Next",
601         "skip": "Skip",
602         "done": "Get started",
603         "key": {
604             "title": "AI highlights (optional)",
605             "connected": "Gemini is connected ? you'll get AI-assisted highlights.",
606             "missing": "No AI key yet ? that's fine. KhelSutra works offline with the built-in
motion detector.",
607             "how": "Want sharper highlights? Get a free Gemini key, add GEMINI_API_KEY=? to
the .env file in your KhelSutra folder, then restart.",
608             "getKey": "Get a free Gemini key"
609         },
610         "compute": {
611             "title": "Where it runs",
612             "local": "Everything runs on this device ? your videos never leave it.",
613             "detector": "Detector: {name}",
614             "ffmpegOk": "Video tools (ffmpeg) ready.",
615             "ffmpegMissing": "ffmpeg not found ? install it so KhelSutra can cut and join
clips.",
616             "gpu": "A GPU was detected ? the high-quality shuttle tracker can run here
later.",
617             "noGpu": "No GPU needed ? the offline motion detector runs on any computer."
618         },
619         "videos": {

```

13. assistant (2026-07-04T11:57:33.197Z)

Locale cites check out. Now let me spot-check the engine code cites.

14. user (2026-07-04T11:57:34.616Z)

```

315     cfg = request.app.state.config
316     if cfg is None:
317         return {}
318     return redact_secrets(cfg.model_dump(mode="json"))
319
320
321 @app.get("/api/health")
322 def health(request: Request):
323     """Cheap liveness/readiness probe (no auth) for the SPA's offline banner
324     (HOSTING_PLAN.md:42). Reports the configured sport + default segmenter so the UI can
325     confirm the engine is reachable and which detector is wired. Never raises."""
326     cfg = request.app.state.config
327     sport = getattr(cfg, "sport", None)
328     indexing = getattr(cfg, "indexing", None)
329     return {
330         "status": "ok",
331         "sport": sport,
332         "default_segmenter": getattr(indexing, "default_segmenter", None),
333     }
334
335
336 # P1: the bundled SPA pins this ? the contract version of the /api surface. BUMP when an
existing
337 # endpoint's request/response shape changes incompatibly (purely additive endpoints need

```

```

no bump).
338     API_VERSION = 1
339
340
341     def _engine_version() -> str:
342         """Installed package version (wheel/editable); 'unknown' if not importable as a
distribution."""
343         try:
344             return _pkg_version("badminton-highlight-indexer")
345         except PackageNotFoundError:
346             return "unknown"
347
348
349     @app.get("/api/version")
350     def api_version():
351         """Version handshake so a bundled SPA can detect engine skew (PACKAGING_PLAN.md
P1)."""
352         return {
353             "engine_version": _engine_version(),
354             "api_version": API_VERSION,
355             "ui": _bundled_ui_version(),
356         }
357
358
359     @app.get("/api/capabilities")
360     def capabilities(request: Request):
361         """Honest probe of what THIS box can do ? the first-run wizard renders the compute
step from
362         it (PACKAGING_PLAN.md P1). No secrets: reports only whether a Gemini key is PRESENT,
never its
363         value. Best-effort + never raises (the wizard degrades gracefully)."""
364         cfg = request.app.state.config
365         indexing = getattr(cfg, "indexing", None)
366         deployment = getattr(cfg, "deployment", None)
367         ffmpeg = shutil.which(ffmpeg_bin()) # P2a: honor RALLY_FFMPEG / RALLY_FFPROBE
368         ffprobe = shutil.which(ffprobe_bin())
369         # WASB weights are best-effort: env first (matches native_wasb_runner), then config.
370         wasb = getattr(indexing, "wasb", None)
371         weights_path = (
372             os.environ.get("WASB_WEIGHTS") or getattr(wasb, "weights_path", "") or ""
373         )
374         # Cheap GPU heuristic: nvidia-smi on PATH (matches scripts/doctor.py). Deliberately
NOT
375         # torch.cuda.is_available() ? torch is absent in the LIGHT tier (so it would false-
negative on a
376         # real GPU box) and importing it would add seconds to this probe. The detector
healthcheck is the
377         # authority on whether GPU inference actually works; this is just the wizard's
first-order signal.
378         nvidia_smi = shutil.which("nvidia-smi")
379         return {
380             "api_version": API_VERSION,
381             "engine_version": _engine_version(),
382             "segmenters": sorted(segmenter_registry.list_available().keys()),
383             "default_segmenter": getattr(indexing, "default_segmenter", None),
384             "ffmpeg": {
385                 "ffmpeg": bool(ffmpeg),
386                 "ffprobe": bool(ffprobe),
387                 "ffmpeg_path": ffmpeg,
388                 "ffprobe_path": ffprobe,
389             },
390             "gpu": {"nvidia_smi_present": bool(nvidia_smi)},
391             "gemini_key_present": bool(os.environ.get("GEMINI_API_KEY")),
392             "wasb_weights_present": bool(weights_path) and os.path.exists(weights_path),
393             "deployment": {

```

394 "profile": getattr(deployment, "profile", "") or "",

15. user (2026-07-04T11:57:36.158Z)

```
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\main.py:35:from
fastapi.middleware.cors import CORSMiddleware
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\main.py:122:# CORS
for the web UI. allow_origins='*' with allow_credentials=True is rejected by browsers per the
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\main.py:127:
CORSMiddleware,
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\main.py:128:
allow_origins=_cors_origins_from_env(),
```

16. user (2026-07-04T11:57:43.336Z)

```
100     )
101     from backend.utils.single_instance import acquire_lock # P0e: one instance per database
102
103     app = FastAPI(title="Sports Highlight Indexer API")
104
105
106     def create_app(config: AppConfig, db: Database) -> FastAPI:
107         """Factory: inject shared state via ``app.state`` so route handlers access it
through
108         ``request.app.state`` ? eliminating the Optional module-level globals that forced
109         ``backend.main`` and ``backend.api.job_worker`` into mypy quarantine.
110
111         Also sets the module-level ``global_config`` / ``db_instance`` for transitional
112         compatibility (helpers and tests still reference them). Route handlers use
113         ``request.app.state`` exclusively."""
114         global global_config, db_instance
115         global_config = config
116         db_instance = db
117         app.state.config = config
118         app.state.db = db
119         return app
120
121
122     # CORS for the web UI. allow_origins='*' with allow_credentials=True is rejected by
browsers per the
123     # fetch spec + is unsafe to carry into multi-tenant; this tool uses no cookies, so
credentials stay
124     # off. The origin list is configurable via the env var above (default ["*"]) so a hosted
deploy can
125     # lock it to its origin; the middleware is fixed at startup.
126     app.add_middleware(
127         CORSMiddleware,
128         allow_origins=_cors_origins_from_env(),
129         allow_credentials=False,
130         allow_methods=["*"],
131         allow_headers=["*"],
132     )
133
134
```

17. user (2026-07-04T11:57:44.171Z)

```
905     worker drains it via `claim_due_job` (EXECUTION_POLICY.md P3 self-scheduling loop).
906     """
907     # M9 (D9): edge-auth tenant resolution. DEFAULT-OFF ? owner_id None (the local
single-user,
908     # unchanged). When edge_auth_enabled, the Cf-Access JWT is required + verified (a
missing or
909     # spoofed identity header fails here with 401, before any work) and tags the job's
```



```

owner_id.
910         try:
911             owner_id = edge_auth.resolve_tenant(request.headers, request.app.state.config)
912         except edge_auth.EdgeAuthError as e:
913             raise HTTPException(status_code=401, detail=f"edge auth: {e}") from e
914
915         allowed_root = getattr(
916             getattr(request.app.state.config, "security", None), "allowed_video_root", None
917         )
918         # In hosted mode a bucket URI is allowed only when it's under the configured
input_root (the
919         # designated bucket) ? not an arbitrary bucket the runtime SA happens to reach
(tenant isolation).
920         allowed_uri_root = getattr(
921             getattr(request.app.state.config, "storage", None), "input_root", None
922         )
923         try:
924             validate_input_path(
925                 req.video_path, allowed_root=allowed_root, allowed_uri_root=allowed_uri_root
926             )
927             # M2: resolve the input once (path or storage URI) ? raises ValueError for an
unsupported
928             # scheme, which we map to a clean 400 (an unknown scheme must never 500).
929             input_ref = storage.resolve_input_ref(
930                 req.video_path, getattr(request.app.state.config, "storage", None)
931             )
932         except ValueError as e:
933             raise HTTPException(status_code=400, detail=str(e)) from e
934         # The local-existence fast-fail applies only to a LOCAL input, and stats the
RESOLVED local path
935         # (input_ref.key ? e.g. local:/// stripped to the bare path), not the raw URI
string. A
936         # bucket/Drive URI can't be cheaply os.path.exists()'d ? its existence is verified
when the worker
937         # fetches it (a fetch miss fails the job loudly). validate_input_path above still
runs for every
938         # input: its null-byte guard always applies, and a configured allowed_video_root
(hosted mode)
939         # rejects a non-local URI as out-of-root.
940         if input_ref.backend == "local" and not os.path.exists(input_ref.key):
941             raise HTTPException(
942                 status_code=404, detail=f"Video file not found at '{req.video_path}'"
943             )
944         # P2 (D11): a bucket/Drive source is fetched to a scratch copy on the worker; fail
fast with 507
945         # if this box can't hold that copy. Best-effort ? an unstatable remote
(network/SDK) or an
946         # unreadable temp fs skips the guard (measuring must not deny a legit job); the
fetch itself
947         # still fails loudly downstream if space genuinely runs out. Local inputs need no
copy ? skipped.
948         if input_ref.backend != "local":
949             need = _remote_input_size_bytes(
950                 input_ref, getattr(request.app.state.config, "storage", None)
951             )
952             require_free_bytes(tempfile.gettempdir(), need, stage="fetch")
953         if req.segmenter and not segmenter_registry.get(req.segmenter):
954             available = list(segmenter_registry.list_available().keys())
955             raise HTTPException(
956                 status_code=400,
957                 detail=f"Segmenter '{req.segmenter}' not found. Available segmenters:
{available}",
958             )
959
960         job_id = uuid.uuid4().hex

```

```

961         if not request.app.state.db.create_job(
962             job_id,
963             req.video_path,
964             req.segmenter,

```

18. user (2026-07-04T11:57:54.068Z)

```

1005         return agg
1006
1007
1008     @app.get("/api/jobs/{job_id}")
1009     def get_job(job_id: str, request: Request):
1010         """Job state: {status, progress, result, reports}. `status` = execution state
1011         (queued|running|done|failed); the pipeline verdict is result["status"]."""
1012         job = request.app.state.db.get_job(job_id)
1013         if not job:
1014             raise HTTPException(status_code=404, detail=f"Unknown job_id '{job_id}'")
1015         result = job.get("result")
1016         return {
1017             "job_id": job["id"],
1018             "status": job["status"],
1019             "progress": job.get("progress"),
1020             "video_id": job.get("video_id"),
1021             "error": job.get("error"),
1022             "result": result,
1023             "reports": (result or {}).get("reports"),
1024             "created_at": job.get("created_at"),
1025             "started_at": job.get("started_at"),
1026             "finished_at": job.get("finished_at"),
1027         }
1028
1029
1030     # --- Chunked upload (B2, PR-2) -----
1031     # The chunked-upload island (in-process registry + helpers + the 4 endpoints) now lives
1032     # in
1033     # backend.api.uploads as a self-contained APIRouter. It resolves the live startup config
1034     # via
1035     # backend.main.global_config, so behavior ? route paths, status codes, body-size limits,
1036     # sequential-part logic, and abort cleanup ? is unchanged.
1037     from backend.api import uploads as uploads_api
1038
1039     app.include_router(uploads_api.router)
1040
1041     # --- Highlight download (B3, PR-2) -----
1042
1043     @app.get("/api/highlights/{video_id}/{filename}")
1044     def download_highlight(video_id: str, filename: str, request: Request):
1045         """Stream a compiled highlight reel ? `filename` is the bare name given to
1046         /api/compile
1047         (which only writes into output_dir; the browser previously got back an unreachable
1048         server-local path)."""
1049         try:
1050             name = safe_output_filename(filename)
1051             except ValueError as e:
1052                 raise HTTPException(status_code=400, detail=str(e)) from e
1053             if not request.app.state.db.get_video(video_id):
1054                 raise HTTPException(status_code=404, detail="Indexed video record not found.")
1055             # #463: when the reel was mirrored to the output bucket, 307 to a short-lived signed
1056             URL (bytes flow
1057             # client<->store, bypassing Cloud Run's ~32 MiB response cap + the ephemeral local
1058             /tmp). The
1059             # exists/sign logic (with the active storage config) lives in
1060             orchestration.signed_reel_url to keep

```

```

1057     # this module lean; best-effort ? any error falls back to the local file below.
1058     try:
1059         signed = signed_reel_url(request.app.state.config, video_id, name)
1060         if signed:
1061             return RedirectResponse(signed, status_code=307)
1062     except Exception as e: # noqa: BLE001 ? signing/exists hiccup ? serve the local
file instead
1063         logger.warning("[highlights] signed-url path failed for %s; serving local: %s",
video_id, e)
1064     output_dir = (
1065         getattr(
1066             getattr(request.app.state.config, "output", None), "output_dir", "output"
1067         )
1068         or "output"
1069     )
1070     path = os.path.join(output_dir, name)
1071     try:
1072         path = resolve_under_root(path, output_dir)
1073     except ValueError as e:
1074         raise HTTPException(status_code=400, detail=str(e)) from e
1075     if not os.path.isfile(path):
1076         raise HTTPException(
1077             status_code=404,
1078             detail=f"No compiled file named '{name}'. Compile first via /api/compile.",
1079         )
1080     media = "video/quicktime" if name.lower().endswith(".mov") else "video/mp4"
1081     return FileResponse(path, media_type=media, filename=name)
1082
1083
1084 @app.post("/api/query")
1085 def query_play_segments(req: QueryRequest, request: Request):
1086     filters = {
1087         "server": req.server,
1088         "receiver": req.receiver,
1089         "duration_min": req.duration_min,
1090         "event_type": req.event_type,
1091     }
1092     segments = request.app.state.db.query_play_segments(req.video_id, filters)
1093     return {"play_segments": segments}
1094
1095
1096 @app.post("/api/compile", status_code=202)
1097 def compile_highlights(req: CompileRequest, request: Request):
1098     """Submit a reel-COMPILE job (202 + job_id); poll GET /api/jobs/{job_id} for the
reel (#329).
1099
1100     Async, mirroring /api/process: the ffmpeg stitch (trim + re-encode each clip +
concat + watermark)
1101     can run minutes on CPU ? past the ~100 s edge/proxy timeout ? so doing it
synchronously returned a
1102     Cloudflare 524 while the engine kept going (the reel built, but the client saw
"failed" and never
1103     got the URL). The cheap checks (video exists, segments match + survive the floor,
safe filename)
1104     fast-fail HERE with a 4xx; the stitch runs on the worker. On `done`, the job's
`result` carries
1105     {filepath, download_url} ? the same shape the sync endpoint returned, now under
/api/jobs/{id}."""
1106     # M9 edge-auth tenant (DEFAULT-OFF ? None); a missing/spoofed identity fails 401
before queuing.
1107     try:
1108         owner_id = edge_auth.resolve_tenant(request.headers, request.app.state.config)
1109     except edge_auth.EdgeAuthError as e:
1110         raise HTTPException(status_code=401, detail=f"edge auth: {e}") from e
1111     try:

```

```

1112         video, _kept, out_name = _resolve_compile(
1113             request.app.state.db,
1114             request.app.state.config,
1115             req.video_id,
1116             req.segment_ids,
1117             req.output_filename,
1118         )
1119     except _CompileError as e:

```

19. user (2026-07-04T11:58:02.724Z)

```

50     # (main.py imports this router; this module must not import main at import time).
51     import backend.main as _main
52
53     sec = getattr(_main.global_config, "security", None)
54     upload_dir = resolve_output_dir(
55         getattr(sec, "upload_dir", None) or "output/uploads"
56     ) # P0a: app-home-rooted (CWD-identical when RALLY_APP_HOME unset)
57     max_bytes = int(getattr(sec, "max_upload_mb", 4096) or 4096) * 1024 * 1024
58     part_bytes = int(getattr(sec, "max_part_mb", 95) or 95) * 1024 * 1024
59     exts = [
60         e.lower().lstrip(".")
61         for e in (getattr(sec, "allowed_upload_extensions", None) or ["mp4", "mov"])
62     ]
63     return upload_dir, max_bytes, part_bytes, exts
64
65
66     def _abort_upload(upload_id: str) -> None:
67         with _uploads_lock:
68             st = _uploads.pop(upload_id, None)
69             if st:
70                 try:
71                     os.remove(st["tmp_path"])
72                 except OSError:
73                     pass
74
75
76     def _normalize_upload_filename(filename: str) -> str:
77         """Normalize browser-provided file names before extension validation.
78
79         Some mobile/file-provider pickers can hand us visually normal names with
80 compatibility
81 characters (for example a fullwidth dot) or trailing whitespace/dots. Keep the
82 existing
83 traversal/null-byte checks in ``safe_output_filename`` as the authority after this
84 cleanup.
85
86         """
87         return unicodedata.normalize("NFKC", filename).strip().rstrip(". ")
88
89
90     @router.post("/api/upload/init")
91     def upload_init(req: UploadInitRequest):
92         """Start a chunked upload. Returns upload_id; PUT sequential parts next."""
93         upload_dir, max_bytes, part_bytes, exts = _upload_cfg()
94         try:
95             name = safe_output_filename(_normalize_upload_filename(req.filename))
96         except ValueError as e:
97             raise HTTPException(status_code=400, detail=str(e)) from e
98         ext = os.path.splitext(name)[1].lower().lstrip(".")
99         if ext not in exts:
100             raise HTTPException(
101                 status_code=400,
102                 detail=f"Extension '{ext}' not allowed. Allowed: {sorted(exts)}",
103             )
104         if req.total_size_bytes is not None and req.total_size_bytes > max_bytes:

```

```

101         raise HTTPException(
102             status_code=413,
103             detail=f"File exceeds the {max_bytes // (1024 * 1024)} MB upload limit.",
104         )
105     os.makedirs(upload_dir, exist_ok=True)
106     # P2 (D11): fail fast with 507 if the box can't hold the declared upload. Best-
107     effort ? an
108     # undeclared size (total_size_bytes=None) or an unreadable disk skips the guard
109     (never blocks a
110     # legit upload); the per-part total cap in upload_part is the hard backstop either
111     way.
112     require_free_bytes(upload_dir, req.total_size_bytes, stage="upload")
113     upload_id = uuid.uuid4().hex
114     tmp_path = os.path.join(upload_dir, f".partial-{upload_id}")
115     open(tmp_path, "wb").close()
116     with _uploads_lock:
117         _uploads[upload_id] = {
118             "filename": name,
119             "tmp_path": tmp_path,
120             "parts": 0,
121             "bytes": 0,
122         }
123     return {
124         "upload_id": upload_id,
125         "max_part_mb": part_bytes // (1024 * 1024),
126         "next_part": f"/api/upload/{upload_id}/part/0",
127     }
128
129 @router.put("/api/upload/{upload_id}/part/{index}")
130 async def upload_part(upload_id: str, index: int, request: Request):
131     """Append one part (raw bytes body). Parts are strictly sequential (0,1,2,...)."""
132     _, max_bytes, part_bytes, _ = _upload_cfg()
133     with _uploads_lock:
134         st = _uploads.get(upload_id)
135         if st is None:
136             raise HTTPException(status_code=404, detail=f"Unknown upload_id '{upload_id}'")
137         if index != st["parts"]:
138             raise HTTPException(
139                 status_code=409,
140                 detail=f"Out-of-order part: expected {st['parts']}, got {index} (parts are
141                 sequential)",
142             )
143         received = 0
144         overflow = None
145         with open(st["tmp_path"], "ab") as fh:
146             async for chunk in request.stream():
147                 received += len(chunk)
148                 if received > part_bytes:
149                     overflow = (
150                         f"Part exceeds the {part_bytes // (1024 * 1024)} MB per-part limit."
151                     )
152                     break
153                 if st["bytes"] + received > max_bytes:
154                     overflow = (
155                         f"Upload exceeds the {max_bytes // (1024 * 1024)} MB total limit."
156                     )
157                     break
158                 fh.write(chunk)
159         if overflow:
160             _abort_upload(upload_id)
161             raise HTTPException(
162                 status_code=413, detail=overflow + " Upload aborted ? re-init to retry."
163             )

```

```

162
163     with _uploads_lock:
164         st["parts"] += 1
165         st["bytes"] += received
166     return {
167         "upload_id": upload_id,
168         "received_parts": st["parts"],
169         "received_bytes": st["bytes"],
170     }
171
172
173     @router.post("/api/upload/{upload_id}/complete")
174     def upload_complete(upload_id: str, req: UploadCompleteRequest):
175         """Finalize: verify part count, move into place, return {path, video_id} for
176         /api/process."""
177         upload_dir, _, _, _ = _upload_cfg()
178         with _uploads_lock:
179             st = _uploads.get(upload_id)
180             if st is None:
181                 raise HTTPException(status_code=404, detail=f"Unknown upload_id '{upload_id}'")
182             if st["parts"] == 0 or st["parts"] != req.total_parts:
183                 raise HTTPException(
184                     status_code=409,
185                     detail=f"Expected {req.total_parts} part(s), received {st['parts']}.",
186                 )
187             final_path = os.path.join(upload_dir, st["filename"])
188             if os.path.exists(
189                 final_path
190             ): # never overwrite an earlier upload with the same name

```

20. user (2026-07-04T11:58:03.536Z)

```

38         return None
39     if hasattr(storage_config, "model_dump"):
40         return storage_config.model_dump()
41     return storage_config if isinstance(storage_config, dict) else None
42
43
44     def consent_attested(job: Dict[str, Any], config: Any = None) -> bool:
45         """The flywheel's consent gate ? currently a **FAKED hard-deny** (always ``False``).
46
47         The flywheel may capture a user's footage into the training pool ONLY with attested,
48         **adults-only**
49         consent (D10 / the CLAUDE.md guardrail: minors excluded; per-user training is a
50         separate explicit
51         opt-in; paid tier never trains). The REAL gate ? reading counsel-defined consent
52         columns + an
53         adults-only attestation + a retention class ? lands with **M9-consent**; until then
54         this denies
55         every job so the harness captures nothing. **Do not relax without counsel sign-
56         off.**"""
57         return False
58
59
60     def capture_job(
61         workspace: "_workspace.VideoWorkspace",
62         *,
63         rally_output: Dict[str, Any],
64         source_path: Optional[str] = None,
65         storage_config: Any = None,
66     ) -> Dict[str, str]:
67         """Durably persist a consented job's artifacts into its per-video ``workspace`` (the
68         raw material a

```

21. user (2026-07-04T11:58:16.577Z)

```

35     # scales to deployment.max_concurrent_remote_jobs to overlap those. This does NOT
parallelize local
36     # work: CPU/GPU-heavy in-process detection + the compile stitch serialize via
37     # orchestration._LOCAL_COMPUTE_LANE regardless of the worker count, so the single-box
invariant holds
38     # and compile/local-override jobs stay serial. The count is fixed at first
_get_executor() creation.
39     #
40     # ? 02 RISK (CODE_CLEANUP_AUDIT §3c): a single worker means ONE stuck or hung job
41     # (e.g. hung Gemini generate, hung WSL GPU call, hung Cloud Run poll) blocks ALL
42     # subsequent jobs ? they stay 'queued' forever. Mitigations already in place:
43     #   - ExecutionPolicy op_timeout_sec kills hung generate/processing calls
44     #   - detector timeout_sec kills hung WSL/GPU calls
45     #   - Cloud Run poll has a max_wait_sec bound
46     #   - ffmpeg/ffprobe subprocesses use bounded media timeouts
47     # A future slice should add a per-job wall-clock timeout (the executor itself
48     # cannot enforce timeouts on the Thread).
49     _job_executor: Optional[ThreadPoolExecutor] = None
50     # Drain worker count, fixed at first _get_executor() creation. 1 (serial) by default;
configure_drain()
51     # raises it for cloud_run (#466). A module global (not a _get_executor arg) so the many
call sites +
52     # the tests that monkeypatch `_get_executor` as a no-arg lambda are unchanged.
53     _drain_max_workers: int = 1
54
55
56     def _cloud_dispatch_possible(config: Any = None) -> bool:
57         """True if this server CAN dispatch a job to Cloud Run ? either the profile sets
58         ``deployment.compute_target=cloud_run``, OR the per-request ``compute='cloud'``
override path is
59         wired (``CLOUD_RUN_JOB`` + ``GOOGLE_CLOUD_PROJECT`` env + an output bucket). Mirrors

```

22. user (2026-07-04T11:58:17.700Z)

```

140         compute=job.get("compute"),
141         force=bool(params.get("force")),
142     )
143
144
145     def _run_claimed_job(job: Dict[str, Any], config: Any, db: Any) -> None:
146         """Run ONE job already claimed (`status='running'`) by `claim_due_job`, recording
its
147         terminal state ? OR parking it for resume on `JobWaiting` (WAIT_AND_RESUME).
148
149         Job `status` is the EXECUTION state (queued|waiting|running|done|failed): 'done'
means
150         the worker finished ? the pipeline's own verdict (validation failure etc.) lives in
151         result["status"]; 'failed' means the worker itself crashed/raised; 'waiting' means
the
152         job self-scheduled and will be re-claimed when `next_poll_at` is due.
153         """
154         import backend.main as _main
155
156         job_id = job["id"]
157         # Dispatch by kind: 'compile' (#329, the async reel stitch) vs 'process'
(NULL/default ? the
158         # original /api/process pipeline). Both record their terminal state the same way;
only process
159         # jobs feed the M8 cost ledger + M11 flywheel (a compile has no detection
usage/segments).
160         is_compile = job.get("kind") == "compile"
161         try:
162             if is_compile:
163                 result = _main._execute_compile(
164                     config,

```

```

165         db,
166         job,
167         progress=lambda stage: db.set_job_progress(job_id, stage),
168     )
169     else:
170         req = _main._job_to_request(job)
171         result = _main._execute_processing(
172             config,
173             db,
174             req,
175             progress=lambda stage: db.set_job_progress(job_id, stage),
176         )
177         if result.get("video_id"):
178             db.set_job_video_id(job_id, result["video_id"])
179         db.mark_job_done(job_id, result)
180         if not is_compile:
181             _maybe_record_job_cost(
182                 job, result, config, db
183             ) # M8 cost ledger (default-OFF; process jobs only)
184             _maybe_run_flywheel(
185                 job, result, config
186             ) # M11 data-flywheel (process jobs only)
187     except _main.JobWaiting as w:
188         # Cooperative re-enqueue: persist progress + next_poll_at, release the worker.
189         logger.info(f"Job {job_id} parked for {w.delay_sec:.0f}s (resume when due).")
190         db.set_job_waiting(job_id, w.delay_sec, progress=w.progress)
191     except Exception as e:
192         logger.error(f"Job {job_id} crashed: {e}\n{traceback.format_exc()}")
193         db.mark_job_failed(job_id, f"{type(e).__name__}: {e}")
194
195
196     def _maybe_record_job_cost(
197         job: Dict[str, Any], result: Dict[str, Any], config: Any, db: Any
198     ) -> None:
199         """M8 cost ledger (COMPUTE_DECOUPLED_SERVING M8, D8). DEFAULT-OFF: a no-op unless

```

23. user (2026-07-04T11:58:26.671Z)

```

42         """An op hung past ``op_timeout_sec`` or never became ready within
43         ``max_wait_sec``."""
44
45         @dataclass(frozen=True)
46         class ExecutionPolicy:
47             """Declarative resilience knobs, attachable per job/op (JSON-serialisable; see
48             EXECUTION_POLICY.md)."""
49
50             mode: WaitMode = WaitMode.WAIT_AND_RESUME
51             poll_every_n_sec: float = 10.0
52             op_timeout_sec: float = (
53                 120.0 # HARD per-attempt deadline ? the hang-killer that was missing
54             )
55             max_wait_sec: float = 1800.0 # total wall budget for a poll loop
56             max_retries: int = 5 # retries (=> max_retries+1 attempts) for failures/hangs
57             backoff_base_sec: float = 3.0 # exponential: backoff_base * 2**attempt
58             on_timeout: str = "reschedule" # reschedule | partial | fail (consumed by P2/P3)
59             resumable: bool = True

```

24. user (2026-07-04T11:58:27.711Z)

```

175         if container_overrides is None
176         else tuple(container_overrides)
177     )
178

```



```

179     def run_infer(self, spec: ComputeJobSpec) -> ComputeResult:
180         out_root = spec.out_uri.rstrip("/")
181         token = self._token or uuid.uuid4().hex
182         done_uri = f"{out_root}/_dispatch/{token}.done"
183         argv: List[str] = [
184             "infer",
185             "--video-uri",
186             spec.video_uri,
187             "--out-uri",
188             spec.out_uri,
189             "--done-uri",
190             done_uri,
191         ]
192         if spec.segmenter:
193             argv += ["--segmenter", spec.segmenter]
194         for kv in (*spec.config_overrides, *self._container_overrides):
195             argv += ["--set", kv]
196
197         execution = self._client.run_job(
198             self._job_name, argv, region=self._region, project=self._project
199         )
200         LOG.info(
201             "cloud-run: dispatched job=%s exec=%s; bucket-polling %s",
202             self._job_name,
203             execution,
204             done_uri,
205         )
206
207         try:
208             marker = poll_until(
209                 lambda: self._read_marker(done_uri),
210                 self._policy,
211                 label="cloud-run-infer",
212                 logger=LOG,
213             )
214         except PolicyTimeout as timeout_exc:
215             marker = self._handle_poll_timeout(done_uri, str(execution), timeout_exc)
216             video_id = str(marker.get("video_id", ""))
217             # A MISSING returncode defaults to FAILURE (1), never success ? a present-but-
unreadable /
218             # garbled marker must not masquerade as rc=0. (The worker always writes both
fields.)
219             rc = int(marker.get("returncode", 1))
220             if not video_id:
221                 LOG.warning(
222                     "cloud-run: job=%s completion marker present but empty/unreadable (no "
223                     "video_id) ? treating as failure",
224                     self._job_name,
225                 )
226                 rc = rc or 1
227             outputs_uri = f"{out_root}/outputs/{video_id}/" if video_id else ""
228             report = self._read_json(f"{outputs_uri}report.json") if outputs_uri else {}
229             LOG.info(
230                 "cloud-run: job=%s done (video_id=%s rc=%d)", self._job_name, video_id, rc
231             )
232             return ComputeResult(
233                 returncode=rc,
234                 outputs_uri=outputs_uri,
235                 video_id=video_id,
236                 report=report,
237                 execution=str(execution or ""),
238             )
239
240     def _handle_poll_timeout(
241         self, done_uri: str, execution: str, timeout_exc: PolicyTimeout

```

```

242         ) -> dict:
243             """Poll deadline hit ? rescue a just-finished result, else cancel + re-raise.
244
245             The poll budget (`max_wait_sec`, default 1800s) can undercut the deployed
246             job's own ``--task-timeout`` (3600s), so a slow-but-healthy worker may finish just past
247             the deadline:
248             ONE final marker check rescues that completed GPU run instead of wasting it. If
249             the marker
250             is genuinely absent, the execution is still running ? best-effort cancel it
251             (otherwise the
252             L4 keeps billing until ``--task-timeout``) and re-raise a ``PolicyTimeout``
253             carrying the
254             execution name so the operator can verify the state in the GCP console. A cancel
255             failure
256             NEVER masks the original timeout."""
257         try:
258             late = self._read_marker(done_uri)
259         except Exception: # noqa: BLE001 ? the final check is best-effort too: absence
260             ? failure
261             LOG.warning(
262                 "cloud-run: final post-timeout marker check failed ? treating as
263                 absent",
264                 exc_info=True,
265             )
266             late = None
267         if late is not None:
268             LOG.warning(
269                 "cloud-run: job=%s exec=%s done-marker found on the final post-timeout
270                 check "
271                 "? rescuing the completed result",
272                 self._job_name,
273                 execution,
274             )
275             return late
276         try:
277             self._client.cancel_execution(execution)
278         except Exception as cancel_exc: # noqa: BLE001 ? best-effort: never mask the
279             timeout
280             cancel_note = f"best-effort cancel FAILED ({cancel_exc})"
281             LOG.warning(
282                 "cloud-run: best-effort cancel of execution=%s failed: %s",
283                 execution,
284                 cancel_exc,
285                 exc_info=True,
286             )
287         else:
288             cancel_note = "cancel requested"
289             LOG.info("cloud-run: requested cancel of execution=%s", execution)
290             raise PolicyTimeout(
291                 f"{timeout_exc} [cloud-run job={self._job_name} "
292                 f"execution={execution or '<unknown>'}; {cancel_note} ? verify the execution
293                 "
294                 "state in the GCP console]"
295             ) from timeout_exc
296
297     # --- storage-backed completion signal (bucket-poll) ---
298     def _read_marker(self, done_uri: str) -> Optional[dict]:
299         """The done-marker dict if the worker has written it, else None ? poll_until
300         keeps waiting."""

```

25. user (2026-07-04T11:58:49.997Z)

```

{"number":471,"state":"OPEN","title":"cloud-serving: control-plane state (SQLite jobs/videos DB)
is per-instance ephemeral ? maxScale>1 splits the queue and /api/jobs polls"}

```

```

{"number":472,"state":"OPEN","title":"ops: api.khelsutra.guru Google-managed cert auto-renew (~90d) can stall behind the proxied CF CNAME ? verify before ~2026-09-30"}
{"number":301,"state":"OPEN","title":"T11 follow-up: run the upload retention sweep as an automated scheduled job (hosted alpha)"}
{"number":332,"state":"OPEN","title":"[safety] Content-safety pre-flight litmus: flag non-sports / NSFW video BEFORE sending it to Gemini"}
{"number":326,"state":"OPEN","title":"Hosted-app observability: request tracing + replay for debugging production CUJ failures"}
{"number":328,"state":"OPEN","title":"Surface the per-video runlog / execution trace to the user (endpoint + UI), not just server logs"}
{"number":447,"state":"OPEN","title":"Add API support to delete a video from the Studio library"}
{"number":443,"state":"OPEN","title":"[security] Default CORS allow_origins=['*'] lets any site script the localhost appliance (R1-12)"}
{"number":327,"state":"OPEN","title":"Allow gs:// / s3:// / gdrive-api:// ingest on an exposed (jailed) box ? scheme-exempt recognized storage backends from the path-jail"}
{"number":175,"state":"OPEN","title":"Shared cloud container (OneDrive) for GPU-extractor golden data ? sync GPU box ? dev sessions"}
{"number":331,"state":"OPEN","title":"[perf] Hosted processing is CPU-bound (ffmpeg/cv2) ? measured per-phase breakdown + levers (GPU/NVENC, parallel extraction, async compile)"}
{"number":349,"state":"OPEN","title":"WASB native: decode is CPU-bound & single-threaded in streaming mode ? L4 GPU ~80% idle (no NVDEC/parallel decode)"}
{"number":172,"state":"OPEN","title":"Calibrate the ship-gate F1@tIoU target (currently provisional ? NOT owner-ratified)"}
{"number":465,"state":"MERGED","title":"fix(hosted): allowed_video_root jail must not reject storage URIs (gs://)"}
{"number":476,"state":"MERGED","title":"fix(storage): GCS keyless signing mints a cloud-platform-scoped token for signBlob (closes #475)"}
{"number":470,"state":"MERGED","title":"deploy(cloudrun): bake the auth extra (PyJWT) into the base image (M9-ready rebuilds)"}
{"number":461,"state":"CLOSED","title":"cloud-serving: browser-uploaded local files not staged to gs:// before L4 dispatch (upload CUJ fails)"}
{"number":463,"state":"CLOSED","title":"cloud-serving: /api/highlights download 500s for reels >32 MiB (Cloud Run response limit)"}

```

26. user (2026-07-04T11:59:03.791Z)

```

110      2. **Build the derived control-plane image** = base + PyJWT: context `Dockerfile` =
`FROM
111      asia-southeast1-docker.pkg.dev/gen-lang-client-0306026826/rally/rally-worker:latest`
+
112      `RUN python3 -m pip install --no-cache-dir "google-cloud-run>=0.10,<1"
"PyJWT[crypto]>=2.8"`; tag
113      `.../rally/rally-control-plane:latest`. `gcloud builds submit <ctx> --tag <img>
--region asia-southeast1`.
114      (No more paths.py/backend overlays needed once the fixes are IN the base image ? the
prior session
115      overlaid #465's OLD blanket paths.py into the live image; the rebuild replaces it
with the corrected code.)
116      3. **Deploy CP with edge-auth OFF** so you can drive it with an identity token (no CF
login needed for
117      the test). Regenerate the service YAML (config below) with `EDGE_AUTH` unset and
118      `gcloud run services replace <yaml> --region asia-southeast1`. Run gcloud from
**PowerShell**.
119      4. **Run the CUJ + verify** (private service ? auth every curl with
120      `-H "Authorization: Bearer $(gcloud auth print-identity-token)"; URL below):
121      - `POST /api/process {"video_path":"gs://khelsutra-rally-
serving/videos/MahadevpuraSingles.MP4","segmenter":"wasb_hybrid","compute":"cloud"}`
122      ? poll `/api/jobs/{id}` to `done` (L4 run ~13 min) ? note segment ids.
123      - `POST /api/compile
{"video_id":<id>,"segment_ids":[1..N],"output_filename":"e2e.mp4"}` ? poll to done.
124      - **463 check:** `GET /api/highlights/{video_id}/e2e.mp4` should now
**307-redirect** to a
125      `storage.googleapis.com` signed URL, and `curl -L` should download the FULL >32 MiB
reel (no edge-500).

```

```

126         Also confirm the reel is mirrored at `gs://khelsutra-rally-
serving/highlights/<video_id>/e2e.mp4`.
127     - **#461 check:** upload a small mp4 via `/api/upload/init`?`/part/0`?`/complete`,
then `POST /api/process`
128         with the returned LOCAL path + `compute:cloud` ? it should STAGE to gs:// and
dispatch (not hard-fail
129         with "cloud-serving needs a cloud input"). (Signing was already verified: `gcloud
storage sign-url`
130         impersonating the compute SA produced a working URL ? the SA has
self-`serviceAccountTokenCreator`.
131     5. **Restore edge-auth ON** (regenerate the YAML with `EDGE_AUTH=1` + redeploy) so
api.khelsutra.guru's
132         live posture is back. Verify `/api/process` sans CF JWT ? 401 and
`https://api.khelsutra.guru/api/health`
133         ? 302 to the CF login.
134     6. **Write** `docs/COMPUTE_DECOUPLED_SERVING/runlogs/DEPLOY_REPORT_cloud-
serving_a1-e2e-<date>.md` (a NEW
135         file ? don't clobber the existing #457 or #464 runlogs), update `[[current-state]]` +
this handoff, and
136         **flip A1 ? DONE** in `docs/ALPHA_LAUNCH_READINESS.md` §7 (all DoD met: endpoint +
health + edge-auth +
137         upload?process?jobs?compile?download).
138
139     **Config for the CP service (base64 into a startup wrapper via
`RALLY_APP_HOME=/tmp/apphome/config.json`):**
140     ```json
141     {"deployment":{"profile":"cloud-
serving","compute_target":"cloud_run"},"indexing":{"skip_ai_handoff":true},
142     "storage":{"backend":"gcs","output_root":"gs://khelsutra-rally-
serving","input_backend":"gcs",
143     "input_root":"gs://khelsutra-rally-serving/videos"},
144     "security":{"edge_auth_enabled":<EDGE_AUTH>,"cf_team_domain":"https://khelsutra.cloudflare
reaccess.com",
145     "cf_access_aud":"c980da67ed88ffff935fcd913ae1eb4da63f9d65805d6e1c5ca76f92f517082b",
146     "allowed_video_root":"/tmp/apphome/output"}}
147     ```
148     CP deploy (per `deploy/cloudrun/README.md` §3a): `--command /bin/bash --args` a `-c`
wrapper that
149     `printf %s '<b64>' | base64 -d > /tmp/apphome/config.json && export
RALLY_APP_HOME=/tmp/apphome && exec
150     python3 -m backend.main --server --host 0.0.0.0 --port 8080`; flags `--no-cpu-throttling
--min-instances 1
151     --cpu 2 --memory 4Gi --port 8080 --timeout 300`; env `DEPLOYMENT_PROFILE=cloud-serving,
152     CLOUD_RUN_REGION=asia-southeast1,GOOGLE_CLOUD_PROJECT=gen-lang-
client-0306026826,RALLY_ALLOWED_HOSTS=`.
153     ? `CLOUD_RUN_JOB` is a RESERVED env name ? DON'T set it (code default `rally-worker` is
correct).
154     Deploy via a `gcloud run services replace <service.yaml>` MANIFEST (shell metachars in
the wrapper break
155     through the cmd-shim otherwise). The prior session's generator `gen_service_yaml.py` +
the derived-image
156     `Dockerfile` lived in the prior scratchpad ? rebuild them from the config + notes above.
157
158     **Deployed state (live now ? owner said KEEP running)
159     - Project `gen-lang-client-0306026826`, region `asia-southeast1`. Compute SA
160     `492532266377-compute@developer.gserviceaccount.com` (has `roles/editor` +
self-`serviceAccountTokenCreator`).
161     - **L4 Job** `rally-worker` (nvidia-l4, 8vCPU/32Gi, FUSE `khelsutra-rally-serving`?`/gcs,
`WASB_WEIGHTS=/gcs/models/?`).
162     - **CP Service** `rally-control-plane` rev **00004, edge-auth ON**, PRIVATE (IAM +
allUsers-invoker so CF can
163         reach it; app still requires the CF JWT). URL `https://rally-control-plane-2bosgeukpa-
as.a.run.app`.
164         Deployed image = `?/rally/rally-control-plane:latest` = rally-worker + google-cloud-
run + PyJWT + the

```

```

165      **OLD blanket #465 paths.py overlay** (the corrected input_root-scoped code is in PR
#465 ? the e2e
166      rebuild replaces it; harmless for the closed 4-email alpha meanwhile).
167      - **M9 LIVE:** `api.khelsutra.guru` ? CNAME `ghs.googlehosted.com` (PROXIED) via Cloud
Run domain-mapping,
168      fronted by the "KhelSutra Studio (alpha)" CF Access app (team
`https://khelsutra.cloudflareaccess.com`,
169      aud `c980da67?082b`, 4-email allowlist:
avi.dullu@sheendullu13@suneetadullu@avidullu.svp@).
170      **Add a tester** = add their email to that app's "alpha allowlist" policy (CF
dashboard or API; token at
171      `G:\My Drive\cloudflare.token.txt`, has DNS + Access-Apps edit but NOT Rulesets/Config
nor
172      `/access/organizations` ? team domain was recovered via the public certs probe). CF
acct
173      `213c12cf60de9041dd8a3521aa206472`, zone `65ef7770a6f25697843b53a093504184`. ? Google-
managed cert
174      auto-renew (~90d) can stall behind CF proxy ? switch to a CF Origin CA cert if it
does.
175
176      ## Gotchas (learned the hard way)
177      - Run **gcloud from PowerShell**, not git-bash (MSYS mangles `/gcs`, `/tmp`); quote
comma-dict flags
178      (`--add-volume "name=?,type=?"`) or the cmd-shim splits them.
179      - The CP uses per-instance **ephemeral SQLite + /tmp** ? DB/reel don't survive a
redploy or a second

```

27. user (2026-07-04T11:59:04.700Z)

```

100      - **Build command:** `npm run build`
101      - **Build output directory:** `web/dist`
102      - **Environment variable:** `VITE_API_BASE = https://api.khelsutra.guru/api` ? this
is what points the SPA at the engine tunnel (PR #64).
103      3. **Custom domain:** add `app.khelsutra.guru` to the Pages project.
104
105      > Every push to `master` now rebuilds + redeploys the SPA. To preview a change first,
Pages builds a per-PR preview URL.
106
107      ---
108
109      ## 4. Cloudflare Access (gate BOTH hostnames)
110
111      1. **Zero Trust ? Access ? Applications ? Add ? Self-hosted.**
112      2. **Application domain:** add **both** `app.khelsutra.guru` **and**
`api.khelsutra.guru` to the *same* application (one app ? one shared session cookie, so a tester
signs in once and both the SPA and its API calls are authorized).
113      3. **Policy:** Action *Allow*, rule *Emails* ? your alpha testers' addresses (free ? 50
users ? fits 10?25 testers). Use one-time-PIN (email OTP) so testers need no Cloudflare account.
114      4. **Copy the application `AUD` tag** ? paste into the engine's `config.json`
`security.cf_access_aud` ($2.2), and set `cf_team_domain` to `https://<your-
team>.cloudflareaccess.com`. Restart the engine.
115      5. **CORS preflight ?:** the browser sends an unauthenticated `OPTIONS` preflight to
`api.` for the cross-origin POSTs. In the Access application's **Settings ? enable ?Bypass
options requests to CORS preflight** (or add an explicit OPTIONS bypass), else the preflight is
redirected to the login page and every API call fails. The engine's CORS middleware
(`allow_credentials=false`, no cookies) then answers the preflight.
116
117      ---
118
119      ## 5. The tunnel (engine ? Cloudflare)
120
121      ```bash
122      cloudflared tunnel login                                # browser auth to your zone
123      cloudflared tunnel create khelsutra-studio              # note the <TUNNEL_UUID>
124      # copy deploy/cloudflared.config.example.yml ? /etc/cloudflared/config.yml, fill

```

```

<TUNNEL_UUID>
125     cloudflared tunnel route dns khelsutra-studio api.khelsutra.guru
126     cloudflared tunnel run khelsutra-studio           # or install as a service:
`cloudflared service install`
127     ```
128     See `deploy/cloudflared.config.example.yml` for the ingress (only `api.khelsutra.guru` ?
http://127.0.0.1:8000`).
129
130     ---
131
132     ## 6. Verify (the posture test + the CUJ)
133
134     **Exposure posture (must hold ? this is what makes it safe to share):**
135     - From OUTSIDE the box: `curl --max-time 5 http://<box-public-ip>:8000/api/health` ?
**refused / timeout** (the engine is `127.0.0.1`-bound; no inbound port). ? The box is not
directly reachable.
136     - `curl -i https://api.khelsutra.guru/api/health` **without** a CF session ? a **302 to
the Access login** (not a 200). ? The gate is in front.
137     - Open `https://app.khelsutra.guru` in a browser ? CF Access login ? after sign-in, the
SPA loads and `/api/health` returns 200 (same session cookie). ?
138
139     **The shareable CUJ (what a tester does):** open `app.khelsutra.guru` ? sign in ?
**upload a video from their device** (the SPA chunked-upload lands it in `upload_dir`, inside
the jail) ? **Process** ? watch progress ? pick rallies ? **Build** ? **download the reel**. All
from the page. *(Pasting a `gs://` URI does NOT work on this box ? see §7: the
`allowed_video_root` jail rejects every non-local URI. Browser upload is the alpha path.)*
140
141     ---
142
143     ## 7. Honest caveats / deferred
144
145     - **`upload_dir` must sit UNDER `allowed_video_root`.** If you enable browser upload, an
upload landing outside the jail is rejected by `/api/process` (a functional break, not a hole).
Keep `upload_dir` inside the jail (§2.2).
146     - **`gs://` / `s3://` / `gdrive://` URIs are REJECTED on the exposed box.** Setting
`allowed_video_root` (required to expose, §2.2) path-jails `/api/process` to *local* paths under
that root; the engine resolves a `gs://` URI to a local path that escapes the jail ? `400`
(pinned by engine `tests/test_api_endpoints.py::test_process_hosted_mode_rejects_nonlocal_uri`).
Cloud-URI ingest works **only** when `allowed_video_root` is unset (the local single-user
posture ? the opposite of this runbook). For alpha, the ingest path is **browser upload** (it
lands inside the jail). *(Scheme-exempting recognized buckets from the jail would be an engine
change, not a config flip.)*
147     - **You MUST set `edge_auth_enabled=true` to expose.** The boot posture (and the in-app
JWT verify) only engage then. Do not put the engine behind the tunnel with edge auth off.
148     - **Cross-origin reel download:** the reel streams from `api.` with `Content-
Disposition: attachment`, so it downloads even cross-origin ? but verify the filename in your
browser during the CUJ (cross-origin <a download> filename hints are ignored; the header is
the authority).
149     - **Upload size:** Cloudflare's free proxy caps request bodies at **100 MB** ? large
matches upload as ~95 MB chunks (the SPA chunked-upload path honours the engine's `max_part_mb`,
default **95** ; lower `security.max_part_mb` to shrink them). `gs://` bucket pre-staging is
**not** an option on this box (see the jail caveat above).
150     - **Box offline = engine offline.** The Pages SPA stays up and degrades gracefully ?
today that means the SetupWizard's offline branch (driven by its `/api/capabilities` fetch),
**not** a persistent "engine offline" banner; the SPA has no `/api/health` ping (unimplemented ?
corrected 2026-07-02). Processing resumes when the box is back. Beta removes this with a queue
(`HOSTING_PLAN.md` §3).
151     - **Gemini = paid + uploads footage to Google.** Keep the key rotated; use it only for
clips you're OK sending.
152
153     ---
154
155     ## 8. Teardown
156     `cloudflared tunnel cleanup khelsutra-studio` + delete the tunnel; remove the Pages
custom domain; disable the Access application; stop the engine service. The box's `output/` +

```

```
`uploads/` retain originals ? sweep them per your retention policy before any wider exposure.
```

```
157
```

```
158     ---
```

```
159     ### Changelog
```

28. user (2026-07-04T11:59:25.839Z)

```
20         court_green_upper: list[int] = Field(default=[85, 255, 255])
```

```
21         court_pixels_min_ratio: float = Field(default=0.05, ge=0.0, le=1.0)
```

```
22
```

```
23
```

```
24     class ValidationConfig(BaseModel):
```

```
25         min_resolution_width: int = 1280
```

```
26         min_resolution_height: int = 720
```

```
27         min_fps: float = 29.9
```

```
28         min_blur_threshold: float = 20.0
```

```
29         max_scene_cuts_per_minute: float = 0.5
```

```
30         pts_max_gap_seconds: float = Field(
```

```
31             default=10.0,
```

29. user (2026-07-04T11:59:26.791Z)

```
360         # mode). Default None preserves the single-user local 'any local file' workflow.
```

```
361         allowed_video_root: Optional[str] = None
```

```
362
```

```
363         # --- Chunked upload (B2, PR-2) ---
```

```
364         # Landing zone for browser uploads (assembled from parts). Per-user subdirectories
```

```
365         # arrive with PR-3 identity scoping. ? In hosted mode, set allowed_video_root to
```

```
366         # contain upload_dir or /api/process will reject the uploaded paths.
```

```
367         upload_dir: str = "output/uploads"
```

```
368         # Whole-file cap (default 4 GB) and per-part cap. Parts stay under ~100 MB because
```

```
369         # free-tier edge proxies cap request bodies there (HOSTING_PLAN §2).
```

```
370         max_upload_mb: int = 4096
```

```
371         max_part_mb: int = 95
```

```
372         allowed_upload_extensions: List[str] = ["mp4", "mov"]
```

```
373
```

```
374         # --- M9 edge auth (Cloudflare Access, D9) ---
```

```
375         # DEFAULT-OFF: edge_auth_enabled=False ? no JWT is required/verified (the local
```

```
single-user
```

```
376         # path is unchanged). When True, /api/process requires + verifies the `Cf-Access-
```

```
Jwt-Assertion`
```

```
377         # header against the team JWKS (so a direct hit to the Cloud Run URL can't spoof the
```

```
identity
```

```
378         # header) and tags the job with the verified user_id (owner_id). The team domain
```

```
(e.g.
```

```
379         # ``https://<team>.cloudflareaccess.com``) supplies the JWKS + issuer; the AUD is
```

```
the CF Access
```

```
380         # application's audience tag. NO secrets here ? these are public identifiers.
```

```
381         edge_auth_enabled: bool = False
```

```
382         cf_team_domain: str = (
```

```
383             "" # e.g. https://<team>.cloudflareaccess.com (issuer + /certs JWKS)
```

```
384         )
```

30. user (2026-07-04T11:59:27.429Z)

```
1         """"Cache-hygiene + retention sweep for the GPU highlight pipeline.
```

```
2
```

```
3         Scans the regenerable intermediates the pipeline leaves behind ? in the WSL stage
```

```
4         dir (`~/clips`), the Windows `output/` tree, and the upload landing zone
```

```
5         (`security.upload_dir`, default `output/uploads`) ? classifies each entry as
```

```
6         **purge** (regenerable: frame caches, staged video copies, handoff clips, temp dirs;
```

```
7         and, past a retention window, uploaded source videos) or **keep** (durable: trajectory
```

```
8         CSVs, the SQLite DB, final reels) and reports the reclaimable space.
```

```
9
```

```

10      **Retention sweep (T11 ? pre-alpha-exposure):** uploaded source videos in the upload
11      dir are a *privacy*-driven purge, not a *disk*-driven one ? once a tester's footage is
12      older than ``--uploads-retention-days`` (default 7) it is swept, along with the orphaned
13      ``.partial-<id>`` temp files aborted uploads leave behind. Reels / DB / CSVs stay
durable
14      per the cache-hygiene policy (the owner's call, 2026-06-21).
15
16      DRY-RUN BY DEFAULT ? it only *shows* what it would delete. Pass ``--apply`` to delete.
17
18      python -m tools.cleanup_caches                                # report only (safe)
19      python -m tools.cleanup_caches --apply                        # delete purge candidates
20      python -m tools.cleanup_caches --keep-video 89605e80ed01     # protect one video's
cache

```

31. user (2026-07-04T11:59:39.193Z)

C:\Users\avidu\Projects\khelsutra-guru\khelsutra\web\src\App.tsx:59:/** Annotate-in-UI (P8) is behind a runtime feature flag ? default OFF (D13: ships gated until consented
C:\Users\avidu\Projects\khelsutra-guru\khelsutra\web\src\components\IngestPanel.tsx:87: // don't dispatch a paid job without consent).

32. user (2026-07-04T11:59:40.588Z)

```

538:- **New-project GPU gate:** `GPUS_ALL_REGIONS` quota = 0 must be raised to ?1 (owner console
588: (native==WSL 1194/1195; deterministic) ? see PR #259. **Spot does NOT bypass
`GPUS_ALL_REGIONS`**
589: (tested) ? both on-demand + Spot need the quota; a `GPUS_ALL_REGIONS?1` increase was filed
via the
675:- **GCP works today:** quota granted (`GPUS_ALL_REGIONS=1`), bucket + SA + `$100` budget
provisioned, and M1/M2/M3.5 ran on an L4 for ~$1?2/run with no subscription. Real money, but
trivial per run ? and **one** portable PyTorch+CUDA stack (local ? GPU VM ? Cloud Run), no
second provider to maintain.

```

33. user (2026-07-04T11:59:41.015Z)

```

340      className={"min-h-11 rounded-full px-4 " + (view === "annotate" ? "bg-ink
text-white" : "text-ink/70")}
341      >
342      {t("annotate.tab")}
343    </button>
344  </div>
345  )}
346
347  {/* P2c first-run wizard: before anything is loaded, walk a new user through AI-
key / compute /
348  videos (from /api/capabilities). Shows once per browser; then the ingest
panel takes over. */}
349  {!videoId && !wizardDone && <SetupWizard onClose={() => setWizardDone(true)} />}
350
351  {/* Front-half (P6 + B-P2): when no video is loaded, offer BOTH ? reopen a
stored video from
352  "Your videos" (VideoLibrary ? GET /api/videos), or add a new one
(IngestPanel ? process ?
353  load). Once a video_id exists, the existing back-half (pick ? build) takes
over. */}
354  {!videoId && wizardDone && (

```

34. user (2026-07-04T11:59:53.702Z)

```

284:      staged_uri = f"{input_root.rstrip('/')}/{video_id}/{os.path.basename(local_src)}"
305:      f"cloud-serving: failed to stage the uploaded file to {input_root}: {e}",

```

35. user (2026-07-04T11:59:55.202Z)


```

C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\orchestration.py:634:
"[COMPUTE] video_id=%s requested=%s actual=%s dispatched=%s status=%s",
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\orchestration.py:766:def signed_reel_url(cfg, video_id: str, name: str) ->
Optional[str]:
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\orchestration.py:856:
logger.warning("[compile] could not mirror reel to %s: %s", reel_uri, e)
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\main.py:241:
signed_reel_url,
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\main.py:1056:      #
exists/sign logic (with the active storage config) lives in orchestration.signed_reel_url to
keep
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\main.py:1059:
signed = signed_reel_url(request.app.state.config, video_id, name)
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\worker\__main__.py:188:      logger.info("[worker] wrote completion marker ->
%s", done_uri)

```

36. user (2026-07-04T12:00:03.189Z)

```

744         )
745         return video, kept, out_name
746
747
748     def reel_object_uri(cfg, video_id: str, name: str) -> str:
749         """The bucket URI a compiled reel is mirrored to for a direct signed-URL download
750         (#463), or "" when
751         the output isn't a cloud bucket (truly-local ? serve the local file). Shared by the
752         compile
753         mirror-upload and the /api/highlights redirect:
754         ``<output_root>/highlights/<video_id>/<name>``. """
755         root = (getattr(getattr(cfg, "storage", None), "output_root", "") or "").strip()
756         if not root or "://" not in root:
757             return ""
758         return f"{root.rstrip('/')}/highlights/{video_id}/{name}"
759
760
761     def _storage_settings(cfg) -> Optional[Dict[str, Any]]:
762         """The active storage settings dict (per-backend config for the storage layer ? S3
763         endpoint/region,
764         GCS project/client/signed_url_ttl), or None. Threaded into storage calls so a non-
765         default backend
766         isn't silently constructed as a fresh default. """
767         sc = getattr(cfg, "storage", None)
768         return sc.model_dump() if sc is not None else None
769
770
771     def signed_reel_url(cfg, video_id: str, name: str) -> Optional[str]:
772         """A short-lived signed URL for a reel mirrored to the output bucket, so
773         /api/highlights can
774         307-redirect off the app's response path (bypassing Cloud Run's ~32 MiB cap + the
775         ephemeral local
776         /tmp; #463). None ? no bucket copy ? serve the local file. Uses the active storage
777         config for BOTH
778         the existence check and the signing (extracted here to keep backend/main.py lean ?
779         P23/P24). """
780         reel_uri = reel_object_uri(cfg, video_id, name)
781         if not reel_uri:
782             return None
783         settings = _storage_settings(cfg)
784         if storage.exists(reel_uri, config=settings):
785             return storage.signed_url(reel_uri, config=settings, method="GET")
786         return None
787
788
789

```

```

780     def _execute_compile(cfg, db, job: Dict[str, Any], progress=None) -> Dict[str, Any]:
781         """Worker handler for a ``kind='compile'`` job (#329): re-resolve the segments,
stitch the reel,
782         and return the {status, filepath, download_url} result the SPA polls for. Returns a
783         ``status='failed'`` result for an EXPECTED problem (missing video / no segments / a
stitch error)
784         so the SPA always gets a clean verdict instead of a worker-crash error."""
785         import backend.main as _main # call-time seam: VideoCompiler stays patchable on
backend.main
786
787         notify = progress or (lambda stage: None)
788         params = json.loads(job.get("params") or "{}")
789         video_id = job.get("video_id") or ""
790         segment_ids = params.get("segment_ids") or []
791         locale = params.get("locale")
792
793         notify("resolving")

```

37. user (2026-07-04T12:00:14.826Z)

```

14     to Bangalore ? lowest latency).
15
16     > **Cost discipline (read first):** a running GPU VM is the only meaningful cost. Use
the cheapest
17     > adequate GPU (**T4**), **STOP or DELETE the VM the moment a run finishes** (a stopped
VM bills only
18     > for its disk), and keep the **$100 budget alert** live. `teardown.sh` switches
everything off.
19
20     ---
21

```

38. user (2026-07-04T12:00:15.776Z)

```

105     updated baseline. The runner reports added/removed clips as **STALE** (exit 4) so a new
clip can't silently
106     pass un-baselined.
107
108     ## Observe ? where results go
109     - **Email** to `NIGHTLY_EMAIL_TO` every run (subject `[KhelSutra nightly]
PASS|REGRESSION|STALE ?`).
110     - **Reports in GCS**: `gs://khelesutra-rally-nightly/reports/<date>/<date>.{md,json}`
(kept 30 days).
111     - **Launcher logs**: `gcloud run jobs executions list --job=nightly-regression-launcher
--region=asia-south1` then `? executions logs read <exec> --region=asia-south1`.
112     - **The GPU run**: `gcloud compute instances get-serial-port-output <vm> --zone=<zone>`
(while it lives) or its Cloud Logging.
113     - **Exit codes**: `0` pass · `1` regression (reel-count changed / a clip errored /
quality dropped) · `2` operational · `3` no baseline · `4` stale (corpus/segmenter changed ? re-
freeze).
114
115     ## Pause / disable / resume / remove
116     ```bash

```

39. user (2026-07-04T12:00:16.543Z)

```

c:\Users\avidu\.claude\projects\c--Users-avidu-Projects-khelesutra-guru\memory\working-agreement-
attribution.md
c:\Users\avidu\.claude\projects\c--Users-avidu-Projects-khelesutra-guru\memory\candidate-
segments-finetuning.md
c:\Users\avidu\.claude\projects\c--Users-avidu-Projects-khelesutra-guru\memory\monetization-
plan.md
c:\Users\avidu\.claude\projects\c--Users-avidu-Projects-khelesutra-guru\memory\monetization-
research-status.md

```

c:\Users\avidu\claud\projects\c--Users-avidu-Projects-khelsutra-guru\memory\collector-indexer-bridge.md
c:\Users\avidu\claud\projects\c--Users-avidu-Projects-khelsutra-guru\memory\golden-set-vendoring.md
c:\Users\avidu\claud\projects\c--Users-avidu-Projects-khelsutra-guru\memory\tracknet-wasb-wsl2-decision.md
c:\Users\avidu\claud\projects\c--Users-avidu-Projects-khelsutra-guru\memory\audio-e1-findings.md
c:\Users\avidu\claud\projects\c--Users-avidu-Projects-khelsutra-guru\memory\code-map-artifact.md
c:\Users\avidu\claud\projects\c--Users-avidu-Projects-khelsutra-guru\memory\roora-guidelines-drive-doc.md
c:\Users\avidu\claud\projects\c--Users-avidu-Projects-khelsutra-guru\memory\sibling-internship-repo.md
c:\Users\avidu\claud\projects\c--Users-avidu-Projects-khelsutra-guru\memory\working-agreement-merging.md
c:\Users\avidu\claud\projects\c--Users-avidu-Projects-khelsutra-guru\memory\gotcha-long-runs-gpu-wsl.md
c:\Users\avidu\claud\projects\c--Users-avidu-Projects-khelsutra-guru\memory\tool-trim-video.md
c:\Users\avidu\claud\projects\c--Users-avidu-Projects-khelsutra-guru\memory\paper-coauthor-aim.md
c:\Users\avidu\claud\projects\c--Users-avidu-Projects-khelsutra-guru\memory\gotcha-shared-checkout-branch-collisions.md
c:\Users\avidu\claud\projects\c--Users-avidu-Projects-khelsutra-guru\memory\file-placement-conventions.md
c:\Users\avidu\claud\projects\c--Users-avidu-Projects-khelsutra-guru\memory\weak-signals-retained.md
c:\Users\avidu\claud\projects\c--Users-avidu-Projects-khelsutra-guru\memory\decision-rigor-norm.md
c:\Users\avidu\claud\projects\c--Users-avidu-Projects-khelsutra-guru\memory\working-agreement-tracked-project-docs.md
c:\Users\avidu\claud\projects\c--Users-avidu-Projects-khelsutra-guru\memory\eval-boundary-tolerance.md
c:\Users\avidu\claud\projects\c--Users-avidu-Projects-khelsutra-guru\memory\launch-report-convention.md
c:\Users\avidu\claud\projects\c--Users-avidu-Projects-khelsutra-guru\memory\eval-dual-set-regression.md
c:\Users\avidu\claud\projects\c--Users-avidu-Projects-khelsutra-guru\memory\current-state-archive.md
c:\Users\avidu\claud\projects\c--Users-avidu-Projects-khelsutra-guru\memory\working-agreement-checkpoints.md
c:\Users\avidu\claud\projects\c--Users-avidu-Projects-khelsutra-guru\memory\feedback-launch-quality-bar.md
c:\Users\avidu\claud\projects\c--Users-avidu-Projects-khelsutra-guru\memory\khelsutra-domain-launch.md
c:\Users\avidu\claud\projects\c--Users-avidu-Projects-khelsutra-guru\memory\gcp-ops-agent-enable.md
c:\Users\avidu\claud\projects\c--Users-avidu-Projects-khelsutra-guru\memory\gcp-vm-always-delete.md
c:\Users\avidu\claud\projects\c--Users-avidu-Projects-khelsutra-guru\memory\machine-pooling-idea.md
c:\Users\avidu\claud\projects\c--Users-avidu-Projects-khelsutra-guru\memory\doc-status-register.md
c:\Users\avidu\claud\projects\c--Users-avidu-Projects-khelsutra-guru\memory\lesson-verify-engine-surface-before-scoping.md
c:\Users\avidu\claud\projects\c--Users-avidu-Projects-khelsutra-guru\memory\cache-hygiene-policy.md
c:\Users\avidu\claud\projects\c--Users-avidu-Projects-khelsutra-guru\memory\next-session-ux-localization.md
c:\Users\avidu\claud\projects\c--Users-avidu-Projects-khelsutra-guru\memory\packaging-app-plan.md
c:\Users\avidu\claud\projects\c--Users-avidu-Projects-khelsutra-guru\memory\next-session-packaging-app.md
c:\Users\avidu\claud\projects\c--Users-avidu-Projects-khelsutra-guru\memory\next-session-multishuttle-rally.md

c:\Users\avidu\.claude\projects\c--Users-avidu-Projects-khelsutra-guru\memory\gcs-data-source-of-truth.md
c:\Users\avidu\.claude\projects\c--Users-avidu-Projects-khelsutra-guru\memory\next-session-golden-fixtures.md
c:\Users\avidu\.claude\projects\c--Users-avidu-Projects-khelsutra-guru\memory\golden-corpus-grew-9-to-15.md
c:\Users\avidu\.claude\projects\c--Users-avidu-Projects-khelsutra-guru\memory\cloudrun-gpu-serving-gotchas.md
c:\Users\avidu\.claude\projects\c--Users-avidu-Projects-khelsutra-guru\memory\ui-driven-cloud-serving-gap.md
c:\Users\avidu\.claude\projects\c--Users-avidu-Projects-khelsutra-guru\memory\compute-stack-gcp-only.md
c:\Users\avidu\.claude\projects\c--Users-avidu-Projects-khelsutra-guru\memory\khelsutra-site-droplet-parity.md
c:\Users\avidu\.claude\projects\c--Users-avidu-Projects-khelsutra-guru\memory\review-trust-ci-over-rerun.md
c:\Users\avidu\.claude\projects\c--Users-avidu-Projects-khelsutra-guru\memory\review-signoff-policy.md
c:\Users\avidu\.claude\projects\c--Users-avidu-Projects-khelsutra-guru\memory\decision-snapshot-regression.md
c:\Users\avidu\.claude\projects\c--Users-avidu-Projects-khelsutra-guru\memory\next-session-rally-finding-b-promotion.md
c:\Users\avidu\.claude\projects\c--Users-avidu-Projects-khelsutra-guru\memory\lesson-branch-rename-closes-open-pr.md
c:\Users\avidu\.claude\projects\c--Users-avidu-Projects-khelsutra-guru\memory\working-root-khelsutra-guru.md
c:\Users\avidu\.claude\projects\c--Users-avidu-Projects-khelsutra-guru\memory\storage-bucket-config-requirements.md
c:\Users\avidu\.claude\projects\c--Users-avidu-Projects-khelsutra-guru\memory\muneeem-ai-workspace.md
c:\Users\avidu\.claude\projects\c--Users-avidu-Projects-khelsutra-guru\memory\project-studio-media-lifecycle.md
c:\Users\avidu\.claude\projects\c--Users-avidu-Projects-khelsutra-guru\memory\muneeem-grocery-replenishment.md
c:\Users\avidu\.claude\projects\c--Users-avidu-Projects-khelsutra-guru\memory\feedback-tail-long-job-logs.md
c:\Users\avidu\.claude\projects\c--Users-avidu-Projects-khelsutra-guru\memory\wasb-l4-perf-cpu-feed-bound.md
c:\Users\avidu\.claude\projects\c--Users-avidu-Projects-khelsutra-guru\memory\session-handoff.md
c:\Users\avidu\.claude\projects\c--Users-avidu-Projects-khelsutra-guru\memory\current-state.md
c:\Users\avidu\.claude\projects\c--Users-avidu-Projects-khelsutra-guru\memory\MEMORY.md

40. user (2026-07-04T12:03:08.981Z)

Structured output provided successfully